

example-network-topology — full documentation

- Example Network Topology
- Topology
- Consolidated services server — minimum specification
- IP address map
- Streaming flow — SRT / NDI
- Why BirdDog Play (not a laptop + VLC) for theatre playback
- Why GL-iNet — segmentation, Tailscale overlay, multi-WAN, and beyond this event
- Live editing — post-production at the mothership
- File sync flow
- ATEM ISO ingest — near-real-time, Nextcloud-aware
- VMix record ingest — near-real-time, Nextcloud-aware
- GLKVM-Cloud — remote administration for the GL-iNet router fleet
- Bandwidth analysis — venue-supplied VLANs
- Tailscale — DERP relay avoidance
- Open questions
- Alternative: plain switches + Tailscale on generic Linux, no GL-iNet routers
- Deployment runbook
- Troubleshooting

Example Network Topology

AI-assisted project. This design and its documents were produced with Claude (Anthropic), directed and reviewed by a human author. It is a design, not a deployment: the throughput figures are real and cited and the configs are generated and ready to apply, but **nothing here has been built**. See `docs/open-questions.md` for what still needs real-world input.

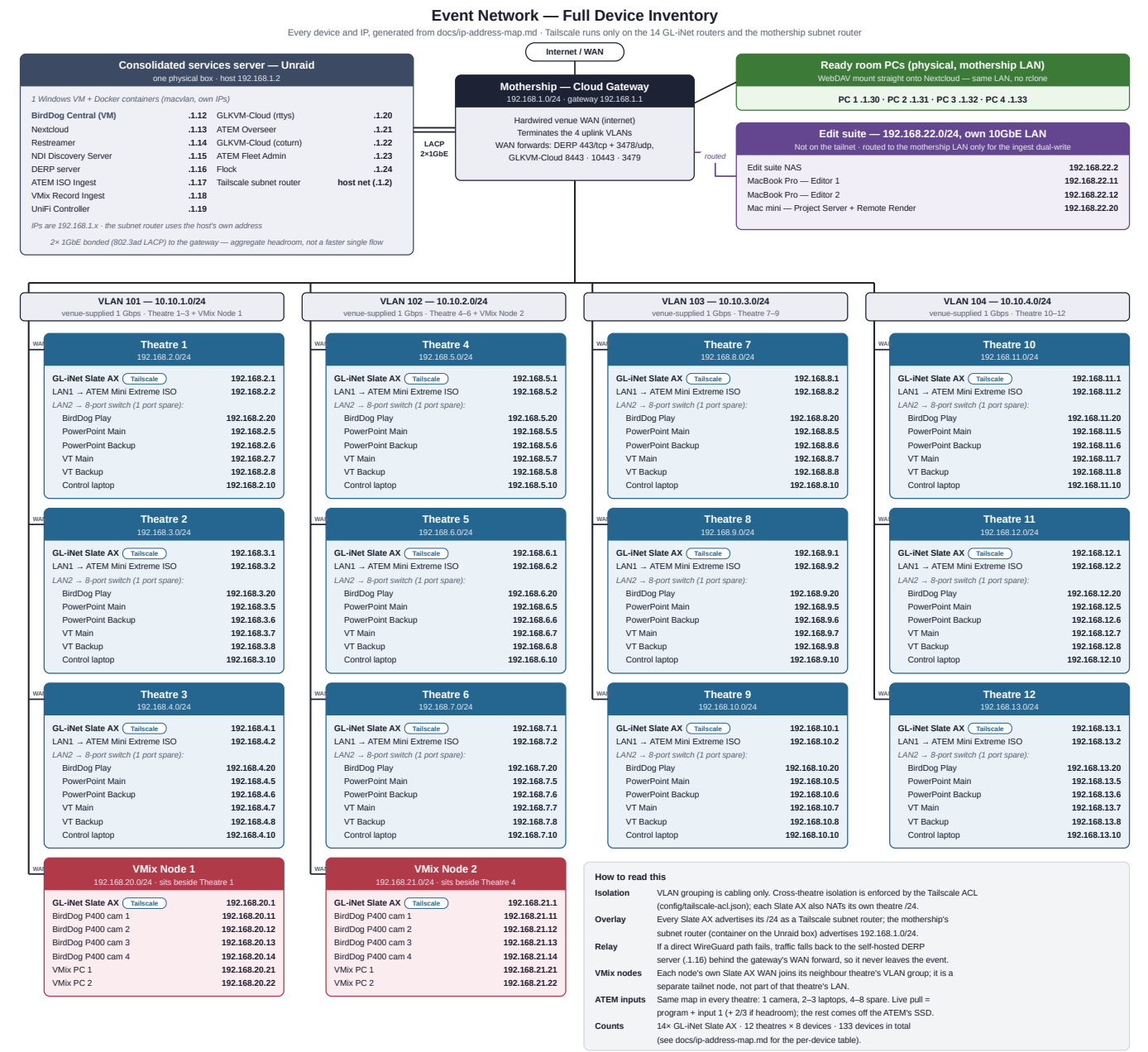
A complete, fully-worked network design for a 12-theatre live event: every theatre streams its program feed and syncs its files back to a central "mothership," all riding over a single Tailscale mesh, with no reliance on venue Wi-Fi or a conference-provided network.

This is an anonymized reference design, published as a worked example of the whole process — topology, bandwidth modelling, hardware selection with rationale, generated configs, deployment runbook. It's a real design for a real event, with the identifying details generalized: all domains are placeholders on the IANA-reserved `example.net` (substitute your own), and nothing here names the event, client, or venue. Product names, prices, and throughput figures are real and cited.

Status: design complete, nothing built yet. Every device has a real IP and the Tailscale/GL-iNet/UniFi configs are generated. A review on 2026-10-06 found the ATEM ISO bitrate and the original router's Tailscale throughput were both far off, and the design changed to match: only the program recording and the camera ISO are pulled live (every theatre wired to one common ATEM input map), and the routers are now GL.iNet **Slate AX** units. Two things still want a measurement on real hardware: one Slate AX's Tailscale throughput, and one real laptop-input ISO file. See `docs/open-questions.md` for what still needs real-world input before this goes from design to build, or `docs/deployment-runbook.md` for the full ordered build sequence.

The shape of it

12 theatres, each its own isolated subnet, connect back to one mothership over Tailscale. Physical uplink cabling is grouped 3-theatres-per-VLAN purely for cable management — the VLANs don't provide isolation themselves, **Tailscale ACLs do**: every theatre can reach the mothership, but never another theatre, and never a VMix node. Two extra "VMix nodes" hang off Theatre 1 and Theatre 4 respectively, each with its own cameras, its own router, and its own place on the tailnet.



Full network topology, showing the mothership, all 12 theatres with every internal device and IP, and both VMix nodes

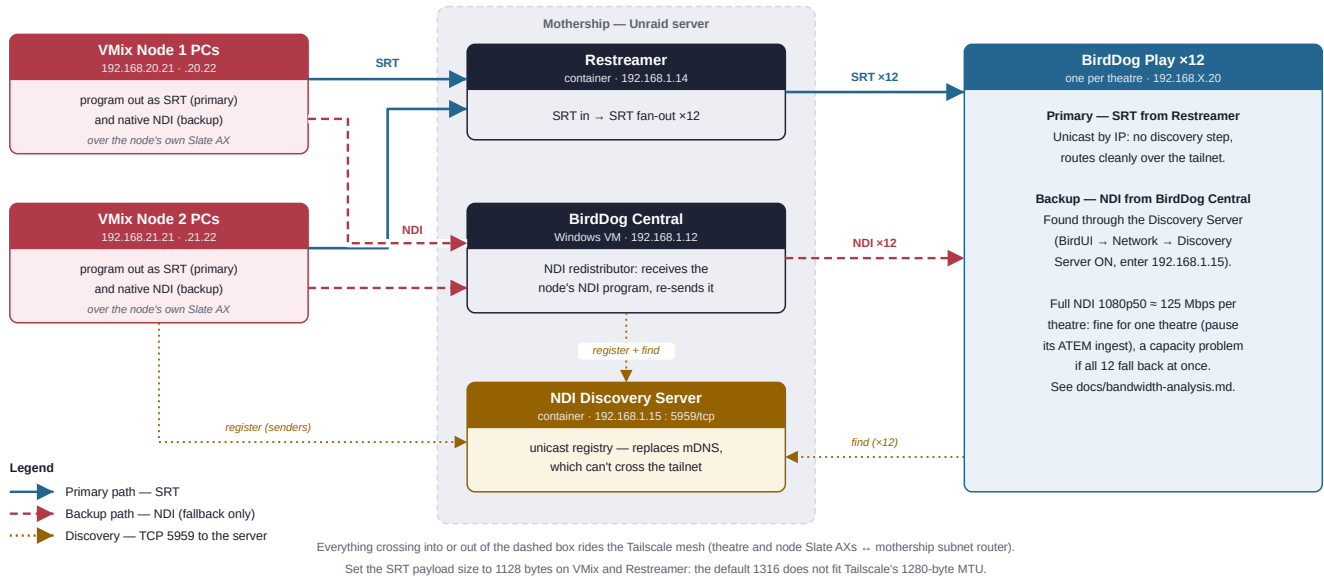
Every box in that diagram is a real device with a real address — 133 of them in total, counting the live-editing suite's own four. docs/ip-address-map.md has the same inventory as a plain text table, if that's easier to search or paste from.

Streaming: SRT primary, NDI backup

Each theatre's **BirdDog Play** receives the event's program feed two ways:

SRT / NDI Streaming Flow

Primary: SRT through Restreamer · Backup: NDI redistributed by BirdDog Central · Discovery: unicast NDI Discovery Server, because Tailscale does not carry mDNS



SRT/NDI streaming flow — VMix PCs to Restreamer to BirdDog Play (primary, solid), BirdDog Central through the NDI Discovery Server to BirdDog Play (backup, dashed)

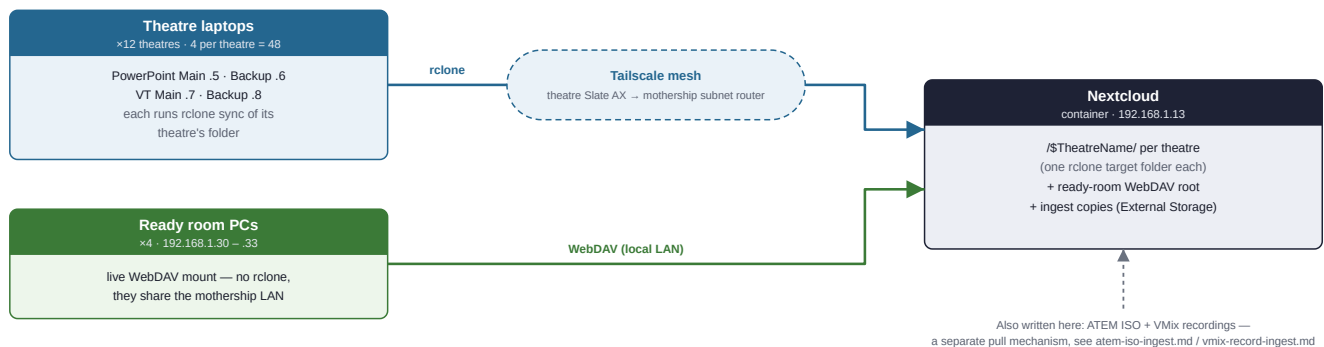
- **Primary — SRT**, unicast by IP from the mothership's Restreamer straight to each theatre's BirdDog Play. No discovery mechanism needed, so it's the more robust path — and it can originate directly from the ATEM's own built-in hardware streaming engine or from VMix, no computer required either way.
- **Backup — NDI**, from BirdDog Central. NDI normally discovers sources over local mDNS multicast, which Tailscale can't carry between theatres — so both ends instead register with a self-hosted **NDI Discovery Server**, a unicast-TCP service built into NDI 5+ specifically for this WAN/VPN case. Confirmed working on BirdDog Play's own firmware (BirdUI's Network panel has a Discovery Server field built in).

Why BirdDog Play at all, rather than a laptop running VLC? Zero on-site config, native dual-protocol support, broadcast-grade unattended reliability, and central management from the mothership — Flock (a purpose-built fleet manager for exactly this device) is the fleet-management tool, with per-unit BirdUI as the fallback. BirdDog Central is retained too, but for a different job: it's the sender side of the NDI backup path above, not a management requirement. Full comparison in `docs/birddog-play-rationale.md`.

File sync: rclone up, WebDAV local

File Sync Flow

Documents into Nextcloud: theatre laptops over the tailnet, ready-room PCs on the local LAN



File sync flow — theatre laptops rclone to Nextcloud over Tailscale, ready room PCs mount Nextcloud via local WebDAV

Theatre laptops (PowerPoint and VT, main and backup — 48 across all 12 theatres) each run `rcclone`, syncing their theatre's folder up to Nextcloud over the tailnet. The mothership's 4 ready-room PCs skip `rcclone` entirely and mount Nextcloud directly over local WebDAV, since they already share its LAN.

ATEM ISO ingest: near-real-time, no manual chunking

Each theatre's ATEM Mini Extreme ISO records 9 H.264 streams (8 input ISOs + program) to a USB SSD — always all of them, and the ISO files at a fixed ~35-70 Mb/s each that can't be turned down. So the network only carries part of it: **the program recording (set to ~8-10 Mbps) plus the camera ISO**, pulled live; the presenter-laptop inputs are added per theatre where there's headroom, and everything else comes off the SSD by physical offload. Every theatre is wired to the same input map (1 camera, 2-3 laptops, 4-8 spare), so one setting (`ATEM_ISO_INPUTS`) picks the same files everywhere. There's no laptop involved in the pull: the mothership reads directly from the ATEM's **built-in FTP server** over Tailscale subnet routing, transferring only the bytes appended since the last check. That matters because the files are *constantly growing*: a naive whole-file re-sync on a schedule falls permanently behind on a multi-hour recording. Two documented ways to get that incremental transfer: FTP's own `REST /resume` command directly (the default — plain `rsync` can't be used for this leg, since it needs SSH or an `rsync` daemon and the ATEM only speaks FTP), or `rcclone mount` (presenting the ATEM's FTP share as a local path) paired with `rsync --append`, which transfers only the new tail without needing `rsync`'s usual full-file checksum pass. Either way, the pull writes straight into the folder Nextcloud has mounted as External Storage, so there's no second upload step and no need to slice the video file itself — full design in `docs/atem-iso-ingest.md`.

VMix record ingest: same mechanism, a Windows SMB share instead

The 4 VMix PCs (2 per node) get the same treatment, just retargeted at a Windows SMB share instead of an FTP server — SMB is a genuine network filesystem protocol, so mounting it is the native way to access it here, if anything a better fit than the ATEM's FTP-only case rather than a stretch of the same pattern. `rcclone mount` presents each PC's share as a local path, and `rsync --append` transfers only the new tail, same reasoning as the ATEM ingest's alternative approach. Each VMix node has its own dedicated Slate AX uplink — not shared with the theatre it physically sits near — so this traffic never competes with that theatre's ATEM ingest or SRT feed. Full design in `docs/vmix-record-ingest.md`.

Self-hosted admin tooling: UniFi Controller + GLKVM-Cloud

Two self-hosted services on the consolidated server, both following the same rule the rest of this design does — no dependency on a vendor's own cloud for managing this design's own infrastructure:

- **UniFi Controller** (self-hosted UniFi Network Application) gives a local, independent management console for the Cloud Gateway, separate from relying on `cloud.ui.com` — the Cloud Gateway ships with its own built-in controller and works fine without this, so it's optional, but not depending on a vendor cloud to administer the mothership's own router fits the pattern everything else here follows.
- **GLKVM-Cloud** gives centralized, browser-based SSH terminal and web-admin proxy access to all 14 GL-iNet routers at once, instead of 14 separate sessions — using GLKVM-Cloud's documented support for embedded OpenWrt devices, not its flagship KVM-hardware feature set (there's no physical KVM unit anywhere in this design). Full design, including a real WAN-port-forward collision this surfaced with the DERP server and how it was resolved, in `docs/glkvmm-cloud.md`.

Fleet management: ATEM Overseer + Fleet Admin, and Flock

Three more mothership services, this time for the theatre hardware itself rather than the network infrastructure — all separate projects (`atem-overseer`, `atem-fleet-admin`, `flock`), not built from source in this repo:

- **ATEM Overseer** is a fleet-wide monitoring/tally dashboard — every theatre's ATEM sends it a dedicated low-bitrate monitoring stream (10 Mbps, folded into the bandwidth model below), giving one central multiview view of all 12

theatres instead of walking to each one.

- **ATEM Fleet Admin** provisions many ATEMs at once from model-aware config forms, either pushed live over the network (reaching all 12 theatres the same way ATEM ISO Ingest does — Tailscale subnet routing, no separate path needed) or exported as loadable XML + a media folder.
- **Flock** does the same fleet-manager job as ATEM Fleet Admin, but for the 12 theatres' BirdDog Play units — LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits. Each BirdDog Play also sends Flock its own 10 Mbps SRT preview stream, same treatment as Overseer's — also folded into the bandwidth model below.

Bandwidth: real numbers, right-sized VLANs

The 4 uplink VLANs turned out to be **venue-supplied, not our infrastructure** — 1 Gbps each, with expensive ports, which flips the goal from "isolate cleanly" to "minimum VLAN count at adequate performance." The full model, grounded in real figures (SRT ~5.8-10.4 Mbps via long-GOP H.264 + SRT/WireGuard overhead, ATEM ingest ~47-83 Mbps for program + camera, plus the ATEM Overseer and Flock monitoring/preview streams — ~10.4 Mbps apiece — together dominating per-theatre upstream, full NDI at 1080p50 a very different ~130 Mbps roughly regardless of content), is in `docs/bandwidth-analysis.md` . Headlines:

- **ATEM ingest is scoped to what fits.** ISO files can't be set below ~35-70 Mb/s each, so pulling every input live would need 175-405 Mbps per theatre. The default live scope is program + camera: ~83 Mbps typical upstream per theatre (73-109 range), with each presenter-laptop input adding ~25 Mbps (an estimate until a real file is measured).
- **4 VLANs** run at ~25-33% upstream at the default scope, and stay comfortable even with both laptop inputs on in every theatre.
- **2 VLANs (6 theatres each)** are viable at the default scope (~50-65%) or with one laptop input (~65-90%), provided a mass NDI fallback (e.g. a Restreamer failure pushing all 12 theatres onto NDI at once, 90% on 2 VLANs) is handled operationally. Not with both laptop inputs everywhere (~81% typical, over 100% high).
- **1 VLAN** doesn't work with any live ingest (~100-131%).
- **The per-router ceiling decided the router.** The original GL.iNet A-1300's 170 Mbps is a kernel-WireGuard figure; Tailscale does its encryption in software, and on that router's 32-bit chip comparable devices manage an eighth to a half of their WireGuard rate, putting it at an estimated ~30-70 Mbps. Its replacement, the **Slate AX** (64-bit, 550 Mbps WireGuard), is estimated at ~150-250 Mbps under Tailscale, enough for the default scope plus a single-theatre NDI fallback with that theatre's ingest paused. Both Tailscale figures are estimates: benchmark one Slate AX before buying 14.

Why Tailscale avoids DERP relay here

Every device in this design — all 12 theatres, both VMix nodes, the mothership — sits behind the *same* shared venue internet connection. That's an unusually favorable setup for Tailscale: direct connections are likely either over the true private LAN path or via a NAT hairpin through that one shared public IP, with public DERP relay only as a last resort. As extra insurance, a self-hosted DERP server (reachable at `derp.example.net:443`) runs on the mothership itself, so even relayed traffic never actually leaves the venue. Full reasoning in `docs/tailscale.md` .

Hardware, confirmed

Role	Hardware	Why
Mothership router	Ubiquiti Cloud Gateway	Can't run Tailscale natively — that role moves to a container on the services server instead
Consolidated services server	Unraid , single physical box (already on hand)	Hosts BirdDog Central as the design's only Windows VM (no GPU passthrough anywhere — the mothership VMix VM was removed, its role never established; theatre program feeds originate from the VMix node PCs), plus Nextcloud/Restreamer/NDI Discovery Server/DERP/ATEM ISO Ingest/VMix Record Ingest/UniFi Controller/GLKVM-Cloud/ATEM Overseer/ATEM Fleet Admin/Flock as Docker containers. Chosen over TrueNAS SCALE for its polished container UX (Community Applications). Calculated target spec (storage pools, RAM, CPU, NIC validation — no GPU required) in <code>docs/server-specification.md</code>

Role	Hardware	Why
Theatre + VMix node routers	GL.iNet Slate AX (GL-AXT1800) ×14	12 theatre routers + both VMix node routers, same model throughout. 64-bit quad-core, 550 Mbps WireGuard (vendor, kernel WireGuard), estimated ~150-250 Mbps under Tailscale — to be benchmarked on one unit before buying 14. Same 1 WAN + 2 LAN layout as the A-1300 it replaces, so the ATEM Mini Extreme ISO still sits on its own dedicated port, away from the rest of the room's traffic. The A-1300 was dropped on 2026-10-06: its 170 Mbps is a kernel-WireGuard figure, and Tailscale on its 32-bit chip is estimated at only ~30-70 Mbps. During an NDI fallback, pause that theatre's ATEM ingest; see docs/bandwidth-analysis.md . Full case for GL-iNet + Tailscale specifically — segmentation, multi-WAN, pay-per-device WiFi economics, other uses beyond this event, and the router comparison — in docs/gl-inet-rationale.md
Theatre playback	BirdDog Play ×12	Zero-config, native SRT+NDI, centrally fleet-managed — see above
Server NICs	Bonded , 802.3ad/LACP	SRT bandwidth isn't constant — LACP hashes flows across both links for aggregate headroom. On the wire the tailnet traffic is one WireGuard flow per router (~14), so the spread is coarser than the application-level flow count suggests, and no single flow gets faster

See docs/open-questions.md for the reasoning behind every one of these, including the couple of things assumed rather than confirmed (and the fallback if an assumption turns out wrong).

Live editing: a separate subsystem, not the theatre network

2× MacBook Pro editing workstations plus dedicated fast storage at the mothership, editing event content as it arrives rather than waiting until after the event — deliberately on its **own dedicated 10GbE LAN** (192.168.22.x), not part of the Tailscale mesh or the theatre-facing network the rest of this repo is sized for. Editing wants sustained throughput far beyond the live budget: nine HD H.264 angles at up to 70 Mb/s is ~80 MB/s per editor before any ProRes or 4K, and exports add to that. Putting that on the existing bonded 1GbE would either starve the editors or risk contention with the live ingest pipelines still running during the event.

Key decisions, each evaluated rather than assumed:

- **Dedicated NAS, not Nextcloud's own storage** — the same pull that ingests each theatre's ATEM/VMix footage into Nextcloud (FTP for the ATEMs, SMB for VMix) writes a second copy straight to the edit-suite NAS at the same time, no chained re-sync. Designed but not yet implemented in the ingest scripts, and it needs a 10GbE port on the Unraid box on the edit LAN. Decouples editing entirely from the live show's infrastructure.
- **A dedicated Mac mini running both the Resolve Project Server and Remote Render** — needed because the whole workflow below is one shared show project both editors work against, not two independent copies (confirmed **not** possible on a Blackmagic Cloud Store appliance itself, which is storage-only). Combining both roles on one box isn't documented as forbidden by Blackmagic but is thinly precedented in practice — see Decision 2 in docs/live-editing.md for the full honesty-check.
- **Blackmagic Cloud's internet sync service — not needed here.** A genuinely separate product from the Cloud Store hardware, built for cross-site collaboration; this is a single-venue setup with no second site to sync with.
- **A pre-built Resolve project structure, generated by resolve-configurator** — a companion project to this design — scaffolds a bin per theatre-day and an empty timeline per session ahead of time, plus writes the exact smart-bin rules as a one-time-apply recipe (Resolve's own scripting API can't create real smart bins). Once footage lands on the NAS and that recipe's been applied, each session's bin auto-fills with just its own clips — editors open a session, drag in, top-and-tail, export.

Also covers a complementary DIT (Digital Imaging Technician) ingest workflow for physical media — the ATEM's own USB SSD and any standalone camera cards — comparing ShotPut Pro, OffShoot, Silverstack, and the companion tools FoolCat (camera reports) and o/PARASHOOT (safe card reformatting). Full design in docs/live-editing.md .

Subnet map

Mothership on 192.168.1.x ; theatres contiguous from .2.x through .13.x , grouped 3 per physical VLAN purely for uplink cabling.

Theatre	Subnet	VLAN (uplink grouping)
1	192.168.2.x	VLAN 1
2	192.168.3.x	VLAN 1
3	192.168.4.x	VLAN 1
4	192.168.5.x	VLAN 2
5	192.168.6.x	VLAN 2
6	192.168.7.x	VLAN 2
7	192.168.8.x	VLAN 3
8	192.168.9.x	VLAN 3
9	192.168.10.x	VLAN 3
10	192.168.11.x	VLAN 4
11	192.168.12.x	VLAN 4
12	192.168.13.x	VLAN 4

VMix Node 1 (off Theatre 1): 192.168.20.x . VMix Node 2 (off Theatre 4): 192.168.21.x . See docs/bandwidth-analysis.md for whether this 4-VLAN grouping is the right count to actually build, or whether to consolidate.

Operations: building it, running it, and the road not taken

Three docs turn the design into something an on-site crew can actually execute:

- docs/deployment-runbook.md — the full ordered build sequence, Phase 0 (confirmations) through Phase 7 (pre-event verification), each step linking back to the doc with the detail. This is the doc to have open on build day.
- docs/troubleshooting.md — the gotchas surfaced *during design* (unreachable theatres, DERP relay when direct was expected, NIC bonds that won't form, ingest falling behind, missing monitoring previews), each as symptom → cause → fix, so they don't have to be rediscovered under show pressure.
- docs/topology-alternative-tailscale-switches.md — the road not taken: plain switches + Tailscale on generic Linux boxes instead of GL-iNet routers. Documented with working configs (config/alt-tailscale-switch/) so the decision can be revisited with evidence, but not recommended — the doc explains why.

Before build — open items

Design is finalized; these are the headline items still needing real-world input before the config in this repo gets applied (the complete list, including a few smaller ones, lives in docs/open-questions.md ; the ordered build sequence is docs/deployment-runbook.md).

- ☐ Benchmark Tailscale on one Slate AX (iperf3, both directions at once, direct path not DERP) before buying 14 — target above ~160 Mbps combined — docs/open-questions.md #19.
- ☐ Patch every ATEM to the common input map, set the program recording to ~8-10 Mbps, and measure one real camera ISO and one laptop ISO file to pin the planning figures — docs/atem-iso-ingest.md , #0.
- ☐ Confirm the Cloud Gateway model (LAG support, routing capacity), and prove the mothership → tailnet routing path end to end — static routes, macvlan host access, ACL — #17, #18.
- ☐ Set the SRT payload size to 1128 bytes on the VMix PCs and Restreamer to fit Tailscale's 1280-byte MTU — #20.
- ☐ Smaller build items: which ATEM model (original or G2) and its ISO file naming, testing pull-iso.py 's new folder walk against a real unit, smart-bin path rule vs the NAS layout, and an Unraid address on the edit LAN — #21–24.
- ☐ Check the existing consolidated services server against the calculated target spec in docs/server-specification.md (CPU/RAM headroom, storage pool layout — no GPU requirement any more).
- ☐ Add the derp.example.net DNS record for the self-hosted DERP server.
- ☐ Verify the DERP hairpin isn't caught by the uplink-VLAN firewall block before relying on it.
- ☐ Empirically verify that a file copied mid-recording plays.
- ☐ Confirm what VMix is actually recording and set up real SMB shares/credentials on all 4 VMix PCs.

- ☐ Confirm the final VLAN count with the venue, and decide whether a mass-NDI-fallback event needs to survive at full quality (drives 3-4 vs. 2 VLANs) — [docs/bandwidth-analysis.md](#).
- ☐ Add the `kvm.example.net` DNS record and confirm `GLKVM_ACCESS_IP`'s exact format for the WAN-remapped GLKVM-Cloud ports — [docs/glkvmm-cloud.md](#).
- ☐ Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's real container images, and whether the ATEM/BirdDog Play can actually originate their monitoring/preview streams alongside their primary feeds — [docs/open-questions.md](#) #11.
- ☐ Implement the second write destination (the edit-suite NAS) in the actual ingest scripts — documented in [docs/live-editing.md](#) but not yet a code change — [docs/open-questions.md](#) #15.
- ☐ Verify the NDI backup path's settled redistribution model — VMix node NDI → BirdDog Central → theatre PLAYs (Restreamer can't emit NDI and Central can't ingest SRT, both confirmed) — including that Central genuinely re-originates the stream rather than pointing receivers at the source — [docs/streaming-flow.md](#), [docs/open-questions.md](#) #16.
- ☐ Decide how much footage the live-editing subsystem needs access to, confirm the Mac mini's combined Project Server + Remote Render role holds up under real load, and whether any standalone cameras are in scope — [docs/live-editing.md](#), [docs/open-questions.md](#) #12-14.

Everything in this repo

Design docs

- [docs/topology.md](#) — full network layout: subnets, VLAN grouping, mothership, VMix nodes, theatre wiring
- [docs/server-specification.md](#) — calculated minimum spec for the consolidated server (storage pools, RAM, CPU, NIC validation; GPU no longer required)
- [docs/ip-address-map.md](#) — every device on the network, with its real IP (133 devices, including the live-editing subsystem's NAS, 2 MacBook Pros, and Project Server/Remote Render Mac mini)
- [docs/streaming-flow.md](#) — SRT/NDI streaming path, the NDI Discovery Server plan
- [docs/birdog-play-rationale.md](#) — why BirdDog Play over a laptop + VLC for theatre playback
- [docs/gl-inet-rationale.md](#) — why GL-iNet + Tailscale for segmentation/routing, multi-WAN/pay-per-device WiFi economics, and other uses (LED processors, mixing desks, mesh) beyond this event
- [docs/live-editing.md](#) — post-production subsystem at the mothership: dedicated NAS/network, Resolve collaboration decisions, the resolve-configurator-driven edit workflow, DIT ingest tool comparison
- [docs/file-sync-flow.md](#) — rclone + WebDAV sync to Nextcloud
- [docs/atem-iso-ingest.md](#) — near-real-time ATEM ISO ingest into Nextcloud, no manual chunking
- [docs/vmix-record-ingest.md](#) — same mechanism, targeting each VMix PC's SMB share
- [docs/glkvmm-cloud.md](#) — self-hosted GLKVM-Cloud for centralized remote administration of the 14 GL-iNet routers
- [docs/bandwidth-analysis.md](#) — real-world throughput math against the venue's 1 Gbps VLANs, and how many are actually needed
- [docs/tailscale.md](#) — DERP-avoidance reasoning, the self-hosted DERP server, verification commands
- [docs/open-questions.md](#) — every decision made, how it was reached, and what's still genuinely open
- [docs/topology-alternative-tailscale-switches.md](#) — plain switch + Tailscale-on-Linux alternative to GL-iNet, not recommended but documented
- [docs/deployment-runbook.md](#) — ordered build checklist synthesizing every doc/config into one sequence
- [docs/troubleshooting.md](#) — known gotchas surfaced during design, symptom → cause → fix

Diagrams ([diagrams/](#)) — topology (full device inventory), streaming-flow, file-sync-flow, live-editing-dataflow (dual-write into the edit suite), and live-editing-project-structure (resolve-configurator → smart-bin recipe → editor workflow), all as standalone SVGs

Offline exports ([export/](#)) — every doc as a self-contained HTML file and an A4 PDF, plus a combined [full-documentation](#) version of each — see [export/README.md](#)

Generated configs

- [config/docker-compose.yml](#) — the entire consolidated server's Docker stack in one file, see [config/README.md](#)

- `config/tailscale-acl.json` — ACL policy (cross-theatre isolation) + the self-hosted DERP region
- `config/tailscale-up-commands.sh` — per-role `tailscale up` template
- `config/tailscale-up-all-devices.sh` — the same, fully instantiated for all 15 tailnet nodes
- `config/gl-inet/` — UCI network config for all 14 routers (12 theatres + 2 VMix nodes), generated from templates
- `config/unifi/` — Cloud Gateway VLAN/DHCP/firewall/port-forward reference
- `config/atem-iso-ingest/` — the FTP-pull ingest script (and rclone+rsync alternative) plus Nextcloud External Storage setup
- `config/vmix-record-ingest/` — the same rclone+rsync combo, targeting each VMix PC's SMB share
- `config/glsvm-cloud/` — the self-hosted GLKVM-Cloud docker-compose stack (`rttys` + `coturn`)
- `config/alt-tailscale-switch/` — configs for the plain-switch/generic-Linux alternative, not recommended but documented

Trademarks

NDI® is a registered trademark of Vizrt NDI AB. See <https://ndi.video>. Other product and company names mentioned in this documentation are the trademarks of their respective owners. This project is not affiliated with or endorsed by any of them.

Topology

Theatre nodes (×12)

Each theatre is its own subnet, NAT'd by its own GL-iNet **Slate AX (GL-AXT1800)** router (previously the A-1300 / Slate Plus; replaced 2026-10-06 because Tailscale's userspace WireGuard on the A-1300's 32-bit CPU was estimated at only ~30-70 Mbps — see `docs/bandwidth-analysis.md`). The router runs Tailscale as a subnet router (LAN + WAN side routing enabled), advertising the theatre's `/24` .

Physical wiring: the Slate AX has 1 WAN + 2 LAN gigabit ports (the same layout the A-1300 had, so the wiring is unchanged). Both LAN ports are used:

- **LAN 1 → ATEM Mini Extreme ISO directly.** Dedicated port, so the router's other LAN traffic (rclone syncs, control laptop, etc.) never shares a switch segment with it. The ATEM carries the heaviest sustained traffic of any theatre device — the near-real-time ingest pull (see `docs/atem-iso-ingest.md`) — camera ISO + program by default — is a continuous ~47-83 Mbps flow, well above BirdDog Play's — so it gets the dedicated port.
- **LAN 2 → 8-port unmanaged Netgear switch,** which fans out to everything else: BirdDog Play, PowerPoint Main/Backup, VT Main/Backup, Control laptop (6 devices + 1 uplink port, 1 spare switch port).

All devices stay on the same `192.168.X.0/24` — this is a physical port split for traffic separation, not a VLAN/subnet split.

Role	Address	Physical connection
GL-iNet Slate AX router (LAN gateway)	<code>192.168.X.1</code>	—
ATEM Mini Extreme ISO	<code>192.168.X.2</code>	Slate AX LAN 1 (direct)
BirdDog Play	<code>192.168.X.20</code>	Netgear switch → Slate AX LAN 2
PowerPoint Main	<code>192.168.X.5</code>	Netgear switch → Slate AX LAN 2
PowerPoint Backup	<code>192.168.X.6</code>	Netgear switch → Slate AX LAN 2
VT Main	<code>192.168.X.7</code>	Netgear switch → Slate AX LAN 2
VT Backup	<code>192.168.X.8</code>	Netgear switch → Slate AX LAN 2
Control laptop	<code>192.168.X.10</code>	Netgear switch → Slate AX LAN 2

`X` per the subnet map in the top-level README: Theatre 1 → `2` , Theatre 2 → `3` , ... Theatre 12 → `13` .

The ATEM's built-in FTP server (its ISO recording drive) is pulled continuously by the mothership over Tailscale subnet routing — see `docs/atem-iso-ingest.md` .

Alternative to the GL-iNet router, documented but not recommended: plain switch + Tailscale on generic Linux — see `docs/topology-alternative-tailscale-switches.md` .

Uplink VLAN grouping

Theatres are grouped 3-per-physical-VLAN purely to organize uplink cabling back to the mothership — **this grouping does not itself provide cross-theatre isolation**. Each GL-iNet Slate AX already NATs its own theatre LAN, and once every router joins the same tailnet, routes are reachable tailnet-wide unless Tailscale ACLs restrict them (see `config/tailscale-acl.json`).

These VLANs are venue-supplied, not our infrastructure — each capped at 1 Gbps, each port expensive. The 4-VLAN/3-theatres-each split below is comfortable at every live-ingest scope; the bandwidth math allows consolidating to 2 VLANs (6 theatres each) for normal SRT-primary operation at the default camera + program scope (~50% typical / ~65% high upstream, higher in the group carrying both VMix nodes) or camera + 1 laptop at typical rates (~65%), but only ~10% headroom in a mass NDI fallback — see `docs/bandwidth-analysis.md` for the full model and recommendation.

VLAN	Theatres	Subnets
1	1, 2, 3	.2.x, .3.x, .4.x
2	4, 5, 6	.5.x, .6.x, .7.x
3	7, 8, 9	.8.x, .9.x, .10.x
4	10, 11, 12	.11.x, .12.x, .13.x

Mothership — 192.168.1.x

Ubiquiti **Cloud Gateway**; all 4 VLANs terminate here via separate links. Also hosts the 4× "ready room" computers (physical, untouched) and the hardwired WAN.

Consolidated services server (Unraid). Everything else that used to be described as separate mothership boxes — Nextcloud, Restreamer, BirdDog Central, the NDI Discovery Server, a self-hosted DERP server, the ATEM ISO ingest pipeline, the VMix record ingest pipeline, a self-hosted UniFi Controller, self-hosted GLKVM-Cloud, ATEM Overseer + ATEM Fleet Admin (fleet monitoring and provisioning for the 12 theatre ATEMs), and now Flock (BirdDog Play fleet manager) — runs on a single physical server instead, virtualized with Unraid. (An earlier revision also carried a mothership-local VMix instance VM; it was removed from the design — the theatre program feeds originate from the VMix *node* PCs, and no mothership role for a local VMix instance was ever established, see docs/open-questions.md #10.) The server already exists (no procurement needed) — known so far: **2× gigabit NICs (bonded — see below), a "decent" discrete GPU** (on hand, though nothing in the design requires it any more — see the GPU section of docs/server-specification.md). Full spec (CPU/motherboard, RAM) is TBD — see docs/open-questions.md and the calculated target minimum spec (storage pool layout, RAM, CPU, GPU tier, all worked from this box's actual workload) in docs/server-specification.md .

Workload	Runs as	Notes
BirdDog Central	Windows VM	Windows-only (tech specs); the design's only VM — retained for the NDI backup path's sender side (docs/streaming-flow.md), not for fleet management (that's Flock's job)
Nextcloud	Docker container	official image + MariaDB + Redis (see config/docker-compose.yml)
Restreamer	Docker container	official datarhei/restreamer image
NDI Discovery Server	Docker container	see docs/streaming-flow.md
DERP server	Docker container	self-hosted relay fallback — see docs/tailscale.md
ATEM ISO Ingest	Docker container	pulls all 12 theatres' ATEM ISO recordings via FTP, dual-writing to Nextcloud and the edit-suite NAS in the same pass — see docs/atem-iso-ingest.md and docs/live-editing.md
VMix Record Ingest	Docker container	pulls all 4 VMix PCs' recordings via SMB, same dual-write pattern — see docs/vmix-record-ingest.md
UniFi Controller	Docker container	self-hosted UniFi Network Application — local management/backup console for the Cloud Gateway, independent of cloud.ui.com — see config/unifi/README.md
GLKVM-Cloud (rttys + coturn)	2 Docker containers	self-hosted remote administration (web UI + SSH terminal) for the 14 GL-iNet routers — see docs/glkvm-cloud.md
ATEM Overseer	Docker container	fleet monitoring/tally dashboard for all 12 theatres' ATEMs — receives each theatre's monitoring stream, see docs/bandwidth-analysis.md
ATEM Fleet Admin	Docker container	bulk ATEM provisioning — model-aware config forms, live network apply via atem-connection over the same Tailscale subnet routing the ISO ingest pipeline uses, or XML+media folder export
Flock	Docker container	BirdDog Play fleet manager — LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits; receives each theatre's BirdDog Play preview stream, see docs/bandwidth-analysis.md
Tailscale subnet router	Docker container, host networking	advertises 192.168.1.0/24 ; replaces the earlier "Nextcloud host" plan now that Nextcloud itself is a container on this same box

Bonded NICs. Decided: bond the 2 gigabit NICs (802.3ad/LACP) rather than split traffic across them — SRT bandwidth isn't constant, and LACP's hash-based distribution spreads the many *simultaneous* flows this box handles (12 theatres' SRT fan-out, each to a different destination, the 12 incoming Overseer monitoring streams and Flock SRT previews, plus rclone/Nextcloud/BirdDog Central/NDI Discovery Server/DERP traffic) across both links by destination-IP/port hash — giving real aggregate headroom for bursty traffic, not just failover. Note this does **not** speed up any single flow; the benefit comes from having many distinct flows to hash across. On the wire, though, everything bound for one theatre leaves the Tailscale container inside a single WireGuard UDP flow to that theatre's Slate AX, so the bond actually hashes

~14 tunnel flows (one per router), not dozens of application flows — still enough to spread, but coarse: one link can end up carrying noticeably more than half.

Assuming the Cloud Gateway supports LAG on the ports the Unraid server lands on (only UDM Pro/SE/Pro Max, UXG Enterprise, and EFG support port aggregation — not base Cloud Gateway models). If it turns out this one doesn't: drop an intermediate LACP-capable switch in between and bond through that instead — the bond doesn't care which device terminates it, only that something in the path speaks LACP, so this is a cheap fallback rather than a blocker.

One thing worth doing regardless of which device terminates the bond: **set Unraid's bond to match Ubiquiti's LACP settings**. Ubiquiti gear hardcodes LACP rate `fast` and hash policy `layer3+4`; Unraid's bonding defaults are `slow` rate and `layer2` hash. The bond still forms with a mismatched hash policy (each end hashes its own transmit traffic independently), but `layer2` hashing on the Unraid side puts everything bound for the one gateway MAC on a single link, which throws away the outbound headroom the bond exists for.

Why Unraid over TrueNAS SCALE: both support Docker and Windows VMs, and Unraid has the more polished one-click container experience (Community Applications) — useful since this may be maintained on-site by AV staff rather than a Linux admin. Trade-off: Unraid requires a paid license (2026 tiers: Starter \$49 for 6 storage devices, Unleashed \$109, Lifetime \$249 — see `docs/open-questions.md` for which tier this box's pools need); TrueNAS SCALE is free. (The original tiebreaker — Unraid's more mature GPU-passthrough workflow — no longer applies now that the VMix VM is gone: nothing on this box needs GPU passthrough. BirdDog Central is NDI routing/control, not video encode, and runs as a plain VM. The choice stands on the container/UX grounds alone.)

Tailscale note: Cloud Gateway (UniFi-OS-based) generally can't run Tailscale natively without an unofficial container hack, so the mothership subnet-router role runs as a container on the Unraid box instead of the Cloud Gateway itself. See `docs/open-questions.md`.

VMix nodes (×2)

Each has its own **GL-iNet Slate AX** router (same model as the theatre routers) and its own Tailscale connection — a separate node on the tailnet, not part of the theatre it physically sits next to, just uplinked near it:

Node	Uplinks near	Subnet	Contents
VMix Node 1	Theatre 1	<code>192.168.20.x</code>	4× BirdDog P400 cameras, 2× VMix PCs
VMix Node 2	Theatre 4	<code>192.168.21.x</code>	4× BirdDog P400 cameras, 2× VMix PCs

That makes **14 GL-iNet Slate AX routers total** across the network — 12 theatre routers plus these 2. Both LAN ports are bridged together on the VMix node routers (unlike the theatre routers, there's no single traffic-sensitive device here that needs its own dedicated port the way the ATEM does).

Each VMix PC's own recording is pulled to the mothership the same way the ATEMs' recordings are — near-real-time, over this node's own dedicated uplink (not the adjacent theatre's) — see `docs/vmix-record-ingest.md`.

Edit suite — `192.168.22.x`

Post-production subsystem at the mothership: 2× MacBook Pro editing workstations, a Mac mini running the DaVinci Resolve Project Server and Remote Render, and a dedicated fast-storage NAS, on their own 10GbE LAN — **not** on the Tailscale mesh (same room as the mothership, no distance for Tailscale to bridge) and not sharing the theatre-facing network's bandwidth budget. Routed to the mothership LAN only so the ingest containers can dual-write footage to the edit-suite NAS in the same pull that feeds Nextcloud — there is no separate recording-pool-to-NAS sync step. Full design, including the storage and project-server decisions and the DIT physical-media ingest workflow, in `docs/live-editing.md`.

See `diagrams/topology.svg` for the full visual layout — every theatre, every device, every real IP — and `docs/ip-address-map.md` for the same inventory as a text table. Per-device configs: all 14 routers in `config/gl-inet/`, the Cloud Gateway in `config/unifi/`, and Tailscale in `config/tailscale-up-all-devices.sh`.

Consolidated services server — minimum specification

The server itself is already on hand (see `docs/topology.md`) — this document calculates the **target minimum spec** to check that hardware against, working forward from the actual workload this box runs (`docs/topology.md` 's VM/container table) rather than a generic "decent server" guess. Where a figure depends on something this repo hasn't pinned down (the event's actual day count/hours), that's flagged explicitly rather than assumed — see `docs/open-questions.md` for the running list.

Storage — three pools, not one array

Unraid's traditional parity array is built around mixed-size spinning disks with a single-write-at-a-time parity bottleneck, absorbed by an SSD cache pool that the overnight "mover" drains onto the array — a good fit for bulk archival storage, a poor fit for a box whose entire storage workload here is already flash. Recommendation: skip the legacy array entirely and use three separate Unraid **pools** (ZFS, available since Unraid 6.12), each sized and mirrored for what it actually holds.

Container/VM pool

Holds `docker.img` , all container `appdata` (including the three databases — MariaDB, MongoDB, Redis — that want fast, low-latency small-I/O access), and the BirdDog Central VM's vdisk.

Item	Estimate
BirdDog Central vdisk	~80 GB (OS + app)
<code>docker.img</code>	~50 GB
Combined <code>appdata</code> (12 containers + the Tailscale subnet router + 3 DBs)	~58 GB
Total	~188 GB

Minimum: 500 GB usable, mirrored NVMe (2× ~512GB+ NVMe) — comfortable headroom over the ~188 GB estimate, and mirrored because rebuilding a Windows VM from scratch mid-event is real downtime, not just an inconvenience. NVMe specifically (not SATA SSD) because VM and database performance is latency-sensitive, not throughput-bound.

Drive class: mainstream consumer NVMe with onboard DRAM cache (not a DRAM-less budget drive) is genuinely sufficient here — no need for the enterprise tier the recording pool below needs. VM/appdata churn is nowhere near enough sustained write volume to meaningfully test even consumer-grade endurance ratings, and the workload cares about consistent low-queue-depth latency (how snappy a container restart or a DB query feels), which good consumer NVMe already delivers.

Content pool — 500 GB (as specified)

Nextcloud's own storage: theatre laptop content synced via rclone (established at 10-20 GB/theatre × 12 theatres = **120-240 GB**, see `docs/bandwidth-analysis.md`), plus Nextcloud's own app/database footprint and versioning overhead (flagged as a real multiplier in `docs/atem-iso-ingest.md` 's versioning caveat — version retention settings directly affect how much of this budget versioning eats into). 500 GB gives roughly 2-4x headroom over the raw 120-240 GB content estimate, which comfortably absorbs version history without needing an aggressive retention policy from day one.

Recommendation: mirrored, SATA SSD is adequate (no NVMe-level latency requirement here) — mirrored less for data-loss prevention (every laptop is still the source of truth for its own content even if this copy is lost) and more for **availability**: a lost content pool mid-event means re-running the initial bulk sync for all 12 theatres, which is itself only a ~20-40 minute operation at 50-100 Mbps (`docs/bandwidth-analysis.md`) but is a real, avoidable disruption during a live event.

Drive class: same reasoning as the container/VM pool — mainstream consumer SATA SSD with DRAM cache, the cheapest reasonable tier that still isn't a DRAM-less budget part. Gentle, sporadic write pattern; no case for spending more here.

Recording pool — 8 TB, all-flash (as specified)

This is where the calculation actually matters — both *why* it has to be flash and *whether 8 TB is enough*.

Why all-flash, not spinning disk — it's not about raw throughput. Combined worst-case write load onto this pool, from `docs/bandwidth-analysis.md`'s own established figures:

Source	Per-unit high case	Count	Total
ATEM ISO ingest, default live scope (camera ISO + program)	~80 Mbps (~83 Mbps incl. WireGuard tax)	12 theatres	~960 Mbps
VMix Record ingest	~30 Mbps (assumed, unconfirmed — see <code>docs/open-questions.md</code>)	4 PCs	~120 Mbps
Combined			~1,080 Mbps ≈ 135 MB/s
If both laptop inputs are also pulled live (<code>ATEM_ISO_INPUTS=1,2,3</code>)	~160 Mbps	12 theatres	~2,040 Mbps ≈ 255 MB/s combined

(ATEM figures are the 2026-10-06 planning figures — camera ISO 35 / 45 / 70 Mbps, laptop ISO 15 / 25 / 40 Mbps *estimated*, program ~10 Mbps — for the program file plus only the ISO inputs pulled live; see `docs/atem-iso-ingest.md` § Common theatre input map. The pool stores payload, so WireGuard overhead isn't counted.)

135-255 MB/s sustained is within a single 7200 RPM HDD's *rated sequential* throughput on paper — so the case for flash isn't the aggregate number, it's the **access pattern**. That load isn't one stream, it's **~28 independent files being appended to concurrently** (12 theatres × camera ISO + program, plus 4 VMix — ~52 with both laptop inputs live), each on its own ~60-90s poll cycle — see `docs/atem-iso-ingest.md` and `docs/vmix-record-ingest.md`), with Nextcloud's own targeted `occ files:scan` indexing calls layered on top of that. A spinning disk pays a real seek penalty (~5-15ms) every time it switches between those ~28+ scattered write targets — under this specific concurrent-scattered pattern, achievable HDD throughput collapses to a small fraction of its rated sequential number, while any flash drive (no seek penalty) handles 135-255 MB/s of concurrent random-ish writes without strain. The "all-flash" requirement is about the write *pattern* this design creates, not the volume.

Does 8 TB actually cover the event? This is the one figure genuinely blocked on an input this repo doesn't have: **actual event duration (days × active-recording hours/day) isn't established anywhere in this design**. Working from the same combined-load figures:

Live scope (all 12 theatres)	Combined rate, typical / high	8 TB covers
Camera + program (default)	~98 / 135 MB/s ≈ 351 / 486 GB/hr	~23 / ~16.5 hours
Camera + 1 laptop + program	~135 / 195 MB/s ≈ 486 / 702 GB/hr	~16.5 / ~11.4 hours
Camera + 2 laptops + program	~173 / 255 MB/s ≈ 621 / 918 GB/hr	~12.9 / ~8.7 hours

(Decimal units throughout — 8 TB = 8,000 GB, as drives are sold. Rate = 12 × the per-theatre ATEM payload (camera + program: 55 typical / 80 high Mbps; each laptop adds 25 / 40) + the same ~120 Mbps VMix placeholder, ÷ 8. Low-case figures (camera ISO at ~35 Mbps) stretch the default-scope coverage to ~27 hours. All figures are to a *completely* full pool; ZFS performance degrades well before 100%, so treat roughly 85-90% of each as the practical figure.)

Basis — planning figures, not measurements. The old ~10 Mbps-per-ISO basis was replaced on 2026-10-06 (Blackmagic: ISO files up to 70 Mb/s, not user-settable); the table above instead assumes only the program file and the chosen ISO inputs are pulled live, at the per-source planning figures in `docs/atem-iso-ingest.md` § Bandwidth. The camera figure rests on user reports and the laptop figure is an unmeasured estimate — one measured file of each settles it (`docs/open-questions.md` #0). The ISOs *not* pulled live arrive via the physical DIT offload to the edit-suite NAS (`docs/live-editing.md`), not this pool; if they are also to be archived here, add each ATEM's full recording (at least ~14-36 GB per theatre-hour more for the two laptop ISOs) to the rate.

At a typical ~8-10 hour active-program day, that's roughly **1.6-2.9 days** of recording at the default scope (high to typical rates) before the pool fills — workable for a short event, tight for a longer one — and only ****~0.9-1.6 days**** if both laptop inputs are pulled live fleet-wide. **Confirm actual event length against this table before treating 8 TB as settled** — if it's a multi-day event without a periodic archive-off step already planned, either the day count needs to fit the budget above, or a nightly archive-to-cold-storage step needs adding to free capacity for the next day. The ingest pipelines write directly into Nextcloud's External Storage location and, in the same pull, the edit-suite NAS (docs/atem-iso-ingest.md , docs/live-editing.md) — neither write has an offload stage that frees *this* pool. Tracked in docs/open-questions.md .

Redundancy — worth adding capacity for, not assumed in the 8 TB figure. This pool is the authoritative archive: the dual-write to the edit-suite NAS (docs/live-editing.md) does give the footage a second landing spot at ingest time, but that copy may be a curated subset (open-questions #12) and one of the candidate NAS units is RAID 0 — so a failure here still risks footage that can't be re-shot. Two ways to get 8 TB *usable* with 1-drive fault tolerance:

Option	Raw capacity needed	Trade-off
2× 8 TB NVMe, mirrored	16 TB raw	Simplest to reason about, fastest resilver, 50% capacity overhead
4× ~2.7 TB NVMe, RAID-Z1	~10.8 TB raw	75% capacity efficiency vs. 50%, slower resilver on failure, more drives to manage

Real drive sizes don't land on these numbers. Enterprise NVMe in the ~1 DWPD tier is sold in 1.92 / 3.84 / 7.68 / 15.36 TB steps (e.g. Samsung PM9A3), so "2× 8 TB mirrored" is in practice **2× 7.68 TB ≈ 7.7 TB usable** (a few percent under 8 TB before ZFS's own reserve), and there is no ~2.7 TB SKU — the realistic RAID-Z1 build is **4× 3.84 TB ≈ 11.5 TB usable** (15.36 TB raw). If 8 TB usable is a hard floor, the mirror needs 2× 15.36 TB; otherwise 2× 7.68 TB is the honest reading of "8 TB" and the coverage table above shrinks by ~4%.

Recommend the mirror for the same reason Unraid was chosen over TrueNAS SCALE elsewhere in this design (docs/topology.md) — operational simplicity, since this may be maintained by AV staff rather than a storage specialist, matters more here than squeezing out the last few TB of efficiency.

Endurance. Since this repo already treats the router hardware as reusable across events (see docs/gl-inet-rationale.md), the same applies here — a full pool fill is roughly 8 TB of actual flash writes (these are byte-range *appends*, not whole-file rewrites, so total written ≈ total recorded, not a multiple of it), and a mirrored pair each independently absorb that same 8 TB per event. Even a deliberately-generous worst-case estimate — ~920 GB/hr sustained (both laptop inputs live, high rates) for a full 24 h/day across ~20 event-days a year, beyond what the pool could even hold without nightly archive-off — comes to roughly 440 TB/year of actual writes to the pool — comfortably inside what even the *lower* enterprise endurance tier (see below) is rated for over a normal warranty period, so endurance headroom isn't actually the tight constraint here once the right drive class is picked.

Drive class: this is the one pool where consumer-grade flash — even a good, DRAM-cached, high-TBW consumer drive — isn't the right call, and endurance isn't the reason. The deciding factor is **power-loss protection (PLP)**: capacitor-backed circuitry that flushes a drive's write cache to permanent storage if power drops mid-write. Consumer/prosumer NVMe drives don't have this. A generator hiccup, a tripped breaker, or a UPS transfer glitch mid-event — exactly the kind of thing a live event's own power setup can produce — risks losing whatever was sitting in the drive's write cache at that instant, on the one pool in this whole design with no second copy of the data anywhere else. That's the actual argument for **enterprise-class NVMe** here specifically (not "enterprise is always better," which isn't true for the other two pools above) — server-validated drives with capacitor-backed PLP, provisioned for sustained high-queue-depth writes without the cache-cliff behavior consumer drives show once their fast write cache fills under prolonged concurrent load. Enterprise drives are also sold in more than one endurance tier (commonly around 1 drive-write-per-day vs. 3× that for a heavier-duty tier) — the lighter of the two tiers already has large margin over this workload's realistic annual write volume, so there's no need to pay for the heavier tier's endurance on top of the PLP requirement that's actually driving this choice.

Write caching

With every pool already flash (no legacy spinning array behind anything), the classic Unraid "SSD cache pool absorbing writes before the overnight mover" pattern doesn't apply here — there's no slow array for it to shield. What actually buffers the bursty, ~28-concurrent-stream ingest pattern described above is:

- **RAM** — the OS page cache smooths write bursts before they hit the drives (see the RAM section below, which already budgets for this).
- **The drives' own onboard DRAM cache** — a real purchasing consideration: choose NVMe drives with onboard DRAM (not DRAM-less/QLC-only budget drives), since sustained multi-stream concurrent writes are exactly the workload DRAM-less drives handle worst.

NIC — validating the already-decided 2× bonded GbE

`docs/topology.md` already settled on 2× gigabit NICs, bonded (802.3ad LACP). Checking that against the heaviest combined load this design has identified — the mass-NDI-fallback disaster scenario from `docs/bandwidth-analysis.md`, layered on top of full ATEM ingest:

Direction	Load	Total
Inbound (theatres → mothership), default live scope (camera + program)	ATEM ingest incl. WireGuard (~686 typical / ~998 high Mbps) + VMix ingest (~120 Mbps) + ATEM Overseer (~125 Mbps) + Flock (~125 Mbps) + rclone (~60 Mbps)	~1,120 typical / ~1,430 high Mbps
Inbound, camera + 2 laptops + program everywhere	ATEM ingest ~1,310 typical / ~1,997 high Mbps + the same ~430 Mbps	~1,740 typical / ~2,430 high Mbps
Outbound (mothership → theatres), all 12 theatres on NDI fallback simultaneously	12 × ~130 Mbps	~1,560 Mbps

(Recomputed 2026-10-06 from the per-source planning figures in `docs/atem-iso-ingest.md` § Bandwidth, replacing the earlier ~1,493 Mbps that rested on ~10 Mbps per ISO stream; rclone's ~5 Mbps per theatre is now counted too. Matches `docs/bandwidth-analysis.md`.)

Both directions run independently over a full-duplex link, so they don't stack against each other — but each direction needs to fit across the bond's two physical 1 Gbps links via LACP's per-flow hashing (each *individual* flow is capped at 1 Gbps, since LACP doesn't split one flow across both links).

How many flows the hash actually sees is the catch. Everything to and from the theatres crosses the wire as **WireGuard**, and the bond hashes the *outer* packet — so each theatre's ATEM ingest + Overseer + Flock + rclone (and, outbound, its SRT or NDI) collapse into **one UDP flow per tailnet peer**: ~14 flows in each direction, not 36+. Each is still far under 1 Gbps (~109 Mbps per theatre inbound at the default scope's high case, ~130 Mbps outbound during NDI fallback), so no *single* flow is the problem — but 12-14 roughly equal flows split 2 ways by an effectively random hash are often uneven. Worked out exhaustively for the high cases above (scratch calculation over all 2¹⁴ assignments, treating each flow's link as a coin flip): **~21% of possible hash assignments put more than ~940 Mbps of usable payload on one link inbound at the default scope's high rates, and ~39% for the NDI outbound case** (8 of 12 NDI flows × 130 Mbps = 1,040 Mbps). With both laptop inputs live fleet-wide it is ~77% inbound even at typical rates — effectively "doesn't fit" on 2× 1GbE, and the high case (~2,430 Mbps) exceeds the bond's aggregate outright. Real hashes are deterministic per peer address/port, so the result is "fine or not, for the whole event," not random per minute — and can't be predicted on paper.

So, at the default live scope: **the aggregate fits (~1,430 Mbps inbound at high rates, ~1,560 outbound, vs. ~2,000), and normal operation (~1,120 Mbps typical inbound, no NDI fallback — ~1% of hash assignments overload a link) is comfortable — but the worst-case disaster scenario is only "probably fits," not "checks out,"** on 2× 1GbE — and that is *before* the mothership-side routing hairpin described below, which on its own breaks the NDI-fallback case unless the VM is routed directly. It does check out once ATEM ingest is paused fleet-wide during a mass NDI fallback (the mitigation below), and it checks out unconditionally on 2× 2.5GbE (see below). Pulling the laptop inputs live fleet-wide does *not* fit 2× 1GbE (see above) — it needs 2× 2.5GbE, or the laptop inputs enabled for only some theatres (`ATEM_ISO_INPUTS` is currently one fleet-wide setting). The inbound figure has moved each time this was revalidated (~1,205 → ~1,493 → now ~1,430 Mbps at the default scope's high case) as Overseer, Flock and the ISO bitrate correction landed — see `docs/bandwidth-analysis.md` 's own worked-through comparison of this exact disaster scenario with and without pausing ATEM ingest fleet-wide, which is the more consequential mitigation than the NIC choice itself.

One load deliberately NOT counted against this bond: the dual-write to the edit-suite NAS. The same ingest containers that produce the inbound figures above also write every recording out again to the edit-suite NAS at `192.168.22.2`

(docs/live-editing.md) — another ~1,080 Mbps of outbound at the default scope's high case, up to ~2,040 Mbps with both laptop inputs live. If that leg transited the theatre-facing bond it would stack on top of the NDI-fallback outbound and break the conclusion above — so it must **not**: the edit suite is its own 10GbE LAN, and this box needs a separate edit-LAN-facing interface (a 10GbE NIC, or at minimum its own dedicated port) for that traffic, counted in the spec alongside the 2× bonded theatre-facing GbE.

Worth it: 2× 2.5GbE, if the box and whatever terminates the bond support it. This is the one change that removes the hash-imbalance risk above outright at the default scope — even the worst possible split (all ~1.4-1.6 Gbps on one link) fits inside a single 2.5 Gbps link — and it is what makes pulling both laptop inputs live fleet-wide viable (~2.4 Gbps worst split at high rates: just inside one link, no headroom). Many current motherboards ship 2.5GbE onboard already; the constraint is more likely the far end of the bond than the server.

Two things upstream of the bond can cap this before the NIC does — neither is about the server, both are worth checking before trusting any figure in this section:

- **The Cloud Gateway's own routing capacity.** Every theatre packet arrives on an uplink VLAN and is *routed* onto Mothership-LAN by the gateway. On a UDM Pro, the 8-port LAN switch is reported to reach the CPU over a single 1 Gbps link on current hardware revisions, capping inter-VLAN routing at ~1 Gbps regardless of LAG (community wiki — not a Ubiquiti spec sheet, verify against the actual unit). The exact model is still unconfirmed — docs/open-questions.md #17.
- **A routing hairpin on the mothership side.** The mothership's containers and the BirdDog Central VM reach the theatre subnets via the Tailscale subnet router at 192.168.1.2 . That path is now implemented as static routes on the Cloud Gateway (config/unifi/network-config.yaml) — the macvlan containers can't use the host as a next hop directly, and the VM uses the gateway as its default route — so every packet crosses the bond twice (container/VM → gateway → back to the host's tailscaled , then out again as WireGuard). For the BirdDog Central NDI fallback that makes the server's transmit ~3,100 Mbps (VM → gateway, then host → theatres), not ~1,560 — over even the bond's aggregate. Mitigations: give the VM its own static routes to the theatre /24 s via 192.168.1.2 (a VM on Unraid's bridge can reach the host directly, unlike a macvlan container), and/or 2× 2.5GbE. The return leg and the ACL side are still open — docs/open-questions.md #18.

RAM

Workload	Estimate	Basis
BirdDog Central VM	8 GB	Routing/control app, not encode
Nextcloud + MariaDB + Redis	6 GB	InnoDB buffer pool + PHP workers + Redis cache
UniFi Controller + MongoDB	3 GB	Manages only the Cloud Gateway (the 14 GL-iNet routers are GLKVM-Cloud's, not UniFi's) — MongoDB's WiredTiger cache doesn't need much at this scale
ATEM Overseer + Flock	5 GB	Unlike the other admin/control-plane containers below, these two are actually decoding video — up to 12 concurrent theatre streams apiece for their monitoring dashboards (see docs/bandwidth-analysis.md), not just pushing config or metadata
Restreamer, NDI Discovery, DERP, both ingest containers, GLKVM-Cloud (rttys+coturn), ATEM Fleet Admin, Tailscale router	9 GB	9 lightweight containers, ~1 GB each budgeted
Unraid OS + ZFS ARC (3 pools, ~9 TB usable combined)	8 GB	Soft ZFS guidance is roughly 1 GB RAM per TB of pool for decent ARC hit rate — this is a floor, not a hard requirement, ZFS is adaptive
Subtotal	~39 GB	Down from ~71 GB before the VMix instance VM (32 GB on its own) was removed from the design
Minimum, with headroom	64 GB	39 GB doesn't map to a clean multi-channel ECC DIMM configuration — 64 GB is the next practical capacity above it with real margin, not just rounding up to the subtotal
Recommended	96 GB	Gives ZFS ARC meaningfully more room across all three pools, and covers any future container additions without revisiting the DIMM population

ECC, not just capacity. All three storage pools above are ZFS — ZFS leans on RAM for checksumming and its ARC read cache, and a bit flip in ordinary (non-ECC) RAM can get silently written into a checksum or into the recording pool's parity/mirror data before anyone notices, which defeats the whole point of using ZFS for the one pool holding unrepeatable footage. ECC RAM catches and corrects that class of error before it propagates. Pair this with a platform

that has *validated* ECC support (see the CPU section below) — ECC only protects against silent corruption if it's actually enabled and working, which isn't guaranteed on every board that merely accepts ECC modules electrically.

CPU

Workload	Estimate	Basis
BirdDog Central	2 cores	Routing/control, not encode
Docker stack (12 containers + the Tailscale subnet router + 3 DBs)	5-7 cores shared	Mostly I/O-bound; Restreamer's SRT relay (remux, not transcode, per this design's primary path) and ATEM Overseer + Flock (each decoding up to 12 concurrent monitoring streams) are the heaviest individual containers
Unraid OS/array overhead	2 cores	
Subtotal	9-11 cores	Down from 17-19 before the VMix instance VM (8 dedicated cores on its own) was removed from the design
Minimum	12 cores / 24 threads, high sustained (not just boost) clock	Rounds above the subtotal's high end — see platform-class discussion below, which matters more than squeezing out the last core or two

The platform class matters more than the core count, and this is worth spelling out rather than just naming a chip. A 16-core desktop CPU is easy to find — the actual constraint is everything *around* it:

- **PCIe lanes.** With up to 6 NVMe drives across the two NVMe pools above (2 in the container/VM mirror, 2-4 in the recording pool; the content pool is SATA), plus the 10GbE edit-LAN NIC (typically ×4 or ×8), this box still wants 20-30+ usable PCIe lanes — though the pressure has eased since the VMix VM (and with it a mandatory ×16 GPU slot) left the design. Mainstream consumer desktop platforms (the socket a typical Ryzen 9 or Core i9/Ultra 9 sits in) are now workable with careful drive placement and bifurcation; a **workstation/HEDT-class platform** (Threadripper PRO or a high-end Xeon W) still clears it without any compromise, and remains the safer choice — but it's now a preference with a real mainstream alternative, no longer the only category that fits.
- **Validated ECC.** Some mainstream consumer boards technically accept ECC memory electrically, but whether ECC actually *functions* — gets detected, enabled, and does correction — varies board-to-board and isn't something to gamble on for the pool holding irreplaceable footage. Workstation platforms build ECC support into the platform itself, officially validated, not a maybe.
- **Sustained clock over peak boost.** The heavy loads here are continuous, not bursty — ATEM Overseer and Flock each decode up to 12 monitoring streams for the whole show, and the ingest pipelines run all day — so a CPU's *base* clock under sustained all-core load is a better predictor than its headline single-core boost number. Worth actually comparing base clocks across whichever specific chips are shortlisted, not just core count.

Trade-off worth being upfront about: workstation/HEDT platforms cost meaningfully more (CPU + motherboard together) than the mainstream desktop alternative, and idle power draw for 24/7 operation runs higher too. That's a real cost, not a rounding error. Since the VMix VM left the design, a high-end mainstream build with *validated* (not merely tolerated) ECC support is a legitimate cheaper alternative — the deciding check is the ECC validation and having enough lanes for the NVMe pools, not the platform label.

(IOMMU/VT-d support — previously a hard requirement here for GPU passthrough to the VMix VM — is no longer required by anything in the design. Virtually all current platforms have it anyway; it just no longer needs specific confirmation.)

GPU — no longer required

An earlier revision of this design carried a mothership VMix instance VM with a passthrough GPU, and this section specified a workstation-class NVENC card for it. **That VM has been removed** (docs/open-questions.md #10 — its role was never established; the theatre program feeds originate from the VMix *node* PCs, which have their own hardware). With it gone, nothing on this box needs a GPU:

- **Restreamer** relays SRT by remuxing, not transcoding — no encode.
- **BirdDog Central** is NDI routing/control — no encode.

- **ATEM Overseer and Flock** decode up to 12 monitoring/preview streams each, but these are 10 Mbps H.264 previews — CPU decode at this scale is already budgeted in the CPU table above.

The "decent" discrete GPU already in the box (`docs/topology.md`) can stay as spare capacity — potentially useful later for container-level decode/encode offload (e.g. if Overseer/Flock support hardware acceleration, or a future transcode workload appears) — but the spec no longer *requires* any GPU, and no replacement or upgrade should be budgeted for one.

Summary — minimum spec to check the existing server against

Component	Minimum	Why this class
CPU	12 cores / 24 threads, strong sustained clock — workstation/HEDT platform preferred, high-end mainstream with validated ECC acceptable	PCIe lane count for up to 6 NVMe drives, validated ECC support; lane pressure eased since the VMix VM (and its x16 GPU slot) left the design
Motherboard	Validated ECC support, enough PCIe 4.0/5.0 lanes for the 2 NVMe pools (up to 6 drives) plus the 10GbE NIC	Same reasoning as CPU — the two are a package
RAM	64 GB minimum, 96 GB recommended, ECC, populate all available channels	ZFS data integrity across all 3 pools; subtotal ~39 GB since the VMix VM's 32 GB left the budget
GPU	None required — the on-hand GPU stays as spare capacity only	Nothing in the design encodes on this box any more; see the GPU section
Container/VM pool	500 GB usable, mirrored, consumer-grade DRAM-cached NVMe	Latency-sensitive VM/DB workload; consumer endurance is more than enough
Content pool	500 GB usable, mirrored, consumer-grade DRAM-cached SATA SSD	Gentle Nextcloud file I/O; no case for spending more
Recording pool	8 TB usable, mirrored (in practice 2× 7.68 TB ≈ 7.7 TB, or 2× 15.36 TB if 8 TB is a hard floor — see the drive-size note above), enterprise-grade NVMe with power-loss protection	PLP is the deciding factor — this pool is the authoritative archive (the edit-suite NAS dual-write copy may be a subset, and may be RAID 0 — see <code>docs/live-editing.md</code>); confirm event duration against the capacity table above first
Network	2× 1GbE, bonded LACP (already decided) — 2× 2.5GbE strongly preferred, required if laptop ISO inputs are pulled live fleet-wide — plus a separate edit-LAN-facing interface (10GbE) for the dual-write leg	2× 1GbE fits normal operation at the default live scope (camera + program) comfortably; the mass-NDI-fallback worst case fits in aggregate but depends on the LACP hash splitting ~14 WireGuard flows evenly (or on pausing ATEM ingest fleet-wide). 2× 2.5GbE removes that dependency. Either way, the edit-suite dual-write must never ride the theatre-facing bond, and the gateway's routing capacity and the mothership-side routing path (open-questions #17, #18) must be checked

Everything above is a calculated **target**, not a purchase order — the actual box is already on hand per `docs/topology.md` . Next step is checking its real spec against this table and updating `docs/open-questions.md` #1 with the result.

IP address map

Authoritative, fully-instantiated device inventory for the whole network — every physical and virtual device, with its real IP. This is the source of truth; other docs reference this file rather than repeating addresses.

Uplink VLANs (Cloud Gateway transit networks — WAN side of each theatre's GL-iNet)

These transit subnets are a proposal — tags and ranges can be changed freely before deployment. They exist purely so each of the 4 physical VLAN groups has a working transit subnet; GL-iNet WAN ports don't need a predictable static address, DHCP is fine, since Tailscale rides over whatever address they get.

VLAN	802.1Q tag	Subnet	Cloud Gateway address	Serves
Uplink 1	101	10.10.1.0/24	10.10.1.1	Theatre 1–3 + VMix Node 1 GL-iNet WAN ports
Uplink 2	102	10.10.2.0/24	10.10.2.1	Theatre 4–6 + VMix Node 2 GL-iNet WAN ports
Uplink 3	103	10.10.3.0/24	10.10.3.1	Theatre 7–9 GL-iNet WAN ports
Uplink 4	104	10.10.4.0/24	10.10.4.1	Theatre 10–12 GL-iNet WAN ports

Mothership LAN — 192.168.1.0/24

Device	IP	Notes
Cloud Gateway (LAN gateway)	192.168.1.1	
Unraid server (host management IP)	192.168.1.2	Tailscale subnet router container shares this IP (host networking)
BirdDog Central	192.168.1.12	Windows VM — the design's only VM (the mothership VMix instance VM at <code>.11</code> was removed, see <code>docs/open-questions.md</code> #10; <code>.11</code> is left unassigned)
Nextcloud	192.168.1.13	Docker container, macvlan
Restreamer	192.168.1.14	Docker container, macvlan
NDI Discovery Server	192.168.1.15	Docker container, macvlan, port 5959
DERP server	192.168.1.16	Docker container, macvlan, port 443 + STUN 3478/udp — see <code>docs/tailscale.md</code>
ATEM ISO Ingest	192.168.1.17	Docker container, macvlan — pulls all 12 theatres' ATEM ISO recordings via FTP, see <code>docs/atem-iso-ingest.md</code>
VMix Record Ingest	192.168.1.18	Docker container, macvlan — pulls all 4 VMix PCs' recordings via SMB, see <code>docs/vmix-record-ingest.md</code>
UniFi Controller	192.168.1.19	Docker container, macvlan, port 8443 (UI) + 8080 (device inform) — self-hosted UniFi Network Application, independent of cloud.ui.com
GLKVM-Cloud (<code>rttys</code>)	192.168.1.20	Docker container, macvlan, port 443 (UI) + 5912 (device) + 10443 (WebSocket) — self-hosted remote administration for the 14 GL-iNet routers, see <code>docs/glkvm-cloud.md</code>
ATEM Overseer	192.168.1.21	Docker container, macvlan — fleet monitoring/tally dashboard for all 12 theatres' ATEMs, receives each theatre's monitoring stream (see <code>docs/bandwidth-analysis.md</code>)
GLKVM-Cloud (<code>coturn</code>)	192.168.1.22	Docker container, macvlan, port 3478 tcp+udp — TURN relay for the above
ATEM Fleet Admin	192.168.1.23	Docker container, macvlan — bulk ATEM provisioning (model-aware config forms → live network apply via <code>atem-connection</code> , or XML+media folder export)
Flock	192.168.1.24	Docker container, macvlan — BirdDog Play fleet manager (LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits); receives each theatre's BirdDog Play preview stream, see <code>docs/bandwidth-analysis.md</code>
Ready Room PC 1	192.168.1.30	
Ready Room PC 2	192.168.1.31	
Ready Room PC 3	192.168.1.32	
Ready Room PC 4	192.168.1.33	

Theatres 1–12

Same 8-device pattern in every theatre; only the subnet octet (X) changes. GL-iNet Slate AX LAN 1 goes directly to the ATEM Mini Extreme ISO, LAN 2 feeds the Netgear switch carrying the rest (including BirdDog Play) — see `docs/topology.md` for the physical wiring.

Theatre 1 — 192.168.2.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.2.1
BirdDog Play	192.168.2.20
ATEM Mini Extreme ISO	192.168.2.2
PowerPoint Main	192.168.2.5
PowerPoint Backup	192.168.2.6
VT Main	192.168.2.7
VT Backup	192.168.2.8
Control laptop	192.168.2.10

Theatre 2 — 192.168.3.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.3.1
BirdDog Play	192.168.3.20
ATEM Mini Extreme ISO	192.168.3.2
PowerPoint Main	192.168.3.5
PowerPoint Backup	192.168.3.6
VT Main	192.168.3.7
VT Backup	192.168.3.8
Control laptop	192.168.3.10

Theatre 3 — 192.168.4.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.4.1
BirdDog Play	192.168.4.20
ATEM Mini Extreme ISO	192.168.4.2
PowerPoint Main	192.168.4.5
PowerPoint Backup	192.168.4.6
VT Main	192.168.4.7
VT Backup	192.168.4.8
Control laptop	192.168.4.10

Theatre 4 — 192.168.5.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.5.1
BirdDog Play	192.168.5.20
ATEM Mini Extreme ISO	192.168.5.2
PowerPoint Main	192.168.5.5
PowerPoint Backup	192.168.5.6
VT Main	192.168.5.7

Device	IP
VT Backup	192.168.5.8
Control laptop	192.168.5.10

Theatre 5 — 192.168.6.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.6.1
BirdDog Play	192.168.6.20
ATEM Mini Extreme ISO	192.168.6.2
PowerPoint Main	192.168.6.5
PowerPoint Backup	192.168.6.6
VT Main	192.168.6.7
VT Backup	192.168.6.8
Control laptop	192.168.6.10

Theatre 6 — 192.168.7.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.7.1
BirdDog Play	192.168.7.20
ATEM Mini Extreme ISO	192.168.7.2
PowerPoint Main	192.168.7.5
PowerPoint Backup	192.168.7.6
VT Main	192.168.7.7
VT Backup	192.168.7.8
Control laptop	192.168.7.10

Theatre 7 — 192.168.8.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.8.1
BirdDog Play	192.168.8.20
ATEM Mini Extreme ISO	192.168.8.2
PowerPoint Main	192.168.8.5
PowerPoint Backup	192.168.8.6
VT Main	192.168.8.7
VT Backup	192.168.8.8
Control laptop	192.168.8.10

Theatre 8 — 192.168.9.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.9.1
BirdDog Play	192.168.9.20
ATEM Mini Extreme ISO	192.168.9.2
PowerPoint Main	192.168.9.5
PowerPoint Backup	192.168.9.6
VT Main	192.168.9.7
VT Backup	192.168.9.8
Control laptop	192.168.9.10

Theatre 9 — 192.168.10.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.10.1
BirdDog Play	192.168.10.20
ATEM Mini Extreme ISO	192.168.10.2
PowerPoint Main	192.168.10.5
PowerPoint Backup	192.168.10.6
VT Main	192.168.10.7
VT Backup	192.168.10.8
Control laptop	192.168.10.10

Theatre 10 — 192.168.11.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.11.1
BirdDog Play	192.168.11.20
ATEM Mini Extreme ISO	192.168.11.2
PowerPoint Main	192.168.11.5
PowerPoint Backup	192.168.11.6
VT Main	192.168.11.7
VT Backup	192.168.11.8
Control laptop	192.168.11.10

Theatre 11 — 192.168.12.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.12.1
BirdDog Play	192.168.12.20
ATEM Mini Extreme ISO	192.168.12.2
PowerPoint Main	192.168.12.5
PowerPoint Backup	192.168.12.6
VT Main	192.168.12.7
VT Backup	192.168.12.8
Control laptop	192.168.12.10

Theatre 12 — 192.168.13.0/24

Device	IP
GL-iNet Slate AX (LAN gateway)	192.168.13.1
BirdDog Play	192.168.13.20
ATEM Mini Extreme ISO	192.168.13.2
PowerPoint Main	192.168.13.5
PowerPoint Backup	192.168.13.6
VT Main	192.168.13.7
VT Backup	192.168.13.8
Control laptop	192.168.13.10

VMix nodes

Each has its own router — GL-iNet Slate AX (GL-AXT1800), same model as the theatre routers — and its own Tailscale connection, not part of the theatre it physically sits next to.

VMix Node 1 — 192.168.20.0/24 (off Theatre 1)

Device	IP
Node 1 router (GL-iNet Slate AX)	192.168.20.1
BirdDog P400 Camera 1	192.168.20.11
BirdDog P400 Camera 2	192.168.20.12
BirdDog P400 Camera 3	192.168.20.13
BirdDog P400 Camera 4	192.168.20.14
VMix PC 1	192.168.20.21
VMix PC 2	192.168.20.22

VMix Node 2 — 192.168.21.0/24 (off Theatre 4)

Device	IP
Node 2 router (GL-iNet Slate AX)	192.168.21.1
BirdDog P400 Camera 1	192.168.21.11
BirdDog P400 Camera 2	192.168.21.12
BirdDog P400 Camera 3	192.168.21.13
BirdDog P400 Camera 4	192.168.21.14
VMix PC 1	192.168.21.21
VMix PC 2	192.168.21.22

Edit suite — 192.168.22.0/24

Post-production subsystem at the mothership — dedicated 10GbE LAN, not part of the Tailscale mesh (same room as the mothership, no reason to route it over the tailnet). See `docs/live-editing.md` for the full design.

Device	IP	Notes
Edit suite NAS	192.168.22.2	Blackmagic Cloud Store or generic 10GbE NAS — see <code>docs/live-editing.md</code> Decision 1
MacBook Pro — Editor 1	192.168.22.11	10GbE via Thunderbolt adapter/dock
MacBook Pro — Editor 2	192.168.22.12	10GbE via Thunderbolt adapter/dock
Mac mini — Project Server + Remote Render	192.168.22.20	Dedicated always-on machine, never an editor's own laptop — see <code>docs/live-editing.md</code> Decision 2

Device count

- Mothership: 19 devices (1 gateway + 1 server host + 1 VM + 12 containers + 4 ready-room PCs)
- Theatres: 12 × 8 devices = 96
- VMix nodes: 2 × 7 devices = 14
- Edit suite: 4 devices (NAS + 2 MacBook Pros + Mac mini Project Server/Remote Render node), see `docs/live-editing.md` Decision 2
- **Total: 133 devices** across the network (excluding the 4 uplink VLAN transit addresses, which aren't per-device).

Streaming flow — SRT / NDI

Primary path (SRT)

```
VMix PCs (both nodes) --SRT--> Restreamer (mothership) --SRT (fan-out)--> BirdDog Play x12 (one per theatre)
```

SRT is unicast-by-IP, so this path has no discovery dependency and should route cleanly over the tailnet.

Set the SRT payload size to 1128 bytes on every sender (VMix's SRT outputs and each Restreamer SRT output). SRT's default 1316B payload makes a 1360B IP packet, which is larger than Tailscale's 1280B tunnel MTU, so default-sized SRT packets would be fragmented (or dropped if DF is set) entering the tunnel. 1128B (6 × 188B MPEG-TS packets) gives a 1172B packet that fits. See [docs/bandwidth-analysis.md](#).

Backup path (NDI)

```
BirdDog Central (mothership) --NDI--> BirdDog Play x12 (one per theatre)
```

How Central gets the program — NDI redistribution (settled). Central works as an NDI *redistributor*: the source VMix node PC's native NDI output registers with the NDI Discovery Server, Central receives it over that node's own uplink (~130 Mbps against the node's Slate AX ceiling — estimated ~150-250 Mbps under Tailscale, unbenchmarked — a **fallback-only** load — pausing that node's VMix record ingest for the duration keeps it under the ceiling, the same lever the theatre side already uses), and Central re-sends it to the theatre PLAYS. Two researched facts pin this shape:

- **BirdDog Central is NDI-only on the input side** — its own user guide lists NDI sources, an NDI file-player, and an NDI re-transmitter; there is no SRT/RTMP ingest anywhere in it (Central 2.0 User Guide). So the feed into Central must already be NDI — it can't terminate the SRT program.
- **Restreamer can't be the NDI sender instead** — upstream FFmpeg removed NDI support in 2019 over a NewTek GPL violation and never reinstated it; datarhei's own FFmpeg build contains no NDI library and the maintainers have declined to ship it for licensing reasons (datarhei/restreamer discussion #391).

Remaining verification items (not design questions) are tracked as [docs/open-questions.md](#) #16. BirdDog Central was deliberately retained for this sender role when its fleet-management duties moved to Flock — see [docs/birddog-play-rationale.md](#).

Bandwidth note: full NDI (not HX) at 1080p50 is ~125 Mbps per stream, roughly constant regardless of content — 12-22x SRT's figures, and without SRT's "static content is cheap" discount. Fine for a single theatre falling back — *provided that theatre's ATEM ISO ingest is paused for the duration*, since NDI plus even the default camera + program ingest (~213 Mbps combined) is over the bottom of the theatre's own Slate AX Tailscale estimate (~150-250 Mbps) — and a real capacity question if this path ever has to carry all 12 theatres at once (e.g. a Restreamer failure). See [docs/bandwidth-analysis.md](#) for the full impact at per-router, per-VLAN, and mothership-NIC level.

Caveat, and the fix: NDI relies on mDNS multicast for discovery, which does not cross a Tailscale-routed boundary — Tailscale is point-to-point WireGuard and doesn't forward multicast/mDNS (still an open feature request, not something MagicDNS papers over). Plain local mDNS discovery will not find BirdDog Central across theatres on its own.

Plan: run an NDI Discovery Server on the mothership. NDI 5+ added this specifically for WAN/VPN scenarios — it's unicast TCP (default port **5959**), so it works fine across the tailnet with no multicast involved:

- Run the discovery server as a small service on the mothership, alongside BirdDog Central.

- Point every NDI participant at that server's IP: the VMix node PCs (senders of the program feed Central redistributes), BirdDog Central (receiver of that feed *and* sender toward the theatres), and all 12 BirdDog Play units (receivers). On Windows/macOS this is the NDI Tools Access Manager "Advanced" tab; on Linux it's `~/.ndi/ndi-config.v1.json` .
- Once a sender has a discovery server configured, it stops using mDNS entirely — it's visible only to finders pointed at the same server.
- Tailscale MagicDNS can supply a stable hostname for the discovery server's address instead of hardcoding its Tailscale IP, if the receiving device's config field accepts a hostname — a convenience, not a substitute for the discovery server itself.

Confirmed on BirdDog Play: BirdUI (the PLAY's web admin — browse to its IP, default password `birddog`) has a Network panel with NDI-specific settings including a Discovery Server toggle. Switch it ON, enter a comma-delimited list of Discovery Server IP address(es), click APPLY. BirdDog's own docs describe this as the mechanism for locating NDI sources "on different subnets" — exactly this cross-theatre case — and it supports multiple servers for failover. Sources: BirdDog PLAY User Guide, BirdUI User Guide. Change the default admin password before the event — BirdUI grants full device config access.

See `diagrams/streaming-flow.svg` — primary path solid, backup path dashed. For why BirdDog Play is the theatre-side receiver at all (vs. a laptop

- VLC), see `docs/birddog-play-rationale.md` .

Why BirdDog Play (not a laptop + VLC) for theatre playback

Each theatre's screen is driven by a **BirdDog Play** — a dedicated, purpose-built NDI/SRT decoder — rather than a general-purpose laptop running a media player. This documents the reasoning, alongside the closest realistic alternative.

Zero-config in the theatre

BirdDog Play needs no on-site setup beyond physical connection: plug it into the theatre screen and the network, point it at a source, done. No OS to patch, no application to configure per-session, nothing for on-site staff to babysit beyond power and a cable. That matters at this scale — 12 theatres, unattended for most of a live event.

Protocol support: SRT primary, NDI backup, both natively

- **SRT** — the primary path in this design (see `docs/streaming-flow.md`) is bandwidth-efficient by design (long-GOP H.264, see `docs/bandwidth-analysis.md` for real figures). In this design it originates from VMix via the mothership's Restreamer, but the protocol keeps options open: the ATEM Mini Extreme ISO also has a **built-in hardware streaming engine** that can stream SRT directly over Ethernet with no computer involved (Blackmagic tech specs), unused here but available as a fallback origin. BirdDog Play receives either without caring which one is sending.
- **NDI** — the backup path (BirdDog Central → BirdDog Play, via the NDI Discovery Server — see `docs/streaming-flow.md`) is handled natively too, with no separate plugin or configuration step.

Remote control from the mothership

None of these require physically visiting a theatre — and the division of labour is deliberate, not three redundant tools:

- **Flock** — the **fleet-management tool in this design**: LAN discovery, tag-based grouping, full BirdUI-parity settings per device, and batch edits across a group (Rust + Docker). Purpose-built for managing many BirdDog Play units as one fleet rather than 12 individual devices, which is precisely this deployment's shape.
- **BirdUI** — each unit's own web admin, reachable directly, as the per-unit fallback (already the mechanism used to configure the NDI Discovery Server setting — `docs/streaming-flow.md`).
- **BirdDog Central** — retained in the design as a mothership VM (`docs/topology.md`) **for the NDI backup path's sender side, not as a management layer** — Flock covers the management job on its own. Central's routing/control console exists as a bonus, not a dependency.

Comparison: BirdDog Play vs. laptop + VLC

The realistic alternative is a laptop in each theatre running VLC, pointed at the same SRT/ NDI sources.

	BirdDog Play	Laptop + VLC
Setup	Plug in, done	Install/patch OS, install VLC, configure stream URL per source
SRT	Native, purpose-built decode	Native from VLC 3+ (SRT CookBook) — the one protocol VLC handles cleanly
NDI	Native, purpose-built decode	Requires a separate third-party plugin (NDI for VLC); reliably <i>outputting</i> VLC's playback as NDI is well-documented, but consistently <i>receiving and playing</i> an NDI source as input is far less so
Remote management	Flock (fleet-wide) with per-unit BirdUI — purpose-built for this device	Nothing built in — would need RDP/VNC/TeamViewer bolted on, none of it purpose-built for stream monitoring/control
Unattended reliability	Broadcast-grade embedded device, designed to run 24/7 unattended	General-purpose OS: updates, driver issues, VLC hangs on a dropped stream sometimes need a manual restart

	BirdDog Play	Laptop + VLC
Recovery from a dropped stream	Automatic reconnect	Not guaranteed — depends on VLC settings, may need manual intervention
Footprint/power	Small, low power, unobtrusive	Full laptop: bigger, hotter, more power, another thing that can be bumped/closed/updated mid-show
Attack surface	Purpose-built embedded device	Full general-purpose OS — much larger surface
Cost per unit	Lower, purpose-built	A laptop is not free even if "already owned" — it's now unavailable for anything else, and carries more ongoing maintenance

Recommendation

BirdDog Play, as already implemented throughout this design. The laptop + VLC path is real and technically works for SRT specifically, but loses on every axis that matters for 12 unattended theatres running for the duration of a live event: zero-config deployment, clean NDI support, purpose-built fleet management (Flock, with per-unit BirdUI), and broadcast-grade reliability without needing on-site intervention when something drops.

Why GL-iNet — segmentation, Tailscale overlay, multi-WAN, and beyond this event

This design already uses GL-iNet Slate AX (GL-AXT1800) routers throughout (previously A-1300s, replaced 2026-10-06 — see the comparison below; also [docs/topology.md](#), [docs/open-questions.md](#)). This document is the fuller "why" — the architectural reasoning behind putting a small router in every theatre instead of a flatter network, why Tailscale is the routing layer between them rather than site-to-site VLANs alone, the WAN-flexibility features this build doesn't currently use but is worth knowing about, and — since this is genuinely reusable hardware, not a one-event purchase — where else this same GL-iNet + Tailscale pattern earns its keep beyond a 12-theatre live event.

Network segmentation: one subnet per theatre

Each theatre is its own isolated /24 (192.168.2.0/24 through 192.168.13.0/24 — see [docs/ip-address-map.md](#)), NAT'd behind its own GL-iNet router, rather than 12 theatres' worth of devices sharing one flat network. Three concrete benefits fall out of that, beyond the obvious "keeps the IP plan tidy":

Individual DHCP servers. Each router runs its own DHCP scope for its own theatre. A DHCP problem — exhaustion, a rogue second DHCP server accidentally plugged in, a lease conflict — is contained to that one theatre's ~8 devices. On a flat network with one central DHCP server, the same fault takes out DHCP for the entire event at once. With 12 theatres + 2 VMix nodes + the mothership as 15 separate broadcast domains on the theatre-facing network (the edit suite's own 10GbE LAN is a further one — see [docs/live-editing.md](#)), there are 15 separate DHCP fault domains instead of one.

Multicast/broadcast containment. mDNS, SSDP, and NDI's own local discovery are all broadcast/multicast-based, which only works within a single L2 broadcast domain. Flatten 12 theatres onto one network and multicast traffic doesn't just fail to reach where it's needed — it also *floods everywhere it isn't*, since every discovery packet from every theatre reaches every other theatre's devices too. This design already runs into exactly that limitation on purpose: NDI's backup path needed a **unicast** NDI Discovery Server (see [docs/streaming-flow.md](#)) precisely because Tailscale's point-to-point routing (like any routed, non-bridged network) doesn't carry multicast across the theatre boundary. Segmentation isn't just tolerating that limitation — it's the reason cross-theatre multicast noise was never a problem to begin with, for anything that doesn't specifically need to be reachable across theatres.

Security — blast-radius containment. A compromised or simply misbehaving device in one theatre (malware on a presenter's laptop, a flaky ATEM firmware bug, an ARP-spoofing or DHCP-starvation attempt) is contained to that theatre's subnet. It cannot see, scan, or reach another theatre's devices or traffic, because there's no route between them — [config/tailscale-acl.json](#) only permits `theatre → mothership` and `mothership → theatre`, never `theatre → theatre`. This is the same point already made elsewhere in this repo — "VLAN grouping = physical uplink cabling only, cross-theatre isolation is enforced by Tailscale ACLs, not by the VLANs themselves" — worth restating here as the *reason* per-theatre segmentation is worth doing at all: it's not really about the subnets, it's about giving the ACL layer clean boundaries to enforce isolation across.

Tailscale as the overlay routing layer

Segmentation alone doesn't connect the theatres to the mothership — something has to route between the isolated subnets on purpose, while still keeping them isolated from each other. That's Tailscale's job here (full detail in [docs/tailscale.md](#)), and it's worth being explicit about why an overlay mesh is the right tool for that instead of relying on the venue's own physical network (VLAN trunking, static routes, a traditional site-to-site VPN concentrator):

- **Direct point-to-point tunnels, not routed via one central concentrator.** Every node-to-node WireGuard connection is negotiated independently and prefers a direct path when one exists — no single choke point every packet has to pass through, which matters for something latency-sensitive like SRT.

- **Doesn't depend on the underlying network cooperating.** A traditional VLAN-routed design needs the venue's switches trunking the right VLANs to the right ports, correctly, everywhere. Tailscale's NAT traversal (direct when possible, DERP relay as fallback — see `docs/tailscale.md`) works regardless of what the underlying physical topology actually looks like, which matters a lot when the venue's network isn't something this design controls in the first place (see `docs/bandwidth-analysis.md` — the venue-supplied-VLAN context that document was written against).
- **Isolation enforced at the overlay, not hoped-for at the physical layer.** The ACLs in `config/tailscale-acl.json` are what actually stop theatre-to-theatre traffic — independent of whether the venue's VLANs are configured exactly as planned. If a VLAN turns out flatter than expected, or a switch port is misconfigured, cross-theatre isolation still holds, because it was never depending on the physical segmentation to begin with.
- **No manual routing tables.** Each router advertises its own subnet (`--advertise-routes`) and accepts the others' where ACLs allow (`--accept-routes`) — see `config/tailscale-up-all-devices.sh`. Nobody is hand-maintaining a routing table across 14+ devices.
- **Identity independent of location.** A router's Tailscale identity is its WireGuard key, not an IP address on a specific network. That matters directly for the multi-WAN/failover discussion below: a router can switch from its wired uplink to WiFi or cellular mid-event, and every other node on the tailnet keeps reaching it under the same identity, with no reconfiguration anywhere else.

Multi-WAN: wired primary, WiFi/cellular failover

This build's current default is a single hardwired WAN per theatre (see `docs/topology.md`) — but GL-iNet's broader lineup supports genuine multi-WAN failover: multiple uplink sources (wired Ethernet, WiFi acting as a WAN client, a USB cellular modem) with automatic failover between them, no manual cable-swap or on-site intervention required. This is a firmware 4.x feature (**NETWORK → Multi-WAN**: Failover or Load Balance across Ethernet, Repeater, Tethering and Cellular) that even the small A-1300's own user guide documents, not just the bigger models — what differs per model is which WAN *media* it has to fail over to (neither the A-1300 nor the Slate AX has a built-in modem, so cellular means a USB dongle or a tethered phone) (GL.iNet Multi-WAN guide, A-1300 user guide). For a deployment where most theatres run unattended for long stretches, that matters — a lost wired connection mid-session isn't something someone can necessarily walk over and fix before it costs real time.

Two failover directions are both worth having, and they're mirror images of each other:

- **Wired as primary, WiFi/cellular as backup** — this build's actual recommendation. If the venue's wired drop to a theatre fails (a bad cable, a switch fault, an ISP issue on the venue's side), the router falls back to venue WiFi or a cellular USB modem automatically. Bandwidth degrades under failover — that's expected and fine, it's a "keep the tailnet reachable and keep functioning at reduced quality" fallback, not a silent full-quality substitute.
- **WiFi as primary, for events that don't need this build's full streaming load.** Not every event this hardware gets reused for will run 12 theatres' worth of continuous SRT/NDI simulcast and near-real-time ISO ingest. For a lighter event — say, a conference room doing PowerPoint sync and basic control traffic, no live camera program feed — venue WiFi alone may comfortably cover it, and using each theatre's GL-iNet as a WiFi client eliminates the need for a wired drop to every room at all. Faster to set up, and avoids paying for (or waiting on) cabling runs the event genuinely doesn't need at that load level. This is the same "match infrastructure to actual load" reasoning `docs/bandwidth-analysis.md` already applies to VLAN count, just applied to WAN medium instead.

Limiting devices on pay-per-device venue WiFi

Some venues charge for WiFi **per connected device**, not by bandwidth tier. Connect every individual device in a theatre — the ATEM, BirdDog Play, both PowerPoint laptops, both VT laptops, the control laptop — directly to venue WiFi, and that's up to 7 billable devices per theatre. Put one GL-iNet router in each theatre instead, and the venue sees **one** device — the router — while everything behind it reaches the internet NAT'd through that single connection.

Worked example — real numbers from two major London exhibition venues' published 2025/2026 rate cards (cited as market examples of the pay-per-device model, not as this design's venue):

	ExCeL London	Olympia London (via eForce)
Single-device WiFi (per device)	£128 (advance) – £184 (late)	£171 (standard) – £205 (late)
Smallest bulk tier	10 devices: £647–932	6 devices: £519–623
7 devices, billed individually	£896–1,288	7×£171–205 = £1,197–1,435
7 devices, cheapest real bulk path	10-pack (over-buying): £647–932	6-pack + 1 single: £690–828
7 devices, hidden behind 1 router	£128–184	£171–205

Per theatre, that's roughly **5-7x cheaper** at ExCeL and **4-7x cheaper** at Olympia (the low end comparing against the cheapest bulk path, the high end against individual billing). Across all 14 GL-iNet routers in this design (12 theatres + 2 VMix nodes, each also with several devices behind it — see `docs/ip-address-map.md`), the gap runs into the thousands of pounds for the event, not a marginal saving.

Important — this isn't a loophole to just quietly exploit. Both venues' published technical policies explicitly prohibit exhibitors from bringing their own WiFi access points, mesh systems, or mobile hotspots onto the show floor without prior written authorization, and both reserve the right to detect and disconnect unauthorized hardware — this is standard, close-to-identical boilerplate at both ExCeL and Olympia, not an oversight. Using a GL-iNet router this way is a legitimate, common arrangement for professional AV vendors — but it means getting the venue's technical services team to authorize it ahead of time (registering the router, agreeing it's not itself broadcasting a public/guest SSID, etc.), not simply plugging one in on the day.

Sources: ExCeL Event Services Rate Card 2026 — IT, ExCeL Wireless Policy, London Book Fair 2025 eForce Internet Order Form (Olympia). Prices change; treat these as illustrative of the *shape* of the saving, confirm current figures with the actual venue before budgeting against them.

GLKVM-Cloud and remote administration

Already built into this design — see `docs/glkvm-cloud.md` for the full detail. In short: a self-hosted instance of GL.iNet's own GLKVM-Cloud platform gives centralized, browser-based SSH terminal and web-admin proxy access to all 14 Slate AX routers at once, using GLKVM-Cloud's documented support for embedded OpenWrt devices — not its flagship KVM-hardware feature set, since there's no physical KVM unit in this design. Worth restating here as part of the broader case for this hardware family specifically: choosing GL-iNet doesn't just get 14 capable routers, it gets a fleet that's centrally manageable as one console instead of 14 separate admin sessions, for free, using tooling the vendor already publishes and this design already self-hosts.

Beyond this event: other places this pattern is useful

None of the following is used in this build — worth documenting anyway, since this is reusable hardware and the same GL-iNet-router-plus-Tailscale pattern generalizes well beyond a 12-theatre conference.

Remote administration of LED video processors (Novastar)

Novastar's LED processor/controller line is normally driven by a laptop physically present on the same local network as the hardware. A GL-iNet router with Tailscale turns that into remote access from anywhere — same principle as GLKVM-Cloud giving remote access to this design's own routers, applied to a different vendor's hardware.

Novastar MX30, via VMP. VMP (Vision Management Platform — not "Video Management Platform") is Novastar's control app for the COEX line the MX30 belongs to. Its own documentation describes adding a controller by **typing its IP address directly** ("Add Controller → enter the IP"), not by picking it off a broadcast-discovered list, and explicitly documents a router-mediated topology as one of its two supported connection methods ("PC and controller connected to the same LAN via a router... suitable for systems in complex environments"). Control traffic runs over TCP port 5200 (UDP 5201 for the UDP variant), per Novastar's own Central Control Protocol documentation — a unicast, IP-addressed protocol, not a broadcast-dependent one, which is exactly the traffic pattern that crosses a Tailscale tunnel cleanly. VMP's manual never explicitly addresses VPN/WAN use (neither endorsing nor ruling it out) — the underlying protocol lining up cleanly with

how Tailscale actually works is a real signal, not a confirmed guarantee. Novastar's own sanctioned path for genuine internet-native access is a separate product, **VNNOX** (a hosted cloud platform) — worth knowing about as the vendor's alternative, though it's a different management layer entirely, not "VMP over the internet." Sources: VMP User Manual, COEX Wiki — Device Connection and Setting, Central Control Protocol Instructions V1.5.0.

A note on "MCTRL880": that specific model doesn't exist in Novastar's current or past product catalog — the closest real sending cards are the **MCTRL660 / MCTRL660 Pro, MCTRL600, and MCTRL4K**. The rest of this section uses MCTRL4K, since that's also the model the Tailscale question below is about specifically; the same reasoning applies to the MCTRL660/600 family.

NovaLCT + MCTRL4K over Tailscale — checked specifically, genuinely uncertain, not a confirmed "yes." No official Novastar documentation, forum thread, or case study addresses this exact combination — worth being upfront about that rather than asserting it works. What's actually known, and why it points toward "probably, with a caveat":

- NovaLCT ships in different variants for different Novastar product families. For Novastar's **Multimedia Player** line (standalone async players, not sending cards), the official manual explicitly documents a cross-subnet workaround: *"If the terminal and NovaLCT are not on the same network segment but they can be pinged... select Specify IP, enter an IP address and click Search."* That's a confirmed, official, IP-based (non-broadcast) connection path — but for a different product family than the MCTRL sending cards.
- For the **Synchronous Control System** variant of NovaLCT — the one that actually talks to MCTRL4K/660/600-class sending cards — the manual only says NovaLCT "connects to the sending card automatically" once the hardware connection is normal, with no equivalent "Specify IP across a routed network" language found in any version checked. That phrasing suggests the initial device-discovery step may depend on local broadcast/ARP rather than a directable IP search — and Tailscale, being a routed L3 overlay rather than a bridged L2 network, **does not carry broadcast/multicast traffic between peers** (the same limitation this design already worked around for NDI — see `docs/streaming-flow.md`).
- Once a connection exists, though, MCTRL-family control traffic itself is unicast TCP to a known IP on port 5200 — confirmed by Novastar's own COEX protocol doc and independently corroborated by two separate community reverse-engineering projects (Bitfocus Companion's Novastar module, an independent Wireshark-based protocol writeup). That part should traverse a Tailscale tunnel without issue.

Practical read: the control *traffic* should work fine over Tailscale; the open question is purely whether NovaLCT's own device-*discovery* step for sending cards needs local broadcast to find the MCTRL4K in the first place. If NovaLCT's Synchronous Control System build has its own "add by IP" option (unconfirmed here — check the actual installed version), that sidesteps discovery entirely and is the path to try first. **The documented fallback that doesn't depend on any of this:** the MCTRL4K also exposes its own **web-based configuration UI**, reachable by pointing a browser at its IP — a plain HTTP-to-a-known-address connection with no broadcast dependency, and Novastar's own docs already describe reaching it this way (they just assume same-LAN, not a routed tunnel — no technical reason a browser would care about the difference). If NovaLCT itself doesn't cooperate over Tailscale, the web UI is the thing to actually try.

Remote WiFi access point for mixing desk control apps

A GL-iNet router can also just be a **portable, purpose-brought WiFi access point** — useful anywhere a venue's own WiFi is unreliable, absent, or not trusted for control traffic, letting an engineer run **Yamaha StageMix, Allen & Heath Qu-Pad, or Mixing Station** (a third-party app supporting many console brands) from an iPad without depending on house WiFi at all.

This is a genuinely different configuration mode from everything else in this document, worth being explicit about: everywhere else here, the GL-iNet router creates its **own routed, NAT'd subnet** with its own DHCP (that's the whole point of the per-theatre segmentation this doc opens with). For mixing-desk control, the safe default is the opposite — configure the router as a **plain bridged access point**, putting the iPad on the exact same L2 broadcast domain as the console, not a separate routed subnet behind it. That distinction is the crux of whether each app actually works:

App	Discovery mechanism	Manual IP fallback	Works across a routed subnet?
Yamaha StageMix	None — manual IP entry is the <i>only</i> method, and the app's own compatibility check assumes same-subnet addressing	Yes, but it's not actually a cross-subnet workaround here	Not supported per the official manual — no documented cross-subnet mode at all

App	Discovery mechanism	Manual IP fallback	Works across a routed subnet?
(CL/QL-series consoles)			
Yamaha StageMix (TF-series and newer)	AUTO mode, mDNS-style, gated by iOS's Local Network permission	Yes — an explicit MANUAL mode	Yes, explicitly documented — connects across a different subnet, at the cost of losing live meter data
Allen & Heath Qu-Pad	Device list, almost certainly UDP broadcast per A&H's general control-protocol architecture	Not documented specifically for Qu-Pad (A&H's general platform docs say manual IP is the intended workaround when broadcast is blocked, but this isn't confirmed for Qu-Pad itself)	Probably, but under-documented — bench-test before relying on it live
Mixing Station	Varies by console brand — broadcast-based for Behringer X32/M32 (OSC/UDP) and Soundcraft (HiQNet, where the <i>mixer</i> initiates the connection to the app), inherited native behavior for others	Yes, universally — the app always supports typing a console's IP	Depends on the specific console's own protocol , not on Mixing Station itself

Bottom line: bridged AP mode is the one setup that works identically across all of these, no per-app or per-console workaround needed. A routed subnet is conditionally viable (TF-generation StageMix explicitly supports it; Mixing Station's manual-IP entry helps, but the underlying console protocol still might not) — treat it as something to test deliberately, not assume, if the router's other NAT/firewall features are wanted badly enough to be worth the added risk. For a one-off "get StageMix working from front of house" use case, bridged AP mode is the boring, reliable default. Sources: QL StageMix V8 User Guide, TF StageMix V4.5 User's Guide, Qu Series Reference Guide, Qu-Pad Help Guide, Mixing Station — Getting Started, Mixing Station — Soundcraft/HiQNet.

Mesh and peer-to-peer: fewer physical drops

Worth being precise about two different things both loosely called "mesh" here, since they operate at completely different layers and don't depend on each other:

- **Tailscale's own overlay mesh** — already how every router in this design reaches the mothership and each other, regardless of physical topology (see the Tailscale section above). It doesn't care whether the underlying WAN path is a dedicated wired drop, WiFi, cellular, or a hop through another GL-iNet unit — Tailscale just needs *some* IP reachability, wired or wireless, to establish its own tunnels on top.
- **GL-iNet's radio-layer repeater/WDS mode** — what this section is actually about: physically extending network reach between GL-iNet units over WiFi (or a wired Ethernet daisy-chain) instead of running a separate venue drop to every location. Worth naming accurately: GL-iNet markets this as repeater/extender (WDS) mode, not a self-healing mesh protocol in the eero/Orbi sense — one-directional dependency on whichever unit is upstream, not automatic multi-path failover between mesh nodes.

Running several adjacent theatres off a single network drop. One router lands the venue's wired uplink and acts as the "gateway" unit for a small cluster of adjacent theatres; the rest relay off it wirelessly (repeater mode) or via a wired Ethernet daisy-chain, while each theatre-side router still runs its own isolated subnet and DHCP scope exactly as described earlier in this document — the segmentation and Tailscale ACL isolation between theatres doesn't change, only how many theatres share one physical uplink. This directly trades against the "expensive per VLAN port" reality already central to `docs/bandwidth-analysis.md`: fewer wired drops needed from the venue, at a real cost.

That cost is worth stating plainly, because it partially undoes an argument made earlier in this document. The segmentation section above frames per-theatre subnets as independent fault domains — a DHCP problem or device fault in one theatre can't touch another. Meshing multiple theatres behind one shared uplink **reintroduces a shared fault point at the physical/WAN layer**: if the gateway unit's own uplink fails, every theatre relaying through it loses connectivity together, not just one. The subnets themselves stay isolated from each other (Tailscale ACLs don't change), but they stop being independently *reachable*. Worth treating as a deliberate, scoped trade-off — fewer drops needed, in exchange for correlated failure across whichever theatres share that one drop — not a strictly-better free option. Bandwidth is shared across the group too, same caveat as any repeater hop.

Going fully peer-to-peer, no wired backhaul at all. The extreme version of the same idea: no theatre gets a wired venue drop, and GL-iNet units relay to each other entirely over WiFi, hop by hop, until reaching whichever unit (or units) actually has a real WAN connection. Useful where running cable is genuinely impractical (temporary/pop-up spaces, venues that charge heavily per drop, awkward physical layouts) — but stacks the same shared-fault-point concern from above across every hop in the chain, plus WiFi's own range/interference/multi-hop-latency limits, which get worse the more hops are strung together. Reasonable for a small cluster of nearby rooms; not something to lean on for 12 theatres at once without real testing against the venue's actual RF environment first.

GL-iNet travel router comparison

Prices and specs below are current as of when this doc was written (mid-2026) — GL-iNet runs frequent promo pricing, and this isn't a live-updated table, so treat exact figures as indicative and re-check before procurement. GBP pricing was only reliably obtainable for a couple of models (GL-iNet's regional store pages don't consistently expose static GBP pricing to an automated check) — USD is the more solid figure throughout. The WireGuard column is GL.iNet's own figure for its built-in (kernel) WireGuard client; this design runs **Tailscale**, which uses its own userspace engine (`wireguard-go`) and is markedly slower on the same hardware — e.g. GL-MT1300 ~90 Mbps kernel WireGuard vs ~20 Mbps Tailscale (tailscale#10524), Beryl AX 300 vs ~150 Mbps each way (GL.iNet forum). No Slate AX Tailscale figure has been measured yet, see `docs/open-questions.md` #19.

Model	Price (USD)	WireGuard throughput	Ports	WiFi	Cellular	Multi-WAN failover	Notes
Slate AX (GL-AXT1800)	\$119.99 list	~550 Mbps (Tailscale ~150-250 Mbps combined — estimate, unbenchmarked)	2× GbE LAN, 1× GbE WAN, 1× USB 3.0	WiFi 6	USB dongle only	Yes (firmware 4.x Multi-WAN)	This build's choice since 2026-10-06 — see <code>docs/topology.md</code> . Qualcomm IPQ6000, 4× Cortex-A53 @ 1.2 GHz (64-bit); same 1 WAN + 2 LAN layout as the A-1300, so the wiring is unchanged.
Slate Plus (GL-A1300)	\$70–100 (promo vs. list — [<code>docs/bandwidth-analysis.md</code>] (bandwidth-analysis.md) uses ~\$100)	~170 Mbps (Tailscale est. only ~30-70 Mbps combined)	2× GbE LAN, 1× GbE WAN, 1× USB 3.0	WiFi 5	USB dongle only	Yes (firmware 4.x Multi-WAN)	Previous choice, rejected 2026-10-06: 32-bit IPQ4018 (4× Cortex-A7 ~717 MHz, 256 MB RAM), firmware stuck at 4.5.x with Tailscale 1.58; Go has no optimised ChaCha20-Poly1305 assembly there, so Tailscale can't carry even the default ingest scope. Smallest/cheapest of the group. Fallback only, running kernel WireGuard instead of Tailscale.
Beryl AX (GL-MT3000)	\$99	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1×	WiFi 6	USB dongle only (needs an add-on board for a real slot)	Yes (firmware 4.x Multi-WAN)	Only 1 LAN port — doesn't fit this build's "ATEM gets its own dedicated port" wiring without an added switch.

Model	Price (USD)	WireGuard throughput	Ports	WiFi	Cellular	Multi-WAN failover	Notes
			USB 3.0				
Flint 2 (GL-MT6000)	\$170	~900 Mbps	4× GbE + 2× 2.5G	WiFi 6	None (USB dongle/tethering)	Yes — failover + load-balancing, plus two Ethernet WANs	Highest throughput of the non-cellular models (Tailscale ~300-450 Mbps est.); much larger/heavier, poor fit for 12-per-theatre portability — the headroom option if the Slate AX benchmark comes in low.
Slate 7 (GL-BE3600)	\$150–170	~540 Mbps	1× 2.5G LAN, 1× 2.5G WAN, 1× USB 3.0	WiFi 7 (dual-band only — no 6GHz/320MHz, so limited real-world WiFi 7 benefit)	USB dongle only	Yes, per its own user guide	Newest of the group; compact. Only 1 LAN port — rejected for the same reason as the Beryl AX. (GL.iNet figures seen for its WireGuard rate range ~490-540 Mbps.)
Spitz AX (GL-X3000)	\$380	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 2.0	WiFi 6	Built-in 5G/4G modem, dual Nano-SIM	Yes — Ethernet/repeater/cellular/tethering	No internal battery — needs external 12V power. The dedicated cellular-failover choice if built-in modem matters more than portability.
Puli AX (GL-XE3000)	\$410	~300 Mbps	1× GbE LAN, 1× 2.5G WAN, 1× USB 2.0	WiFi 6	Built-in 5G modem, dual Nano-SIM	Yes	Same cellular capability as Spitz AX, plus a built-in 6400 mAh battery — genuinely portable/untethered for short sessions, at a real price premium.

"Mesh" caveat: every model above advertises **repeater/extender (WDS) mode**, not a true self-healing mesh protocol between multiple units — worth being precise about this distinction rather than assuming parity with consumer mesh systems (eero, Orbi, etc.). See the mesh/peer-to-peer section above for what that means in practice for running several theatres off one drop.

For this build specifically, the Slate AX replaced the Slate Plus (A-1300) on 2026-10-06, after research found the A-1300's ~170 Mbps was GL.iNet's *kernel* WireGuard figure — Tailscale's userspace engine on its 32-bit CPU is estimated at only ~30-70 Mbps combined, short of even the default camera + program ingest scope (~89 Mbps combined, typical, per docs/bandwidth-analysis.md). The Slate AX keeps the same **2 LAN ports for the dedicated-ATEM-port wiring — still the harder constraint than raw throughput**, and why Beryl AX and Slate 7 remain traps regardless of their higher ceilings — on a 64-bit CPU with an estimated ~150-250 Mbps under Tailscale. That estimate is extrapolated from the Beryl AX measurement, not measured: benchmark one unit before buying 14. The scenario that actually bites is still a theatre's NDI fallback plus its ATEM ingest at the same time (~213 Mbps combined, typical) — over the bottom of that range, so the config-only mitigation (pause that theatre's ATEM ingest during the fallback) stays part of the design. Spitz AX/Puli AX are the models worth reaching for specifically *because* of their built-in cellular — i.e. exactly the multi-WAN-failover and WiFi-as-primary-link scenarios earlier in this document, where a wired drop isn't guaranteed or a genuine backup path matters more than shaving cost/size. **Flint 2 is the headroom option** if the Slate AX benchmark disappoints or a further stream is ever proposed: desktop-sized, but comfortable even with ingest running through an NDI fallback.

The Comet KVM range

GL-iNet's separate KVM-over-IP product line — not used in this build (see `docs/glkvm-cloud.md` for why: GLKVM-Cloud here is being used purely for router administration, no physical KVM hardware involved), but worth documenting on its own merits since it's the same self-hosted-cloud philosophy applied to a genuinely different problem: out-of-band access to a physical machine's console — BIOS, boot failures, a hung OS — independent of whether that machine's own network stack is even up. Same benefit already described for GLKVM-Cloud's router-admin use here: no dependency on GL.iNet's vendor cloud, works without a public IP, reachable over Tailscale just like everything else in this design.

Model	Price (USD)	Video	Ports	Cellular	Rack/multi-server	Notes
Comet (GL-RM1)	\$70–90	4K@30 claimed — one independent review reports the HDMI input actually caps around 2K@60; not independently confirmed either way here	1× GbE, HDMI in, USB-C/USB2 for KB/mouse emulation	No	Single-server only	The base model — noted (and ruled out of scope) in <code>glkvm-cloud.md</code> .
Comet Pro (GL-RM10)	\$180	4K@30, hardware H.264 encode, ~30-60ms latency	1× GbE, USB for KB/mouse, HDMI passthrough	No (WiFi 6 only)	Single-server	Adds a small onboard touchscreen and a USB "fingerbot" accessory for physical power-button control.
Comet X (GL-RM4PE)	\$270	4K@30	Quad HDMI/USB — up to 4 servers per unit, single PoE Ethernet port powers the whole unit	No	Yes — 10"/19" rack-mount	The one genuinely built for a server-room/rack deployment rather than one-off remote access — the closest fit if this line is ever used to manage a rack of machines instead of a single box.
Comet 5G (GL-RM10RC)	\$300	Up to 4K, hardware H.264 encode	1× GbE, HDMI in/out (loop), 2× USB-C, 1× USB 2.0	Built-in 5G RedCap + 4G LTE fallback	Single-server	The cellular-backed model — genuine out-of-band access even if the target machine's own network <i>and</i> the venue's primary internet are both down, since the KVM's own uplink doesn't depend on either. Directly mirrors the multi-WAN failover reasoning above, just applied to a KVM unit instead of a theatre router.
Comet Q (GL-RMQ1)	\$80–130	N/A — DisplayPort Alt-Mode passthrough	USB-C only	No	N/A	Different category, not a straight comparison: a mobile-device (phone/tablet/laptop) remote-control dongle, not a server HDMI KVM. Was still in crowdfunding as of research — confirm general-availability/shipping status before treating pricing as current retail.

If this line is ever adopted, Comet 5G is the standout for exactly the same reason Spitz AX/Puli AX stand out among the routers: a genuinely independent uplink, not sharing fate with whatever network the thing it's managing (or the venue) is on. Comet X is the pick if the target ever becomes "a rack of servers" rather than one box. Same purpose-built-over-general-purpose-workaround logic this repo already applies elsewhere — see `docs/birddog-play-rationale.md` for the same argument made about BirdDog Play vs. a laptop running VLC.

Summary

For this build specifically, GL-iNet Slate AX routers plus Tailscale earn their place on three independent grounds that all point the same direction: **segmentation** contains DHCP, multicast, and security faults to a single theatre instead of the whole event; **Tailscale** routes between those segmented subnets without depending on the venue's own physical network cooperating, and enforces cross-theatre isolation at the overlay regardless of how the underlying VLANs actually turn out; and **GLKVM-Cloud** gives centralized administration of all 14 units as one console, self-hosted, for free.

The features this build doesn't currently use — multi-WAN failover, WiFi-as-primary-link, built-in cellular, mesh/repeater backhaul — aren't hypothetical extras. They're exactly the reasons this is reusable hardware rather than single-event kit: the same units, the same Tailscale-mesh architecture, and the same self-hosted-cloud philosophy (GLKVM-Cloud for routers, the Comet line for out-of-band KVM) apply directly to remotely administering LED processors, running a WiFi AP

for mixing-desk control apps, or cutting the number of wired drops a venue needs to provide — all without re-deriving the architecture from scratch each time.

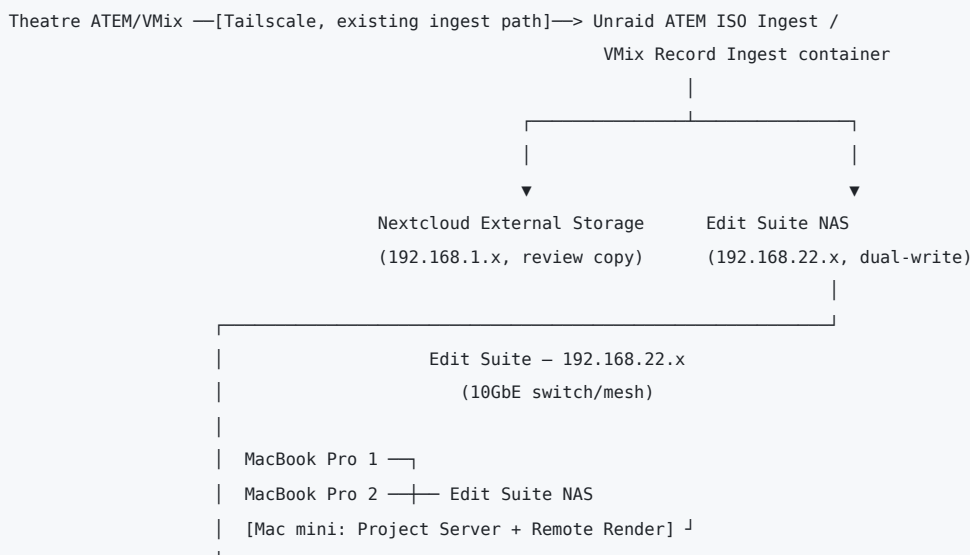
Live editing — post-production at the mothership

A separate subsystem at the mothership: 2× MacBook Pro editing workstations plus dedicated fast storage, editing event content as it arrives rather than waiting until after the event. Deliberately **not** part of the theatre-facing network this whole repo otherwise documents — different traffic pattern (sustained multi-gigabit scrubbing/export, not the bursty low-bitrate streams everything else here is sized for), different security boundary (no reason for 12 theatres to reach an editor's workstation or vice versa), and a genuinely different failure domain (an editing workstation crashing shouldn't touch the live show).

Why this needs its own network segment, not just a corner of the existing one

Every bandwidth figure elsewhere in this repo ([docs/bandwidth-analysis.md](#)) is sized around what the *live event* needs — SRT/NDI streaming, ATEM ISO ingest, file sync. Editing is a different animal: scrubbing a multicam timeline or exporting wants sustained throughput in the hundreds of Mbps *per editor* (multiple Gbps only for 4K or intermediate codecs), not the tens of Mbps this design budgets per theatre. For scale: the ATEM Mini Extreme ISO is an HD switcher whose ISO files are H.264 at up to 70 Mb/s each (Blackmagic tech specs), so a full 9-angle multicam is ~630 Mbps ≈ 79 MB/s per editor at the spec ceiling. (Each theatre actually patches only a camera and up to two laptops — the common input map — so a real session's multicam is nearer camera + 2 laptops + program, ~105 Mbps ≈ 13 MB/s typical, on planning estimates.) 4K would only enter through the VMix recordings (BirdDog P400s are 4K-capable; VMix's record settings are still unconfirmed — see [docs/vmix-record-ingest.md](#)). Putting that load on the mothership's existing 2× bonded 1GbE ([docs/server-specification.md](#)) would mean either starving the editors or risking contention with the live ingest pipelines still running during the event — the one thing this whole design goes out of its way to avoid elsewhere (see the NDI-fallback mitigation's own margin concerns in [docs/bandwidth-analysis.md](#)).

Recommendation: a dedicated 10GbE LAN for the edit suite, physically separate from the mothership's theatre-facing network, proposed as [192.168.22.0/24](#) (next free block after the VMix nodes — see [docs/ip-address-map.md](#)):



Same ingest containers already documented in [docs/atem-iso-ingest.md](#) and [docs/vmix-record-ingest.md](#) , writing to a **second destination** as well as Nextcloud — one pull off the ATEM/VMix source, two writes, not a chained re-sync (see Decision 1 below). No Tailscale within the edit suite itself — everything in this subsystem is in the same room as the mothership, unlike the theatre uplinks this repo's Tailscale mesh exists to solve for ([docs/tailscale.md](#)). A simple routed connection back to [192.168.1.x](#) is enough for anything that still needs it; the edit suite doesn't need to be reachable *from* any theatre, and vice versa. **The dual-write itself must not ride that routed path:** the Unraid server needs its own

interface on `192.168.22.0/24` (a 10GbE NIC — see `docs/server-specification.md` § NIC), with the NAS share mounted on the host and bind-mounted into the ingest containers, so the second write never touches the theatre-facing bond. That interface has **no address assigned yet** in `docs/ip-address-map.md`.

MacBook Pros have no built-in Ethernet at all — each needs a Thunderbolt-to-10GbE adapter or dock (e.g. Sonnet Solo10G, MSRP 199.99, 200-230 street as of 2026-10-06; OWC-class equivalents) to actually hit these speeds; budget for that alongside whatever storage is chosen below. The Mac mini (Decision 2) ships with 2.5GbE as standard, so order it with Apple's 10GbE build-to-order option rather than adding an adapter.

Throughput sanity check. A single 10GbE link delivers roughly 900+ MB/s over SMB in practice (ProVideo Coalition's Cloud Store Mini review measured >900 MB/s read/write from a Mac). Against that: 2 editors × ~79 MB/s (9-angle HD multicam), plus the ingest dual-write (at the default live scope of camera + program per theatre, ~98 MB/s typical / ~135 MB/s high including the ~120 Mbps VMix placeholder; ~172 / ~255 MB/s if both laptop inputs are pulled live too — see `docs/server-specification.md` — and physically capped at roughly ~225–375 MB/s by the twelve Slate AX uplinks' *estimated* ~150–250 Mbps), plus Remote Render reads, is well under half of one 10GbE port. 10GbE is comfortable headroom here rather than a hard requirement; it becomes necessary only if the VMix recordings turn out to be 4K or an intermediate codec (ProRes and similar).

See `diagrams/live-editing-dataflow.svg` for this same dual-write path as a diagram.

Decision 1: Nextcloud's storage vs. a dedicated NAS

Dedicated NAS, kept in sync via the same rclone/rsync pattern already built for ingest. Three reasons Nextcloud's own storage (the Unraid recording pool) is the wrong target to edit directly against:

- **It's sized and provisioned for the ingest workload, not editing.** The recording pool's whole design brief (`docs/server-specification.md`) is absorbing 16 concurrent low-bitrate append streams reliably — a completely different I/O shape from 2 editors scrubbing/exporting simultaneously.
- **Contention risk during the live event.** ATEM ISO Ingest and VMix Record Ingest are still actively writing to this same pool while editors would be reading from it — exactly the kind of shared-resource risk this repo's segmentation philosophy exists to avoid everywhere else.
- **Nextcloud's own serving stack (PHP/WebDAV) adds overhead a raw SMB/NFS-native NAS mount doesn't have** — fine for occasional file sync, not ideal for sustained media scrubbing.

A dedicated NAS, fed by the **same pull, writing to a second destination** — not a chained re-sync reading Nextcloud's own copy back out again, just the existing `config/atem-iso-ingest/` and `config/vmix-record-ingest/` containers' incremental append step (`pull-iso.py`'s FTP REST resume for the ATEMs by default, `rsync --append` for VMix and the ATEM alternative) writing to both `/mnt/user/nextcloud-external/...` (Nextcloud's External Storage, for the near-real-time review path this repo already documents) and a second mount on the edit-suite NAS, in the same pass. One read off the ATEM/VMix source, two writes — cheaper and simpler than pulling twice, and the edit suite never has to wait on Nextcloud's own indexing (`occ files:scan`) to see new footage, since it's not going through Nextcloud at all. Same incremental-transfer reasoning as everywhere else in this repo: the append step only transfers new bytes as footage keeps growing, on both destinations. (The second write is a documented design, not yet a code change — the actual scripts currently write only the Nextcloud destination; tracked as `docs/open-questions.md` item 15.)

NAS options, roughly by cost (US prices as of 2026-10-06; Blackmagic raised Cloud Store prices more than once in 2026 on flash/DRAM costs, so re-check before ordering):

Option	~Price	Notes
Blackmagic Cloud Pod (BYO USB drives)	\$445	1×10GbE, but it shares USB-C drives and each disk port is USB 3.0 (5 Gb/s) — the drives, not the network, set the ceiling (a few hundred MB/s per disk)
Blackmagic Cloud Dock 2 / Dock 4 (BYO drives)	\$649 / \$1,575	2× / 4× 10GbE, takes 2.5" U.2 or SATA drives (≤11 mm), exFAT/HFS+ only . Each drive is shared as its own independent volume — no RAID, so no redundancy either . Cheapest path to real throughput if drives are sourced separately
Generic 10GbE NAS (Synology/QNAP-class,	Varies, comparable to Cloud Dock +	Not Resolve-specific, but plain SMB/NFS works identically; more flexible (can also run Docker containers — relevant to Decision 2), and the only option here with real RAID redundancy

Option	~Price	Notes
populate with own drives)	drives	
Blackmagic Cloud Store Mini, 8TB	\$5,059	4× M.2 NVMe, RAID 0 — no redundancy , 1×10GbE + 1×1GbE + USB-C, up to 50 concurrent SMB/NFS connections. Confirm the RAID0 risk is acceptable — a drive failure mid-event means falling back to a fresh pull from the mothership's own recording pool, not losing anything permanently, but it is a real mid-event disruption
Blackmagic Cloud Store Mini, 16TB	\$8,435	Same, more headroom if editing against more than a curated subset of the event's footage

For 2 editors working from a curated subset of footage (not the full 8TB recording pool — see the open question below on how much actually needs to be pulled), the Cloud Store Mini 8TB is a reasonable fit and gets Resolve-aware sync to Blackmagic Cloud "for free" if that's ever wanted later (see Decision 4). A generic 10GbE NAS is the cost-conscious or redundancy-conscious alternative. Either way, the Project Server is a separate dedicated Mac mini (Decision 2), not something hosted on the NAS itself — see below.

Decision 2: a dedicated Mac mini running the Project Server and Remote Render

The workflow this section is built around (see below) has **resolve-configurator build one shared show project** — a single set of Theatre/Date bins and per-session timelines both editors work against, populated by smart bins as footage lands — not two editors each on their own private copy. That design choice settles the "do we even need Collaboration" question from an earlier draft of this doc: **yes**, because both editors need to see the same live-updating bin/timeline structure, not separate forks of it. So this is a dedicated node, not an optional one.

What that node needs to run:

- **The DaVinci Resolve Project Server** — a small standalone app bundling a pinned PostgreSQL version, syncing project metadata (bins, timelines, locks) between editors in real time. **It does not move media** — actual footage still needs to be reachable by both editors via shared storage regardless (the NAS from Decision 1).
- **Resolve's Remote Render feature** — dispatches an export job from an editor's Deliver page to a separate networked Resolve Studio install, so the editor's own laptop isn't tied up rendering while cutting continues. Confirmed (Blackmagic's own Reference Manual): this is one job to one machine at a time — **not** a distributed render farm; Resolve never splits a single job's frames across multiple nodes.

Recommendation: a single dedicated Mac mini running both, rather than a Docker container on a NAS (dropped from an earlier draft — Blackmagic ships the Project Server as a native macOS/Windows/Linux app, not a container image, so this isn't a realistic option) and rather than either editor's own laptop. The most-cited failure mode across DIT/post-production forum threads is exactly the laptop case: if that machine sleeps, reboots, or the lid closes, *both* editors lose the shared project, not just one.

Combining Project Server and Remote Render duty on one Mac mini — confirmed workable, but thinly documented, so flagging the confidence honestly:

- Blackmagic's own documentation does not state a prohibition on co-locating the two roles, and architecturally there's no reason it wouldn't work — the Project Server is just a background PostgreSQL-backed service, Remote Render is just a licensed Resolve Studio instance running in a render role. But this specific combination isn't something Blackmagic explicitly confirms in writing, and it's a thin pattern in practitioner discussion — one forum thread asked this exact question ("could the project library be on this Mac mini as well?") and never got a direct answer; one real-world example found keeps a small Mac mini as project-server-only, separate from any render node. Worth treating as **very likely fine for a 2-editor event-scale deployment, not a Blackmagic-certified pattern** — a reasonable design decision, not a guaranteed one.
- **Both the render node and every editor's machine need DaVinci Resolve Studio**, not the free version — confirmed directly in Blackmagic's manual ("remote rendering does not work with the free version of DaVinci Resolve"). **\$295 perpetual license per seat**, one-time cost. Blackmagic's own forum states an activation-code license can be active on 2 machines simultaneously, which in principle could cover an editor's laptop plus the render node from one seat — but this is a general Studio licensing term applied to this scenario, not a Remote-Render-specific clause, so budget one license per machine unless that's tested and confirmed to work as expected.

- **Remote Render requires a Network project library (PostgreSQL), not a Local one** — confirmed by a Blackmagic DaVinci support engineer on their own forum. In other words, Remote Render can't be added on top of two independent local projects; it only works once the Project Server (or Blackmagic Cloud) infrastructure already exists. That's further reason these two decisions are coupled here, not separable.
- **The media storage volume must be mounted on both the requesting machine and the render node** — confirmed in Blackmagic's manual. The Mac mini needs the same Edit Suite NAS mount the MacBook Pros use, not just database connectivity.
- **Performance is workable for 1080p H.264 but not fast** — no source benchmarks this exact scenario, so treat this as reasoned inference from adjacent data: Apple Silicon's H.264 hardware encode gap (base/Pro chips: one encode engine vs. two on Max, four on Ultra) matters far more at 4K/UHD than at 1080p, where independent reviews found little real difference between a Mac mini and a Mac Studio. But remote rendering itself runs slower than local rendering regardless of hardware — one documented case measured roughly half the fps remotely vs. locally. **Spec the Mac mini at Pro-chip tier**, not the base chip, given it's doing double duty as database host and render node; a base-tier mini is a real risk for anything beyond light 1080p H.264 jobs. Current lineup (announced 2026-08-25, Apple Newsroom): **M6 from \$899, M5 Pro from \$1,699** (US, as of 2026-10-06), 2.5GbE standard with a 10GbE build-to-order option — so the realistic order is an **M5 Pro with 10GbE**. M2 Pro/M4 Pro minis are no longer current models but remain adequate if one is already on hand. (The M5 Pro still has a single video encode engine, so the base/Pro vs. Max encode-engine point above is unchanged.)
- **Known gotcha to plan around:** submitting multiple render jobs to the same remote node at the same time can silently fail — send them one at a time. Fine at 2-editor scale, worth documenting as an operational note rather than something to engineer around.

Decision 3: using Blackmagic Cloud Store as the project server

No — not currently supported, and not just "not recommended." Blackmagic Cloud Store appliances are documented as SMB/NFS media storage plus a Resolve-aware sync target for the separate Blackmagic Cloud service (Decision 4) — nothing in Blackmagic's own documentation or independent DIT/post-production sources indicates they expose a general-purpose compute or container environment capable of hosting the Project Server's PostgreSQL database. Practitioner guidance consistently treats "where the media lives" (Cloud Store or any NAS) and "where the project database lives" (the dedicated Mac mini from Decision 2) as two separate questions with two separate answers — the project server needs its own host regardless of which storage option was picked.

Decision 4: the cloud store's built-in internet sync functionality

Not needed for this design, and probably not worth turning on. "Blackmagic Cloud" is a genuinely separate product from the physical Cloud Store hardware — an internet-based sync service (own subscription: **~\$5/monthperprojectlibrary * * + ** 15/TB/month** for synced media — the \$15/TB figure is confirmed; the \$5 library fee is stated by Blackmagic staff/users on Blackmagic's own forum and by third-party guides, but could not be confirmed on a Blackmagic pricing page as of 2026-10-06, so re-check before quoting it) built for the case where collaborators are in *different physical locations* and don't want to deal with VPNs/firewalls to reach each other. Everything in this design is in one room at one venue — there's no second site to sync with, so this service would add an ongoing subscription cost and an internet-facing dependency for a problem this topology doesn't have. **Worth reconsidering only if there's a genuine need for someone off-site (a remote producer, a head-office stakeholder) to review cuts** — that's a real use case Blackmagic Cloud is built for, just a different one from what's being solved here. Track as a possible future need, not a current requirement.

Workflow: from ATEM pull to finished edit

Everything above (dedicated NAS, project-server decision) exists to support one actual pipeline — footage should already be sitting where an editor expects it, organized by session, by the time anyone sits down to cut. The pieces:

Theatre ATEM/VMix —[existing ingest containers]—> dual write:

└> Nextcloud (review copy)
└> Edit Suite NAS

|

▼

resolve-configurator (run ahead of the event, from the same session CSV nc-filedropbatch already uses for presenter uploads) has already built, into a shell Resolve project:

- one bin per theatre, per day
- one empty timeline per session inside each day's bin
- backgrounds/title-slide assets, pre-imported
- a smart-bin RECIPE per session (Resolve's scripting API can't create real smart bins – applied once, by hand, before the event; see resolve-configurator's own README for why)

|

(one-time, pre-event: apply each recipe in Resolve's UI to turn it into a real smart bin)

▼

As footage lands on the NAS, each session's smart bin auto-fills with just its own clips (filtered by record-drive path + date + the session's timecode window) – no manual sorting.

|

▼

Editor opens that session's pre-built timeline, drags the now-populated smart bin's clips in, top-and-tails, exports (optionally via Remote Render – see Decision 2).

What resolve-configurator actually builds (from its own README, not assumed): project format settings (frame rate/resolution, or any Resolve project setting via passthrough), a Theatre / Date bin structure, an empty timeline per session named from a template (e.g. Start Time - Presenter), pre-imported background/title-slide assets (global or per-theatre), and — since Resolve's scripting API genuinely cannot create smart bins, confirmed for the versions currently in use — a written recipe (smart-bins.md / smart-bins.json) giving the exact filter rule for each session (file path contains that theatre's record-drive name, date created is the session date, start timecode falls in the session's derived window). The tool builds everything the API *can* do automatically; the smart bins themselves are a one-time manual step per session, applied once ahead of the event from that recipe — after which they behave exactly like any other Resolve smart bin, auto-filtering as matching footage arrives.

Once that one-time setup is done, the editor's own job is genuinely just: open the session's timeline (already sitting in the right day/theatre bin, already named), the smart bin next to it already shows only that session's clips (no scrubbing through a whole day's unsorted recordings), drag them onto the timeline, top-and-tail to the actual session length, export. All of the organizing work happens ahead of time or automatically — editors are cutting, not filing.

See diagrams/live-editing-project-structure.svg for this whole pipeline as a diagram — from the session CSV through resolve-configurator's build, the smart-bin recipe's one-time hand-apply step, and the auto-fill down to the editor's own four-step job.

Ingest workflow: physical media (master drives, camera cards)

A separate, complementary path to the network-based ATEM ISO pull already built (docs/atem-iso-ingest.md) — that pipeline is optimized for near-real-time *review* access, but the ATEM's own USB SSD (and any standalone camera's SD/CFexpress cards, if this event uses cameras beyond the NDI-fed BirdDog P400s) remains the authoritative master, same "master vs. mirror" framing as the network ingest doc. Physically walking a drive/card to the edit suite for a proper checksummed offload — not just relying on the network pull — is standard DIT (Digital Imaging Technician) practice for anything that matters enough to edit. **It is also now the only route for most of the ISO footage:** the network ingest pulls

just the program file and the ISO inputs in `ATEM_ISO_INPUTS` (default: input 1, the camera — see `docs/atem-iso-ingest.md`). The laptop ISOs (inputs 2–3, unless enabled live), the backup inputs (4–8), and the audio `.wav`s and `.drp` project are recorded on the ATEM's SSD and reach the edit suite only through this offload — at a session break or end of day. Plan the offload cadence around when editors need the laptop angles, not just around end-of-day archiving.

Tool comparison

Tool	Vendor	Price	Role
ShotPut Pro	Imagine Products	169 <i>perpetual</i> (+59-70/yr updates after year 1), \$60/30-day rental	Industry-standard dedicated checksummed offload + verify + PDF/CSV/MHL report — the classic "DIT cart" tool
OffShoot / OffShoot Pro	Hedge	\$169 / \$249 one-time per activation (incl. 1 year of updates; \$79 to extend), \$49/30-day rental (prices as of 2026-10-06)	Same core job (fast checksummed offload, multiple simultaneous destinations), simpler UI, no metadata/look-management layer
Silverstack Offload Manager	Pomfort	\$169/yr; project licenses €45 (14 days) / €59 (1 month) — USD project price not confirmed (as of 2026-10-06)	Pomfort's own cut-down offload-only tier, positioned against ShotPut/OffShoot for smaller productions
Silverstack XT / Lab	Pomfort	XT ~\$899/yr; Lab \$1,099/yr, Lab project licenses \$159 (14 days) / \$249 (1 month) / \$399 (2 months) (as of 2026-10-06; plain Silverstack was folded into XT at v9)	Full DIT station: offload+verify+metadata/lens data+look management (CDL/LUT), and (Lab only) audio sync + transcode/dailies — a much bigger tool than this event likely needs unless additional standalone cameras with real production metadata needs are added
FoolCat	Hedge	\$29/mo, \$89-129/activation, \$299 bundle	Companion report generator , not a copy tool — HTML/PDF camera reports with thumbnails, plugs directly into OffShoot's offload queue
o/PARASHOOT	OTTOMATIC GmbH	Free	Companion card-safety tool — confirms a card's files exist at the backup destination (filename+size, not checksum) then reversibly blanks the card so the camera prompts a clean reformat; pairs with OffShoot

Recommendation for this event's actual scale: ShotPut Pro or OffShoot Pro as the primary checksummed-copy tool — both are cheap, reliable, and exactly matched to "offload a drive/card, verify it, get a report" without paying for Silverstack's much larger metadata/look-management scope this design doesn't currently need (no standalone camera color pipeline in scope, no lens metadata being tracked). Add **FoolCat** only if stakeholders want visual per-session camera reports (director/producer-facing dailies-style PDFs) rather than just a copy-verification log. Add **o/PARASHOOT** (it's free) if any actual camera SD/CFexpress cards are in play and get reformatted/reused between sessions — cheap insurance against a card getting wiped before its footage is confirmed safe. **Silverstack XT/Lab is the right call instead** only if this event's scope grows to include standalone cameras needing real lens/metadata tracking or an in-house look/color pipeline — worth revisiting if that changes, not a default recommendation for the design as currently scoped.

A typical chained workflow, once decided: **ShotPut Pro/OffShoot (checksummed copy to the edit-suite NAS) → FoolCat (camera report, optional) → o/PARASHOOT (safe card reformat, optional)** — not competing alternatives, a real multi-stage pipeline other productions run this way.

Devices — new addition to the inventory

Device	Proposed IP	Notes
Edit suite NAS (Cloud Store or generic 10GbE NAS)	192.168.22.2	See Decision 1
MacBook Pro — Editor 1	192.168.22.11	10GbE via Thunderbolt adapter/dock
MacBook Pro — Editor 2	192.168.22.12	10GbE via Thunderbolt adapter/dock
Mac mini — Project Server + Remote Render	192.168.22.20	Dedicated always-on machine, not an editor's own laptop — see Decision 2; order with the 10GbE option

Not yet listed: the **Unraid server's own edit-LAN interface** on `192.168.22.0/24`, which the dual-write needs (see the start of this doc). It needs an address in `docs/ip-address-map.md` — not assigned here, since that file is the authoritative inventory.

See `docs/ip-address-map.md` for how this fits the rest of the network's addressing.

Open questions this section introduces

- **How much of the event's footage actually needs editing access** — the full 8TB recording pool, or a curated/selected subset? Drives NAS capacity sizing in Decision 1. Sizing now rests on the 2026-10-06 planning figures (camera ISO 35 / 45 / 70 Mbps, laptop ISO 15 / 25 / 40 Mbps *estimated*, program ~10 Mbps — see `docs/atem-iso-ingest.md` § Bandwidth): the live dual-write is ~20 / 25 / 36 GB per theatre-hour at the default camera + program scope, and the physical offload adds the rest of each ATEM's recording on top (at least the two laptop ISOs, ~14–36 GB per theatre-hour between them, plus whatever the backup inputs carry). Neither is measured yet (`docs/open-questions.md` #0).
- **Whether the smart-bin recipe's filters match what actually lands on the NAS.** The recipe filters on "file path contains that theatre's record-drive name" and "date created is the session date". The ingest destination layout is now `TheatreN/ISO/<recording folder>/...` (`pull-iso.py` keeps the ATEM's recording-folder structure, but not the drive name), and a file's creation date on the NAS is when ingest first wrote it, not when the ATEM recorded it (normally the same day, but not for a late final pull after midnight). Either the NAS path layout should preserve a per-theatre/drive identifier the recipe can match, or the recipe should match the `TheatreN` path instead — check against resolve-configurator's actual output before the event.
- **Whether combining Project Server and Remote Render duty on one Mac mini holds up in practice** — Decision 2 flags this as architecturally sound but thinly documented by Blackmagic and practitioners alike; worth a real test before the event, not just before this doc.
- **Whether this event uses any standalone cameras with SD/CFexpress cards** beyond the NDI-fed BirdDog P400s — affects whether the physical-media DIT workflow has real cards to ingest beyond the ATEM's own USB SSD, and whether Silverstack's deeper metadata/lens-data features become worth their cost.
- **Whether the Cloud Store Mini's RAID 0 (no redundancy) is an acceptable risk** given the fallback is a fresh pull from the mothership's recording pool, not permanent data loss — but still a real mid-event disruption if it happens.
- **Implementing the dual-write in the actual ingest scripts** — `pull-iso.py` and `mount-and-sync.sh` currently write only the Nextcloud destination; the second write this whole section depends on is a real code change, tracked as `docs/open-questions.md` item 15.

File sync flow

Theatre laptops → Nextcloud (rclone)

PowerPoint Main/Backup and VT Main/Backup laptops in each theatre run `rclone`, syncing their theatre's folder (`/$TheatreName/`) up to the Nextcloud file server at the mothership.

```
Theatre laptop (PowerPoint/VT, Main/Backup) --rclone sync--> Nextcloud:/$TheatreName/
```

Ready room → Nextcloud (WebDAV)

Ready room PCs at the mothership write directly to Nextcloud via a WebDAV mount — a live local mount on the mothership LAN, not rclone, since there's no WAN hop involved.

```
Ready room PC --WebDAV mount--> Nextcloud (local to mothership LAN)
```

See `diagrams/file-sync-flow.svg`.

The other file flows into the same Nextcloud

This doc covers only the *document* sync — theatre laptops and ready-room PCs. Two recording pipelines write into the same Nextcloud by a completely different mechanism (near-real-time incremental pulls, not rclone sync of finished files), and both of them also write to a second destination (documented design; not yet implemented in the scripts — see `docs/open-questions.md` item 15):

- `docs/atem-iso-ingest.md` — each theatre's ATEM ISO recordings, pulled over FTP
- `docs/vmix-record-ingest.md` — each VMix PC's recordings, pulled over SMB
- `docs/live-editing.md` — the same pulls dual-write to the edit suite's NAS alongside Nextcloud

ATEM ISO ingest — near-real-time, Nextcloud-aware

Goal: get each theatre's ATEM Mini Extreme ISO recordings onto the mothership's Nextcloud as close to real time as the hardware allows, without corrupting anything and without the theatre's uplink falling permanently behind. The ATEM records up to 9 H.264 files (8 input ISOs + program, **up to 70 Mb/s per ISO file** per Blackmagic's spec); only the **program file plus the ISO inputs chosen in `ATEM_ISO_INPUTS`** (default: input 1, the camera) are pulled live — see Common theatre input map and Bandwidth. Everything else stays on the ATEM's SSD for the physical offload.

The two hardware facts that shape this

- **The ATEM has a built-in FTP server**, and files can be browsed/transferred over it *while recording is still in progress* — confirmed via multiple independent accounts (aaronparecki.com, Directory Opus forum). This is also the only mechanism available — the unit has no NDI output and its Ethernet port doesn't expose the drive over SMB/NFS, only FTP. **This holds for the original ATEM Mini Extreme ISO only.** The newer **ATEM Mini Extreme ISO G2** has a 10G Ethernet port and shares its CFexpress/USB recording media as a network disk (Blackmagic tech specs) — if the theatres have G2 units, a mount-based pull (like the VMix ingest) becomes possible. Confirm which model is actually deployed.
- **The files are very likely safe to read mid-write.** Blackmagic markets editing an ISO recording in DaVinci Resolve *before the event even finishes* — that only works if the container format is structured so a partial file is valid up to whatever's been flushed so far (in practice, this means fragmented MP4 — periodic `moov` / `mdat` boxes rather than one index at the very end). Treat this as **very likely true given the marketed capability, but worth a quick empirical check** (copy a file mid-recording, confirm `ffprobe` / a media player can open it) before relying on it operationally.

Common theatre input map

Decided 2026-10-06: **every one of the 12 ATEMs is patched identically**, so one ingest setting covers the whole fleet and an ISO file's input number means the same thing in every theatre.

ATEM input	Source	Pulled live?
1	Camera	Yes (default)
2	Presenter laptop 1 (PowerPoint Main)	Optional — only if router headroom allows (<code>ATEM_ISO_INPUTS=1,2</code>)
3	Laptop 2 (VT Main / second presenter)	Optional (<code>ATEM_ISO_INPUTS=1,2,3</code>)
4–8	Backups / spare	Never live — stays on the ATEM SSD
Program	The switched output	Always pulled

Why choose inputs at all. The ATEM's ISO recording is all-or-nothing per switcher — it can't disable individual inputs (an empty input just produces a near-empty file), and the ISO bitrate isn't user-settable; only the frame rate changes the target (~45 Mbps at 24/25/30p, ~70 at 50/60p, VBR — see Bandwidth). So the choice has to be made on the *pull* side: what crosses the theatre's uplink live, not what gets recorded. Everything is still recorded; everything not pulled live comes in via the physical DIT offload of the ATEM's SSD (or between sessions) — see `docs/live-editing.md` § physical media.

The program recording is different: it follows the ATEM's configurable Streaming/record quality setting rather than the fixed ISO rate. Set it so the program file is **~8–10 Mbps** (one third-party test measured ~4.8 Mbps at a lower setting) — the deployment runbook carries this as a per-unit step.

`ATEM_ISO_INPUTS` (comma-separated input numbers, default `1`; empty = program only) is read by both ingest methods and set fleet-wide in `docker-compose.yml` / `.env.template`. `pull-iso.py` **walks each ATEM's recording folders recursively** (up to `<root>/<recording>/Video ISO Files/<file>`), pulls every `.mp4` / `.mov` that isn't an ISO file (i.e. the program recording), and pulls an ISO file only if the input number in its name is in `ATEM_ISO_INPUTS`. It recognises ISO

files by `CAM <n>` in the file name — the naming as far as is known, still to confirm on a real unit (`docs/open-questions.md` #21); override the regex with `ATEM_ISO_INPUT_PATTERN` if it differs. The mirror keeps the ATEM's folder structure under `TheatreN/ISO/`. The `rclone-mount-rsync/` alternative applies the same selection with rsync include/exclude filters built from the same variable (it matches `CAM <n>` by glob, not by `ATEM_ISO_INPUT_PATTERN`). Audio `.wav` source files and the `.drp` project are not pulled live by either method; they arrive with the physical offload.

Why not plain rsync, and why not plain rclone sync

Plain rsync can't connect at all — it needs SSH or an rsync daemon on the far end, and the ATEM only speaks FTP. There's no bridging that directly; the ATEM never exposes rsync or SSH.

A naive periodic `rclone sync` /whole-file re-copy doesn't survive the bandwidth math. Real numbers: at the default live scope (camera ISO + program, ~45–80 Mbps, ~55 typical), a 3-hour session is already ~61–108 GB (~74 GB typical). Re-uploading the *entire current file* every 10 minutes stops fitting in that 10-minute window at the Slate AX's estimated ~150–250 Mbps Tailscale ceiling once the session is ~27–45 minutes old at the typical rate (~19 minutes at the 80 Mbps high case on a 150 Mbps router) — and that ignores everything else sharing the uplink — after which the sync falls permanently behind. Genuinely incremental transfer isn't a nice-to-have here, it's required.

Two ways to get genuinely incremental transfer — both implemented, pick one (see `config/atem-iso-ingest/`):

	Mechanism	Why it's incremental
<code>pull-iso.py</code> (default)	FTP's own <code>REST <offset></code> command — the same "resume from byte offset" mechanism <code>curl --continue-at /</code> <code>lftp --continue</code> / <code>wget -c</code> use	Deterministic: explicitly requests only bytes past the last known offset. No cache layer involved.
<code>rclone-mount-rsync/</code> (alternative)	<code>rclone mount</code> presents the ATEM's FTP share as a local FUSE path; <code>rsync --append</code> (not rsync's general checksum-diff algorithm) transfers only the tail past the destination's current size	Also deterministic — <code>--append</code> seeks straight to the destination's size rather than reading/hashing the whole file, so it doesn't depend on rclone's VFS cache behaving well for a growing remote file — a real concern for mount-based approaches that <code>--append</code> sidesteps entirely

Both are legitimate. `pull-iso.py` has fewer moving parts (no persistent FUSE mounts to supervise); `rclone-mount-rsync/` uses tools you may already operate day to day, at the cost of 12 FUSE mounts to keep healthy. Neither uses rsync's *general* block-checksum algorithm (which genuinely would require reading the whole file every pass) — that algorithm only saves network bytes when both ends speak the rsync wire protocol to each other, which isn't possible against an FTP-only source regardless of what sits in between.

Architecture

Centralized entirely on the Unraid server as its own container (`192.168.1.17`) — no software installed on any theatre laptop, consistent with how BirdDog Central/NDI Discovery Server/DERP are already consolidated there (the mothership VMix VM was removed — see `docs/open-questions.md` #10). Reaches each theatre's ATEM over Tailscale subnet routing (see `docs/tailscale.md` — this is exactly the "route between subnets where needed" case the subnet-router setup was built for). Described below for the default (`pull-iso.py`) — the `rclone-mount-rsync/` alternative differs only in the pull mechanism, writing into the same Nextcloud folders the same way.

```

ATEM (192.168.X.2, FTP) --[Tailscale subnet route]--> Unraid: atem-iso-ingest container
      |
      | incremental REST-resume pull
      |
      v
local mirror file, growing in lockstep
(written directly into the folder Nextcloud
has mounted as External Storage – Local)
      |
      | targeted `occ files:scan` (short cycle)
      |
      v
      |
      | visible in Nextcloud, growing

```

1. **Pull step** (`atem-iso-ingest` container, new — see `config/atem-iso-ingest/`): for each theatre, on a schedule (e.g. every 60–120s), walk the ATEM's FTP recording folders recursively, and for each wanted media file (program + the inputs in `ATEM_ISO_INPUTS` — see Common theatre input map), `REST -resume` from the last recorded byte offset and append the new bytes to a local mirror file at the same relative path. A small state file tracks per-file offsets so a restart doesn't re-pull from scratch.
2. **No separate upload/chunking step.** The pull step writes directly into `/mnt/user/nextcloud-external/TheatreN/ISO/` — the same path mounted into Nextcloud as an External Storage (Local) folder for that theatre. There's no WebDAV re-upload of the growing file at all, so the "whole-file re-sync is too slow" problem simply doesn't apply on this side — it's a same-host filesystem write, not a WAN transfer.
3. **Nextcloud awareness:** a short-interval, *targeted* `occ files:scan --path=...` (not `--all`) picks up the new file size on Nextcloud's side. Because this only touches one theatre's folder and runs locally on the same box, it's cheap even at a 1-2 minute cadence — unlike scanning the whole Nextcloud tree, which would not be.
4. **On session end**, a final pull + final targeted scan catches the last bytes and whatever moov/index finalization is *appended* when the ATEM's recording actually stops. **It does not catch finalization that rewrites earlier bytes in place** — MP4 muxers commonly patch a box size or header near the start of the file when recording stops (OBS's hybrid-MP4 "soft remux" is a documented example: obsproject.com). Both ingest methods are append-only and would leave such a mirror with stale header bytes. Whether the ATEM does this is unconfirmed; until it is, treat the mirror as "review copy" and either re-pull the finished file in full or run a checksum comparison against the source before treating it as final (see Open items).

Second write destination for live editing. The pull step also writes the same growing mirror file to a second mount — the edit suite's dedicated NAS (`192.168.22.x`), not just Nextcloud's External Storage. One read off the ATEM, two writes, not a separate re-sync reading Nextcloud's copy back out. See `docs/live-editing.md` for the full editing subsystem this feeds — not yet reflected in `config/atem-iso-ingest/pull-iso.py` itself, which currently only writes the one Nextcloud destination; adding the second write path is a real code change, tracked in `docs/open-questions.md`.

Bandwidth

Resolved by decision (2026-10-06): the old ~10 Mbps-per-ISO working figure was wrong, and the design now pulls only a chosen subset live. Blackmagic's tech specs say the ISO inputs are recorded as "H.264 .mp4 files at up to 70Mb/s quality" (blackmagicdesign.com). The ISO bitrate is not user-settable — only frame rate changes the target, ~45 Mbps at 24/25/30p and ~70 at 50/60p, VBR (Blackmagic forum; the feature request for a setting is still open). User reports put the *average* at ~30–45 Mbps per ISO (e.g. ~15 GB/h per source ≈ 33 Mbps, forum); older firmware showed 90–120 Mbps, with a silent drop around firmware 9.6.x (forum). Only the **program** file follows the configurable quality setting, so ~10 Mbps is realistic for it alone. Pulling all 9 files live (~370–570 Mbps per theatre at 45–70 Mbps per ISO) is therefore off the table on the chosen router; the fix is the **common input map** plus `ATEM_ISO_INPUTS`. Still to measure on a real unit: the actual camera and laptop ISO bitrates, and the program file at the chosen quality setting (`docs/open-questions.md` #0).

Planning figures (low / typical / high): camera ISO **35 / 45 / 70 Mbps** (user reports; 70 = vendor cap), laptop (slides) ISO **15 / 25 / 40 Mbps — an unmeasured estimate**, one measured file pins it, program **10 Mbps**. Ingest in isolation, per theatre, including the ~4% WireGuard overhead:

Live scope (ATEM_ISO_INPUTS)	Payload	Ingest incl. WireGuard	vs. Slate AX (~150–250 Mbps est.)
Camera + program (1 , default)	45 / 55 / 80 Mbps	47 / 57 / 83 Mbps	comfortable
Camera + 1 laptop + program (1,2)	60 / 80 / 120 Mbps	62 / 83 / 125 Mbps	fits
Camera + 2 laptops + program (1,2,3)	75 / 105 / 160 Mbps	78 / 109 / 166 Mbps	fits at typical; tight at high
4 ISOs (2 cameras + 2 laptops) + program	110 / 150 / 230 Mbps	114 / 156 / 239 Mbps	marginal

(Per-theatre ATEM output at the default scope is ~20 / 25 / 36 GB per hour; at camera + 2 laptops ~34 / 47 / 72 GB/h.)

This rides the theatre's uplink in the *opposite direction* from the incoming SRT feed (~5.8 Mbps baseline / ~10.4 peak, see docs/bandwidth-analysis.md) — if the link is genuinely full-duplex-capable at its rated throughput, those two shouldn't compete much. But the ingest does **not** have the upstream direction to itself: the theatre's ATEM Overseer monitoring stream and Flock SRT preview (~10.4 Mbps each) run theatre → mothership alongside it, plus bursty rclone (~5), a fixed ~26 Mbps. Total upstream per router is then ~73 / 83 / 109 Mbps at the default scope and ~104 / 135 / 192 Mbps at camera + 2 laptops. The authoritative per-router model is in docs/bandwidth-analysis.md ; the table above covers the ingest in isolation only. The Slate AX's ~150–250 Mbps is an **estimate** extrapolated from a Beryl AX Tailscale measurement, not a Slate AX benchmark — benchmark one unit bidirectionally before relying on it (docs/open-questions.md #19). (The previous router, the A-1300, was dropped on 2026-10-06 because its ~170 Mbps figure is GL.iNet's kernel WireGuard number; Tailscale on its 32-bit CPU is estimated at only ~30–70 Mbps combined.) ATEM_ISO_INPUTS is the fleet-wide default in both ingest methods, and ATEM_ISO_INPUTS_T<n> overrides it for one theatre (e.g. ATEM_ISO_INPUTS_T3=1,2 adds Theatre 3's main laptop only). Use the override rather than raising the default: at camera + 2 laptops in every theatre, the fleet's inbound total can exceed the mothership's 2× 1GbE bond at the high end (docs/server-specification.md § NIC).

Master vs. mirror

Frame this as: **the ATEM's own SSD remains the authoritative final master** (complete, guaranteed-valid once recording stops) — pulled in full at the end of each session/day regardless. **Nextcloud holds a near-real-time, continuously-updated mirror** for early review/rough-cut purposes during the event. That framing resolves the tension between "want it fast" and "want it guaranteed correct": you get both, from two different consumers of the same underlying data.

Nextcloud versioning caveat

Nextcloud's versioning app may retain snapshots on repeated changes to the same file path. Configure version retention/expiry for the ISO folders (or disable versioning there specifically) so a multi-hour growing recording doesn't accumulate many near-duplicate historical versions before Nextcloud's own expiry curve catches up. Not a blocker, just worth tuning once this is running for real.

Open items

- **Measure real file bitrates on a unit** — the live scope is decided (see Common theatre input map), but the planning figures aren't measured: record a camera ISO and a laptop/slides ISO at the event's frame rate and firmware, and the program file at the chosen quality setting, then `ffprobe` them (or file size ÷ duration). The laptop figure (15 / 25 / 40 Mbps) is a pure estimate; one file pins it. Tracked as docs/open-questions.md #0.
- **Confirm the ATEM model (original vs. G2)** — the G2 exposes its media as a network disk, which changes which pull mechanism is the natural fit (see the hardware facts at the top).
- **Test the recursive walk against a real ATEM.** (Previously: "`pull-iso.py` doesn't recurse" — fixed.) Blackmagic's ISO recordings are written as a *folder* per recording (program file, a Video ISO Files subfolder, an Audio Source Files subfolder of `.wav` s, and a `.drp` Resolve project); `pull-iso.py` now walks that tree (MLSD, falling back to

NLST + a `CWD` probe, since the ATEM's FTP command set is unconfirmed) up to three levels deep and keeps the folder structure in the mirror. Untested against real hardware — confirm the walk finds the files and the depth limit is enough (docs/open-questions.md #22). `.wav` / `.drp` are deliberately not pulled live by either method; they come with the physical offload.

- **Decide the end-of-session integrity step.** Both methods are append-only (see step 4 under Architecture). For `rclone-mount-rsync/`, `VERIFY=1` now does a full `--checksum` comparison and repairs any differing blocks in place; note that this reads every file in full through the FTP mount — i.e. a full re-download over the theatre's uplink — so run it after the session, not during. `pull-iso.py` has no equivalent yet.
- **Confirm partial-file playability empirically** — copy a file mid-recording, check it opens/scrubs correctly, before treating the "safe to read mid-write" assumption as settled rather than "very likely."
- **Tune the scan interval and pull interval** against real session lengths and available Unraid CPU/disk headroom, once the server's full spec is known (see docs/open-questions.md).
- **Confirm the ATEM's actual FTP file/folder naming**, in particular that ISO files carry `CAM <n>` in their names — input selection depends on it. If a real unit names them differently, set `ATEM_ISO_INPUT_PATTERN` for `pull-iso.py` and edit `build_filters` in `rclone-mount-rsync/mount-and-sync.sh` ; a file that doesn't match is treated as a program recording and pulled, so a mismatch fails toward pulling *everything*, not nothing (docs/open-questions.md #21).
- **If using the `rclone-mount-rsync/` alternative**, test `rsync --append` against a genuinely growing file on a real ATEM before trusting it operationally — confirm it only transfers the new tail each pass (e.g. watch network throughput or `--itemize-changes` output), and budget for supervising 12 persistent FUSE mounts (auto-remount on failure), which `pull-iso.py` doesn't need at all. Also confirm new bytes show up promptly: FTP has no change notification, so a growing file's size on the mount only refreshes when rclone's directory cache expires (default 5 minutes — the script now sets `--dir-cache-time 30s` ; rclone mount docs).

VMix record ingest — near-real-time, Nextcloud-aware

Goal: same as `docs/atem-iso-ingest.md` — get each VMix PC's growing recording onto the mothership's Nextcloud as close to real time as possible, without corrupting anything and without falling permanently behind. Covers all 4 VMix PCs: VMix Node 1 (`192.168.20.21` / `.22` , off Theatre 1) and VMix Node 2 (`192.168.21.21` / `.22` , off Theatre 4) — see `docs/ip-address-map.md` .

Why this is actually simpler than the ATEM case

The ATEM Mini Extreme ISO only exposes its recording drive over FTP — a file-transfer protocol, not a filesystem-access protocol, so a mount-based approach there needs `rclone mount` as a FUSE bridge (that's the ATEM doc's *alternative* path; its default pulls via FTP `REST` resume with no mount at all). **VMix PCs run Windows, which natively serves SMB** — a real network filesystem protocol, designed to be mounted. There's no bridging trick needed here; SMB is the native way two machines share a filesystem on a LAN. That makes this the more natural fit for a mount-based approach, not a stretch of the ATEM pattern onto a different protocol.

Architecture

Same shape as the ATEM ingest, reusing the identical tools (`config/vmix-record-ingest/` mirrors `config/atem-iso-ingest/rclone-mount-rsync/`), as its own Unraid container (`192.168.1.18`):

```
VMix PC (192.168.2X.2X, SMB share) --[Tailscale, via that VMix node's own Slate AX]--> Unraid
|
rclone mount (SMB backend)
|
v
rsync --append into Nextcloud's
External Storage (Local) folder
|
targeted `occ files:scan`
|
v
visible in Nextcloud, growing
```

`rsync --append` (not `rsync`'s general checksum-diff algorithm) is what makes this incremental — it seeks straight to the destination's current size and transfers only the new tail, rather than reading/hashing the whole file. Same reasoning as the ATEM ingest's `rclone-mount-rsync` alternative; see that doc for the full explanation of why this differs from `rsync`'s usual delta-transfer mode. (Checked with `rsync 3.2.7`: on a growing file, a repeat `--append` pass sends only the new tail and the sender seeks straight to the old size. Two limits of the same mode: a file whose earlier bytes are rewritten in place — e.g. an MP4 header patched when recording stops — is never corrected by `--append` , and `--append-verify` skips any file that is already the same size on both sides, so it is not a whole-file integrity check. `VERIFY=1` therefore runs a full `--checksum --inplace` pass instead — see the script.)

SMB, like FTP, has no change notification in `rclone`, so a growing file's size on the mount only refreshes when `rclone`'s directory cache expires — default 5 minutes, which would quietly make the 90s sync interval a ~5-minute one. The script sets `--dir-cache-time 30s` (`rclone mount docs`).

Second write destination for live editing. Same as the ATEM ingest: the pull also writes to the edit suite's dedicated NAS (`192.168.22.x`) alongside Nextcloud's External Storage, one read off the VMix PC's SMB share, two writes. See

`docs/live-editing.md` for the editing subsystem this feeds — not yet implemented in `config/vmix-record-ingest/mount-and-sync.sh`, tracked in `docs/open-questions.md`.

A native Linux CIFS mount (`mount -t cifs`) is a simpler alternative to `rclone mount` for SMB specifically — no FUSE bridge needed at all, since the kernel has first-class SMB client support. The tooling here uses `rclone` anyway, to reuse the exact same config/script pattern already built for the ATEM ingest rather than introduce a third mechanism. Worth reconsidering if operational simplicity outweighs consistency.

Why bandwidth isn't a shared concern with the ATEM ingest

Each VMix node has its own dedicated GL-iNet Slate AX (GL-AXT1800) uplink and its own Tailscale connection — it is not part of the theatre subnet it physically sits near (see `docs/topology.md`). So VMix Node 1's record-ingest traffic rides over Node 1's own link, entirely separate from Theatre 1's ATEM ISO ingest and SRT traffic. There's no contention between the two ingest pipelines sharing a link — each just needs to fit inside its own node's ceiling — an estimated ~150–250 Mbps combined under Tailscale, not yet benchmarked (`docs/open-questions.md` #19).

Open items

- **Confirm what VMix is actually recording** — program mix only, or per-input ISO the same way the ATEMs do — and at what resolution/codecs/bitrate. None of this is assumed here; it directly determines the real bandwidth load on each VMix node's uplink. (Note `docs/server-specification.md` does pencil in ~30 Mbps per PC for pool sizing — an explicit placeholder, not a confirmed figure.)
- **Set up SMB sharing on the real PCs** — confirm the actual recording folder path and share name, and use a dedicated, minimal-privilege (read-only) local account for the ingest process rather than an admin/shared login.
- **Same partial-file-safety question as the ATEM ingest, but easier to answer:** the answer depends entirely on which container VMix is set to record (a plain MP4 with its index written only at the end is not readable until recording stops; a fragmented MP4 or a streaming-friendly container is). Not established here — check empirically (open a currently-recording file's ingested copy in a player), and check the *finished* copy too, since an append-only mirror can miss a header rewritten at stop (see above).
- **Confirm the version-retention setting on Nextcloud** covers these folders too (see the same caveat in `docs/atem-iso-ingest.md`) — same risk of accumulating snapshots on a repeatedly-changing external-storage file.

GLKVM-Cloud — remote administration for the GL-iNet router fleet

Self-hosted GLKVM-Cloud (`gl-inet/glkvm-cloud`), run as Docker containers on the consolidated services server, gives centralized remote access to all **14 GL-iNet Slate AX routers** (12 theatres + 2 VMix nodes) — their web admin UI and an SSH terminal, both reachable through one browser session, without exposing any individual router's admin interface to the WAN or needing 14 separate tunnels. Same self-hosted-over-vendor-cloud rationale already used for the DERP server and the UniFi Controller: no dependency on GL.iNet's own `glkvm.com` cloud, no subscription, no third party in the path when administering the router fleet during a live event.

There is no physical KVM unit in this design. GLKVM-Cloud's flagship product is a KVM-over-IP hardware line (the Comet/GL-RM1) for out-of-band access to bare-metal servers — not used here. What's actually in scope is the platform's separately-documented **HTTP/HTTPS web proxy and device-management capability for embedded devices like OpenWrt and Raspberry Pi** — the Slate AX routers already run OpenWrt/UCI (see `config/gl-inet/`), which puts them squarely in that supported category.

Why centralize router administration at all

14 identical GL-iNet Slate AX routers (12 theatre routers, 2 VMix node routers) is exactly the kind of fleet that benefits from one console instead of 14 separate admin logins — same motivation as Flock for the BirdDog Play fleet (see `docs/birddog-play-rationale.md`). GLKVM-Cloud's documented features that apply directly:

- **Web-based SSH terminal** to any registered router, through the browser — no SSH client, no per-router WAN exposure, no VPN client needed on the admin's own device.
- **HTTP/HTTPS proxy to each router's own web admin (LuCI)** — same centralization for the GUI side.
- **Batch command execution across multiple devices** — push a config check or command to some or all of the 14 routers at once, rather than 14 individual sessions.
- **User group management + LDAP/OIDC** — if more than one person needs scoped admin access during the event, without sharing one shared router password.

Software: self-hosted GLKVM-Cloud — two containers

The upstream `docker-compose/docker-compose.yml` (reference) defines **two** services, both landing on the consolidated services server with their own macvlan IPs (same pattern as every other container there):

Service	Image	IP	Ports	Purpose
<code>rttys</code>	<code>glzhitong/glkvm-cloud:latest</code>	<code>192.168.1.20</code>	443/tcp (Web UI), 5912/tcp (device connection), 10443/tcp (WebSocket proxy)	The actual GLKVM-Cloud application
<code>coturn</code>	<code>coturn/coturn:edge-alpine</code>	<code>192.168.1.22</code>	3478 tcp+udp	TURN relay for WebRTC (separate off-the-shelf container, not GL.iNet's own code)

`coturn` exists in the upstream stack for GLKVM-Cloud's live remote-desktop/KVM feature, which doesn't apply to routers (there's no framebuffer to capture on an OpenWrt device) — deployed anyway as the standard reference stack rather than assuming it's safe to drop; not confirmed whether `rttys` 's SSH-terminal/web-proxy features have any dependency on it internally. Revisit once actually running.

The image needs upstream's endpoint and templates mounted. The `glzhitong/glkvm-cloud` image's own endpoint is bare `rttys` with no config; upstream's compose overrides it with a bind-mounted `docker-entrypoint.sh` that renders `rttys.conf` and `turnserver.conf` from templates using the `RTTYS_*` / `TURN_*` / `GLKVM_ACCESS_IP` variables. Without those mounts the token and password are silently ignored. This repo's compose files now do the same, mounting from an upstream checkout.

Manual Docker/docker-compose deployment supports both x86_64 and arm64 (deployment docs). Minimum self-host requirements per GL.iNet: 1 CPU core, ≥1 GB RAM, ≥40 GB storage, ≥3 Mbps — trivial next to what BirdDog Central and the rest of the stack already need on this box.

Registering the 14 routers

Each Slate AX registers to the self-hosted instance by running a connection script copied from the GLKVM-Cloud web UI (OpenWrt supports SSH/shell, so this is a normal `opkg /script`-based install, same mechanism as any other device type GLKVM-Cloud supports). Since every router already runs Tailscale and accepts routes back to `192.168.1.0/24` (see `docs/tailscale.md`), **that registration traffic rides the existing tailnet — no WAN port-forward needed for it.** WAN exposure is only about the *admin's* browser reaching the GLKVM-Cloud web UI itself from a device that isn't on the tailnet (see below).

WAN reachability — and the DERP port collision

Remote (off-venue) admin access to the GLKVM-Cloud web UI needs to be reachable from the internet, the same way `derp.example.net` is (`docs/tailscale.md`) — useful since, per this repo's own rule, only the routers and the mothership's Tailscale container are on the tailnet — no admin laptops or other client devices (see `docs/tailscale.md`). But the Cloud Gateway has one public IP, and DERP already owns WAN `443/tcp` and WAN `3478/udp` — a straight 1:1 forward of GLKVM-Cloud's own port numbers would collide with both. Resolution: forward different WAN-side port numbers to GLKVM-Cloud's real (internal, unchanged) ports —

WAN port	Proto	Forwards to	Why remapped
8443	TCP	<code>192.168.1.20:443</code>	WAN 443/tcp already goes to DERP
10443	TCP	<code>192.168.1.20:10443</code>	no collision, forwarded as-is
3479	TCP+UDP	<code>192.168.1.22:3478</code>	WAN 3478/udp already goes to DERP's STUN; kept TCP alongside it on the same alternate port for one consistent "DERP = 3478, GLKVM-Cloud = 3479" mental model, even though 3478/tcp alone was actually free

Suggested hostname: `kvm.example.net`, same domain as `derp.example.net`, pointed at the same public IP — reached as `https://kvm.example.net:8443`.

What `GLKVM_ACCESS_IP` and `TURN_PORT` actually do (from upstream's `scripts/docker-entrypoint.sh` and `templates/`): `GLKVM_ACCESS_IP` is written into `rttys`'s `webrtc-ip` and `coturn`'s `external-ip`, and the auto-detect fallback only accepts an IPv4 address — so set it to the venue's **public IPv4**, not the hostname (and expect to update it if that IP isn't static). `TURN_PORT` is both the port `rttys` advertises for TURN and the port `coturn` listens on; since they're separate containers, this repo sets it to `3479` on `rttys` (the WAN-side port) and `3478` on `coturn` (where the WAN 3479 forward lands). The web UI port isn't advertised anywhere — the browser just uses `:8443` — and `10443` is forwarded unchanged.

Not fully verified against a real deployment — confirm before relying on it: the end-to-end behaviour of the `:8443` web UI and the remapped TURN port. If the remap doesn't work cleanly, the fallback is a second public IP or an SNI-routing reverse proxy in front of both DERP and GLKVM-Cloud on the literal 443 — more moving parts, only worth it if the simple remap doesn't work. Tracked in `docs/open-questions.md` (question 7).

See `config/glkvm-cloud/` for the docker-compose template, and `config/unifi/network-config.yaml` for the port forwards above.

Bandwidth analysis — venue-supplied VLANs

New constraint that changes the calculus: the 4 uplink VLANs are not our infrastructure — they're supplied by the venue over their existing structured cabling, each hard-capped at **1 Gbps**, and **each VLAN port is expensive**. The goal is minimum VLAN count at adequate performance, not the reverse. This replaces the earlier framing in `docs/open-questions.md` (which assumed we controlled VLAN addressing/count freely) — the *addressing* there is still fine, the *count* now has a real cost pressure behind it that this doc quantifies.

Assumption to confirm with the venue: the 1 Gbps figure is treated here as a standard switched-Ethernet full-duplex rating (1 Gbps *each direction*, not a shared aggregate) — true for virtually all modern structured cabling, but worth a one-line confirmation rather than assuming.

Revised 2026-10-06. Research that day found both load-bearing numbers this doc used to rest on were wrong — the ~10 Mbps-per-ATEM-ISO-stream basis and the A-1300 router's ~170 Mbps ceiling — and two design decisions followed: a **narrower live-ingest scope** (camera + program by default, see below) and a **different router, the GL.iNet Slate AX (GL-AXT1800)**. Every derived figure below was recomputed from the planning figures in the per-traffic table by a script, not adjusted by hand.

Tailscale/WireGuard overhead

Every figure below that crosses the tailnet (SRT, NDI, ATEM ingest, rclone) is wrapped in a WireGuard tunnel between the theatre's Slate AX and the mothership's Tailscale container — that adds a real, quantifiable tax on top of the raw payload. For an IPv4 outer path, WireGuard's per-packet overhead is 60 bytes (20B IP + 8B UDP + 32B WireGuard header + auth tag) (header breakdown). That is 4.0% of a 1500B packet — but Tailscale's tunnel MTU is **1280B**, not 1500B, so a full-size packet inside the tunnel carries 60/1280 ≈ **4.7%** overhead (≈4.4% for a default 1360B SRT packet). All figures in this doc have a flat 4% folded in, which understates every tailnet figure by under 1% (e.g. NDI ~130 → ~131 Mbps) — immaterial at the precision used here, so the figures are left as-is.

The 1280B tunnel MTU also matters for SRT itself, not just the overhead maths. SRT's default payload is 1316B (7 × 188B MPEG-TS packets), i.e. a 1360B IP packet — larger than Tailscale's 1280B MTU, so every SRT packet would be IP-fragmented (or dropped, if sent with DF set) on its way into the tunnel. Set the SRT payload size on the senders (VMix PCs, Restreamer's outputs) to **1128B** (6 × 188B → 1172B IP packet, fits under 1280B). See `docs/streaming-flow.md`.

Presenter internet does NOT get this tax. It's a separate path — presenter laptops go straight out to the internet via the Slate AX's normal NAT/WAN, never touching the Tailscale tunnel at all (see `docs/tailscale.md`: only the 14 Slate AX routers and the mothership run Tailscale; nothing routes presenter traffic through it). It may still share the same physical VLAN link and the same router (unconfirmed — see the recommendation at the end of this doc), but it never shares the encrypted tunnel or its overhead.

Real-world per-traffic-type figures (WireGuard overhead included where it applies)

The live-ingest scope this doc now sizes for (the same input map on all 12 ATEMs — see `docs/atem-iso-ingest.md`):

ATEM input	Source	Pulled live?
1	Camera	Yes (default)
2	Presenter laptop 1 (PowerPoint Main)	Optional — only if router headroom allows (<code>ATEM_ISO_INPUTS=1,2</code>)
3	Laptop 2 (VT Main / second presenter)	Optional (<code>ATEM_ISO_INPUTS=1,2,3</code>)
4–8	Backups / spare	Never live; stays on the ATEM SSD
Program recording	—	Always pulled ; the ATEM's Streaming/record quality set so this file is ~8–10 Mbps

The ATEM records every input regardless (ISO recording is all-or-nothing per switcher); anything not pulled live arrives via the physical DIT offload of the ATEM SSD or between sessions (docs/live-editing.md).

Planning figures are given as **low / typical / high**:

Traffic	Direction	Basis	Figure
SRT, baseline (static/motion-graphics)	Mothership → theatre	3-6 Mbps encoder for low-motion 1080p H.264 (4.5 Mbps midpoint used), +25% SRT overhead (Haivision — 25% is SRT's default <i>ceiling</i> for retransmission headroom, not steady-state use, so this is conservative), +4% WireGuard	~5.8 Mbps
SRT, peak (opening/closing, higher motion)	Mothership → theatre	6-8 Mbps encoder + 25% overhead + 4% WireGuard	~10.4 Mbps
NDI backup, active (1080p50, full — not HX)	Mothership → theatre	Official NDI spec (docs.ndi.video) — roughly constant regardless of content , unlike SRT's H.264 long-GOP; the "mostly static graphics" discount does not apply here. +4% WireGuard	~130 Mbps
ATEM camera ISO, per input	Theatre → mothership	H.264 VBR, not user-settable — Blackmagic's spec is "up to 70 Mb/s" (tech specs); only frame rate moves the target (~45 Mbps at 24/25/30p, 70 at 50/60p — forum); users report ~30-45 Mbps averages (e.g. ~15 GB/h per source ≈ 33 Mbps — forum). 70 = vendor cap	35 / 45 / 70 Mbps raw
ATEM laptop (slides) ISO, per input	Theatre → mothership	Estimate, unmeasured — mostly-static slides should encode well under a camera at the same fixed VBR target, but nothing published pins it. One measured file from a real unit replaces this row	15 / 25 / 40 Mbps raw
ATEM program recording	Theatre → mothership	Follows the configurable Streaming quality setting (one test measured ~4.8 Mbps); set to land at ~8-10 Mbps	10 Mbps raw
ATEM live ingest, default scope (camera + program)	Theatre → mothership	(camera + program) + 4% WireGuard	47 / 57 / 83 Mbps
ATEM Overseer monitoring stream	Theatre → mothership	10 Mbps per theatre (your figure) + 4% WireGuard — flat, doesn't scale with the ingest scope, see docs/topology.md	~10.4 Mbps
Flock BirdDog Play preview stream	Theatre → mothership	10 Mbps SRT per theatre (your figure) + 4% WireGuard — flat, same treatment as the Overseer stream, see docs/topology.md	~10.4 Mbps
rclone file sync, steady-state	Theatre → mothership	sporadic post-ingest updates only — see below	~5 Mbps avg
rclone file sync, worst case	Theatre → mothership	multiple laptops syncing simultaneously	~15 Mbps
rclone file sync, initial bulk ingest	Theatre → mothership	10-20 GB/theatre, one-time	not a sustained-load concern — see below
Presenter internet	Both, shared VLAN link, no WireGuard tax	web/slides/occasional demo video — unconfirmed whether it shares this VLAN or is separate infrastructure	~10-25 Mbps allowance

Resolved (2026-10-06): the old "10 Mbps per ISO stream" basis was wrong, and so was the router ceiling it was checked against. This doc previously sized ATEM ingest as 5-9 ISO streams × 10 Mbps (~62-94 Mbps per theatre) against the A-1300's ~170 Mbps. What the research found:

- **ISO files are ~35-70 Mbps each, and the rate can't be turned down.** The ATEM Mini Extreme ISO (and G2) record each ISO input as H.264 "up to 70 Mb/s" (Blackmagic tech specs). There is no ISO bitrate setting — only the frame rate changes the target (forum; the feature request is still open, forum). User reports average ~30-45 Mbps per ISO; older firmware showed 90-120 Mbps, with a silent drop around firmware 9.6.x (forum) — so check the firmware on the units too. ISO recording is all-or-nothing per switcher (inputs can't be disabled; empty inputs make near-empty files).
- **"10 Mbps" is realistic for the program file only**, which follows the configurable Streaming quality setting (~4.8 Mbps measured in one test).
- **The A-1300's 170 Mbps is a kernel-WireGuard figure, not a Tailscale one** — see the per-router section. Planning estimate for Tailscale on an A-1300: ~30-70 Mbps combined (unmeasured).

At 45 Mbps × 6 ISOs the old pull-everything plan would have needed ~280 Mbps per theatre before Overseer/Flock — nowhere near any travel router running Tailscale. Hence the two decisions this revision is built on: **pull live only**

the camera ISO + program by default (optionally one or two laptop ISOs, set per theatre with `ATEM_ISO_INPUTS_T<n>`), and **replace the A-1300 with the Slate AX**. One `ffprobe` (or file size ÷ duration) of a real camera and a real laptop ISO file from the actual ATEM, at the event's frame rate and firmware, still settles the exact numbers — tracked in `docs/open-questions.md` .

Initial bulk ingest isn't a bandwidth risk if scheduled right. 15 GB at a generously throttled 100 Mbps takes ~20 minutes; even at a conservative 50 Mbps, ~40 minutes. Run it during load-in/setup, not concurrent with a live session, and it's a non-issue regardless of VLAN count. Don't let this drive the VLAN-count decision.

Per-theatre sustained budget (during a live session, excluding the one-time bulk ingest)

Fixed upstream per theatre, whatever the ingest scope: Overseer (~10.4) + Flock (~10.4) + rclone (~5) = **~26 Mbps** (~36 with the rclone worst case). The ingest scope sits on top:

Live scope	Ingest (incl. WG)	Total upstream	Combined through the router (+ SRT down)
Camera + program (default)	47 / 57 / 83	73 / 83 / 109	~89-119
Camera + 1 laptop + program	62 / 83 / 125	88 / 109 / 151	~115-161
Camera + 2 laptops + program	78 / 109 / 166	104 / 135 / 192	~141-203
4 ISOs (2 cameras + 2 laptops) + program	114 / 156 / 239	140 / 182 / 265	~188-275

All Mbps, low / typical / high. The combined column runs from typical upstream + SRT baseline (5.8) to high upstream + SRT peak (10.4). The rclone worst case adds ~10 Mbps to any upstream figure (e.g. 109 → 119 at the default scope's high end). Downstream is SRT plus presenter internet: **~21 Mbps** = SRT baseline (~5.8 Mbps) + ~15 Mbps presenter; **35 Mbps** = SRT peak (~10.4 Mbps) + the 25 Mbps presenter ceiling. The laptop rows inherit the laptop-ISO **estimate** — treat them as less firm than the camera rows.

Upstream is still dominated by ATEM ingest — ~69% of typical upstream at the default scope, ~81% with both laptops added — with Overseer and Flock the flat secondary contributors. See `docs/topology.md` for what those two streams are for.

The per-router ceiling: why the A-1300 was replaced by the Slate AX

Every byte a theatre sends/receives over the tailnet has to pass through that one router's crypto, regardless of how much headroom the VLAN has — a **separate, tighter bottleneck** than the shared VLAN.

The A-1300's 170 Mbps was never a Tailscale figure. GL.iNet's number is its *kernel* WireGuard client-mode benchmark (GL-A1300 product page). Tailscale does its crypto in Go userspace (`wireguard-go`), and on the A-1300's 32-bit IPQ4018 (4× Cortex-A7 ~717 MHz, 256 MB RAM, kernel 5.4, firmware stuck at 4.5.x with Tailscale 1.58) Go has no optimised ChaCha20-Poly1305 assembly. Comparable gaps measured on other GL.iNet hardware: GL-MT1300 ~90 Mbps kernel WireGuard vs ~20 Mbps Tailscale (tailscale#10524); Beryl AX (64-bit) 300 Mbps WireGuard vs ~150 Mbps each way Tailscale (GL.iNet forum). **Planning estimate for Tailscale on an A-1300: ~30-70 Mbps combined (unmeasured).** That fails even the default scope in normal operation (~89 Mbps combined, typical) — so the A-1300 was dropped on 2026-10-06.

The replacement: GL.iNet Slate AX (GL-AXT1800), ×14. Qualcomm IPQ6000, 4× Cortex-A53 @ 1.2 GHz (64-bit, so Go's optimised crypto paths apply), 1× WAN + 2× LAN gigabit — the same port layout as the A-1300, so the LAN1 → ATEM / LAN2 → switch wiring in `docs/topology.md` is unchanged — USB 3.0, WireGuard up to 550 Mbps (vendor, kernel WireGuard), \$119.99 list (specs, product page). **Expected Tailscale throughput ~150-250 Mbps combined — an estimate** extrapolated from the Beryl AX measurement above; no Slate AX Tailscale benchmark was found. Benchmark one unit (`iperf3 --bidir` from a theatre LAN host to the mothership, direct path, not DERP) before buying 14 — this is still the single most load-bearing unverified number in the design.

Against that ~150-250 Mbps estimate (percentages at the low / high end of the range):

Scenario	Upstream	Downstream	Combined	vs. ~150-250 Mbps (est.)
Normal, default scope (camera + program), typical	83.0 Mbps	5.8 Mbps	88.8 Mbps	59% / 36% — comfortable
Normal, default scope, high	109.0 Mbps	10.4 Mbps	119.4 Mbps	80% / 48%
Normal, camera + 2 laptops + program, typical	135.0 Mbps	5.8 Mbps	140.8 Mbps	94% / 56% — fits, thin at the low end
Normal, 4 ISOs + program, typical	181.8 Mbps	5.8 Mbps	187.6 Mbps	125% / 75% — marginal
NDI fallback, this theatre only, default-scope ingest still running (typical)	83.0 Mbps	130.0 Mbps	213.0 Mbps	142% / 85% — only fits if the real figure lands in the upper part of the range
NDI fallback, ingest paused (Overseer + Flock + rclone + NDI)	25.8 Mbps	130.0 Mbps	155.8 Mbps (165.8 with rclone worst case)	104% / 62% (111% / 66%) — fits unless the benchmark lands at the very bottom of the estimate

Practical mitigation (unchanged in kind): if a theatre's SRT feed fails and it falls back to NDI, pause *that theatre's* ATEM ingest for the duration — near-real-time review footage is a lower priority than the live show staying on air, and the ATEM's own SSD keeps recording every input regardless. This is a cheap, config-only fallback response, not a design change. The floor it can't go below is the two monitoring streams plus NDI: 10.4 + 10.4 + 130 = **150.8 Mbps** — which is why the Slate AX benchmark needs to come in above ~160 Mbps combined for a single-theatre NDI fallback to be safe at all.

Worked through fully — NDI fallback + ATEM paused, at every level this design has a bottleneck at, since "under the ceiling" at one figure doesn't tell the whole story on its own (default scope, camera + program):

Level	Without the mitigation (ATEM still ingesting)	With it (ATEM paused)
Single theatre's own Slate AX (upstream + downstream combined, vs. ~150-250 Mbps est.)	213 Mbps typical / 239 Mbps high — only fits near the top of the estimate	155.8 Mbps realistic / 165.8 Mbps worst case — fits at ≥ ~170 Mbps; over at the very bottom of the estimate
That theatre's VLAN group, upstream side only (NDI itself doesn't touch upstream — full-duplex, separate capacity)	n/a — this is exactly what the mitigation removes	Drops to just rclone + Overseer + Flock per theatre (25.8-35.8 Mbps) — even a fully-loaded 6-theatre group sits at 15.5-21.5% upstream, nowhere near a concern
Mothership's own bonded NIC, mass-fallback disaster case (all 12 theatres on NDI at once)	Downstream 1,560 Mbps; upstream ~1,080-1,120 Mbps typical / ~1,390-1,430 Mbps high (incl. VMix) — both sides genuinely loaded	Downstream unchanged at 1,560 Mbps, but upstream drops to ~310-430 Mbps — the mitigation's biggest payoff shows up here
Central's own NDI feed in (the source VMix node's program arriving for redistribution — see docs/streaming-flow.md)	+130 Mbps inbound per active node program (max +260 if both nodes' programs are being redistributed) — rides the node's uplink, then the mothership NIC inbound	Same +130-260 Mbps — this feed persists through the mitigation; stacked on the unmitigated high-end inbound (~1,433 + 260 ≈ 1,693 Mbps) it stays under the bond's ~2,000 Mbps aggregate counted once. Counted properly it arrives twice (gateway → Tailscale container on .1.2 , then .1.2 → Central), see the hairpin note below

The one number worth remembering: ~151 Mbps — the floor a single theatre's router has to carry during an NDI fallback even with all ingest paused. The Slate AX benchmark has to clear it with room to spare. Pausing ATEM ingest fleet-wide during a genuine mass NDI fallback is unambiguously the right call regardless — it cuts the mothership's inbound from ~1,080-1,430 Mbps to ~310-430 Mbps at exactly the moment the network is under most stress.

Mothership NIC (2× bonded GbE)

Inbound at the mothership = 12 theatres' upstream + the 4 VMix PCs' record ingest (assumed 20-30 Mbps/PC, ~83-125 Mbps with WireGuard — see "VMix nodes' uplinks" below):

Live scope (all 12 theatres)	Inbound, typical	Inbound, high	vs. ~2,000 Mbps bond
Camera + program (default)	~1,080-1,120 Mbps	~1,390-1,430 Mbps (~1,550 with rclone worst case)	Fits; same order as the previously-documented worst case
Camera + 1 laptop + program	~1,390-1,430 Mbps	~1,890-1,930 Mbps	Typical fits; high is at the bond's edge

Live scope (all 12 theatres)	Inbound, typical	Inbound, high	vs. ~2,000 Mbps bond
Camera + 2 laptops + program	~1,700-1,750 Mbps	~2,390-2,430 Mbps	Over the bond at the high end — not fleet-wide on 2× GbE
4 ISOs + program	~2,270-2,310 Mbps	~3,260-3,300 Mbps	Not viable on 2× GbE

Outbound in the mass-NDI-fallback case is unchanged at $12 \times \sim 130 = \sim 1,560$ Mbps. Each theatre's traffic is one WireGuard flow to the bond's LACP hash (~109 Mbps per theatre at the default scope's high end), so the hash-imbalance caveat in docs/server-specification.md still applies. Rate actually landing on the recording pool from live ingest (no WireGuard): ~0.30 TB/h typical / ~0.43 TB/h high at the default scope, ~0.57 / ~0.86 TB/h with both laptops — the rest of each ATEM's recording arrives later via DIT offload, not over this NIC. Upshot: **laptop ISOs can be enabled per theatre where a theatre needs them, not fleet-wide**, unless the bond becomes 2× 2.5GbE.

Mothership-side routing caveat for the NIC figures above: those figures assume mothership LAN devices (BirdDog Central VM, the macvlan containers) hand theatre-bound traffic straight to the Tailscale subnet router on the Unraid host (192.168.1.2). If they instead follow their default gateway and the Cloud Gateway bounces it back to .2 via a static route (see static_routes in config/unifi/network-config.yaml), every such byte crosses the bond an extra time in each direction — at mass-NDI-fallback scale (1,560 Mbps out of Central) that alone would exceed the bond's ~2,000 Mbps. Give Central's VM a direct route to the theatre/VMix subnets via 192.168.1.2 ; for the macvlan containers note that by default the Unraid host can't reach its own macvlan containers at all, unless "Host access to custom networks" is enabled (docs/open-questions.md #18).

Router options compared (utilization column = NDI fallback + default-scope ingest + Overseer + Flock, typical: ~213 Mbps combined):

Model	WireGuard (vendor, kernel)	Tailscale (combined)	LAN ports	Form factor	~Price	Utilization
A-1300 (GL-A1300) — previous choice, rejected 2026-10-06	170 Mbps	~30-70 Mbps (est., 32-bit Cortex-A7)	2	Pocket travel router	~\$100	~300-710%
Slate AX (GL-AXT1800) — chosen	550 Mbps	~150-250 Mbps (est., from Beryl AX)	2	Travel router	\$119.99	~85-142%
Beryl AX (GL-MT3000)	300 Mbps	~150 Mbps each way (measured)	1 only	Pocket travel router	~\$100-140	— (port count rules it out)
Slate 7 (GL-BE3600)	~490-540 Mbps (GL.iNet figures differ)	not found	1 only	Travel router	—	— (port count rules it out)
Flint 2 (GL-MT6000)	900 Mbps	~300-450 Mbps (est.)	5 (2×2.5GbE+4×1GbE, minus WAN)	233×137×57mm, 761g	~\$170	~47-71%

Beryl AX and Slate 7 are traps for this design specifically — better throughput, but only 1 LAN port, so they can't replicate the dedicated-ATEM-port wiring (docs/topology.md) without an external switch ATEM would then have to share anyway, undoing exactly the isolation that wiring was for.

Flint 2 is the headroom option — ports to spare, comfortable margin even with ingest running during an NDI fallback — but it's a desktop router (233×137×57mm, 761g; ~6x the volume and ~4x the weight of the A-1300's 118×85×30mm, 181g), no longer travel-router class for a 14-unit touring kit, at ~40% more per unit than the Slate AX. Its Tailscale figure is an estimate too. Worth it if the Slate AX benchmark comes in low.

Fallback if the Slate AX can't be had: the A-1300 running GL.iNet's kernel WireGuard instead of Tailscale (~120-170 Mbps combined) — enough for the default scope in normal operation (~89-119 Mbps), but not for an NDI fallback, and it gives up Tailscale's NAT traversal/ACL model for those links.

Is the current plan (4 VLANs, 3 theatres each) sufficient?

Yes, at every ingest scope. Per-VLAN upstream at the default scope (camera + program): $3 \times 83 = \sim 250$ Mbps typical (25% of 1 Gbps), $3 \times 109 \approx \sim 330$ Mbps high (33%). With both laptop ISOs added on all three theatres: ~405 Mbps

typical / ~577 Mbps high (41-58%). Even the full 4-ISO scope tops out at ~795 Mbps (80%) — though that scope doesn't fit the routers anyway. Downstream is lower still. The per-router ceiling above is the one that bites first, and no VLAN count changes that.

Consolidating further (fewer, more expensive VLANs handling more theatres each)

Upstream per VLAN, typical / high (excluding the VMix nodes — see their section):

VLANs	Theatres/VLAN	Camera + program (default)	Camera + 1 laptop + program	Camera + 2 laptops + program	Verdict
4 (current)	3	249 / 327 Mbps (25% / 33%)	327 / 452 Mbps (33% / 45%)	405 / 577 Mbps (41% / 58%)	Comfortable at every scope
3	4	332 / 436 Mbps (33% / 44%)	436 / 602 Mbps (44% / 60%)	540 / 769 Mbps (54% / 77%)	Comfortable; laptops-everywhere getting warm at the high end
2	6	498 / 654 Mbps (50% / 65%)	654 / 904 Mbps (65% / 90%)	810 / 1,153 Mbps (81% / 115%)	Viable at camera + program; camera + 1 laptop only at typical rates; camera + 2 laptops too tight
1	12	996 / 1,308 Mbps (100% / 131%)	1,308 / 1,807 Mbps	1,620 / 2,306 Mbps	Not viable with any live ISO ingest

2 VLANs (6 theatres each) is the furthest safe consolidation with live ingest kept, and only at the default scope (or with one laptop ISO if the measured laptop rate comes in at or below the typical estimate). The Theatre 1-6 group also carries both VMix nodes under a 2-VLAN split, which pushes it hotter — see "VMix nodes' uplinks". This holds for normal operation (SRT primary) — **see the NDI fallback scenario below**, which is a downstream constraint and applies regardless of ingest scope.

If the NDI backup path actually activates — very different math

The bandwidth model above assumes SRT is what's actually flowing downstream. NDI (full, not HX) is a fundamentally different animal: **~130 Mbps at 1080p50 with WireGuard overhead, roughly constant regardless of content** (official NDI spec) — NDI's compression doesn't get cheaper for static/motion-graphics content the way SRT's H.264 long-GOP does. That's ~12-22x the SRT figures above, and it changes which scenario the VLAN-count decision actually needs to survive.

A single theatre falling back is a non-event at the VLAN level — 130 Mbps extra on one theatre's downstream fits comfortably even inside a fully-loaded 6-theatre VLAN group (though see the per-router finding above — that theatre's own Slate AX is a tighter constraint than the VLAN). The real VLAN-level question is a **mass fallback**: since BirdDog Central/NDI is a separate path from Restreamer/SRT, a Restreamer failure (a single container, single point of failure) would push every theatre onto NDI simultaneously — that's the scenario worth sizing for, not the single-theatre case.

VLANs	Theatres/VLAN	Downstream if ALL theatres in the group fall back to NDI at once (incl. presenter internet)	Verdict
4 (current)	3	450 Mbps (45%)	Still comfortable
3	4	600 Mbps (60%)	Workable, less margin than 4 but not tight
2	6	900 Mbps (90%)	Tight — no real margin left
1	12	1800 Mbps (180%)	Not viable at any scale

(Unchanged by the 2026-10-06 revision — this is all downstream, and the ingest scope only moves upstream.) A mass NDI fallback at 2 VLANs leaves almost no headroom (90%), while the current 4-VLAN split stays comfortable (45%).

Recommendation for this fork: if the design needs to gracefully survive a mass simultaneous NDI fallback (Restreamer going down, most plausible trigger), **stay at 3-4 VLANs rather than consolidating to 2**. If a mass fallback event is judged acceptable to handle operationally instead (e.g. running degraded/reduced-quality NDI, or accepting some theatres go

dark until Restreamer recovers), 2 VLANs remains viable at the default ingest scope — but that's an explicit choice being made, not a free consequence of the SRT numbers alone.

Splitting further instead (more, cheaper-per-theatre but more numerous VLANs)

Going the other direction (6 or 12 VLANs, 1-2 theatres each) buys essentially nothing on the bandwidth side — the current 4-VLAN plan runs at ~25-33% upstream at the default scope (~41-58% with both laptop ISOs everywhere) and a comfortable 45% even under a mass NDI fallback, nowhere near needing more headroom. It also does nothing for the per-router finding above, since that's a single-router problem, not something more VLANs can fix. The only genuine benefit of more/smaller VLANs is **fault isolation**: a failed venue port or switch takes out fewer theatres. Given the explicit cost pressure on VLAN count, the bandwidth numbers don't support this direction — it's a resilience trade-off to make consciously if at all, not one the traffic model asks for.

The highest-leverage lever: real-time ATEM ingest is the dominant cost

ATEM live ingest is ~69% of typical upstream per theatre at the default scope (~76% with one laptop ISO, ~81% with two) — still the dominant single item, which is exactly why the 2026-10-06 revision narrowed *what* gets pulled live rather than trying to pull everything. It exists purely for near-real-time review during the event — see `docs/atem-iso-ingest.md` 's "master vs. mirror" framing: **the ATEM's own SSD remains the authoritative master regardless**, offloaded in full after each session either way. Each laptop ISO added to `ATEM_ISO_INPUTS` costs ~16-42 Mbps per theatre (estimate); the camera ISO ~36-73 Mbps.

If VLAN count/cost turns out to be the binding constraint, deferring ATEM ingest to end-of-session pulls (instead of continuous near-real-time) drops per-theatre upstream from ~83 Mbps typical to ****~25.8 Mbps realistic / ~35.8 Mbps worst case**** (rclone + Overseer's 10.4 Mbps + Flock's 10.4 Mbps, none of which go away — **unlike ATEM ISO, neither monitoring stream is deferrable, both are live feeds**) — at which point **1 VLAN for all 12 theatres sits at ~31% realistic / ~43% worst-case utilization**. Still the single biggest lever available (bigger than any VLAN-topology change), and the only one that makes 1 VLAN viable for upstream. Losing same-day review access to footage (not the final deliverable) is the cost. Has nothing to do with the per-router NDI-fallback finding above — that's purely a downstream/NDI issue.

VMix nodes' uplinks

Not yet assigned to a VLAN group in the existing docs. Given the cost pressure, they shouldn't get dedicated VLANs of their own for just 2 nodes — fold each into whichever group its adjacent theatre already belongs to (Node 1 → Theatre 1's group, Node 2 → Theatre 4's group). VMix record-ingest bitrate is unconfirmed (see `docs/vmix-record-ingest.md` open items); at an **assumed** 20-30 Mbps/PC each node adds 2 PCs' worth (~42-62 Mbps with WireGuard).

- **4 VLANs:** each node's group (Theatres 1-3, Theatres 4-6) carries one node: ~291-311 Mbps typical / ~369-389 Mbps high at the default scope (29-39%); ~447-467 / ~618-639 Mbps with both laptop ISOs on (45-64%). Comfortable.
- **2 VLANs (Theatres 1-6 / 7-12):** both nodes land in the same group, so it carries 4 PCs' worth (~83-125 Mbps) on top of its theatres: ****~581-623 Mbps typical (58-62%) and ~737-779 Mbps high (74-78%)**** at the default scope; ~737-779 / ~987-1,028 Mbps (74-103%) at camera + 1 laptop. So under a 2-VLAN split the Theatre 1-6 group can't take laptop ISOs at the high end — re-check once VMix's real bitrate and the laptop ISO rate are measured, or move Node 2 to the other group.

The per-router NDI-fallback caveat bites *harder* here than at the theatres: during any NDI fallback, the source node's own uplink carries the ~130 Mbps NDI program feed up to BirdDog Central for redistribution (`docs/streaming-flow.md`). With that node's VMix record ingest paused it's 130 + the ~10-21 Mbps SRT program contribution ≈ 140-151 Mbps — 93-101% of the Slate AX's estimated ~150 Mbps floor, 56-60% at 250. With record ingest running alongside it, ~182-213 Mbps — fits only if the benchmark lands in the upper part of the range. Same pause-the-ingest lever as the theatre-side mitigation, applied at the node.

Recommendation

- **Confirm the full-duplex assumption with the venue** before finalizing any count.
- **Benchmark Tailscale on one Slate AX before buying 14.** `iperf3 --bidir` from a LAN host to the mothership over a direct (non-DERP) path. The ~150-250 Mbps used throughout is an estimate extrapolated from the Beryl AX; it needs to clear ~160 Mbps combined for a single-theatre NDI fallback (ingest paused) to be safe, and ~215-240 Mbps for one to ride through *without* pausing ingest. If it comes in low, the Flint 2 is the headroom option.
- **Measure one real camera ISO and one real laptop ISO file** (`ffprobe` , or `size ÷ duration`) at the event's frame rate and firmware. The laptop figure in particular is an unmeasured estimate.
- **Run the default live scope — camera ISO + program — on all 12 theatres** (`ATEM_ISO_INPUTS=1` , program file at ~8-10 Mbps via the Streaming quality setting). Enable laptop ISOs per theatre (`ATEM_ISO_INPUTS_T<n>=1,2` or `1,2,3`) only where a theatre needs them and the benchmark leaves room; not fleet-wide on the 2× GbE bond.
- **Set up the per-theatre NDI-fallback mitigation regardless of VLAN count** — pause that theatre's ATEM ingest when it falls back to NDI. Without it, one theatre at the default scope needs ~213 Mbps combined, over the Slate AX estimate's low end; with it, ~156 Mbps.
- **VLAN count: 4 VLANs remain comfortable at every ingest scope; 2 VLANs are viable at camera + program (50-65% upstream), or camera + 1 laptop at typical rates (65%),** but re-check the Theatre 1-6 group with both VMix nodes in it (up to ~78% high at the default scope) and the mass-NDI fork below. 1 VLAN is not viable with any live ISO ingest (~100% typical at the default scope) — only with ingest deferred to end-of-session.
- **Decide how much a mass NDI fallback needs to survive at full quality** — the actual fork in the VLAN-count recommendation:
 - If a Restreamer failure pushing all 12 theatres onto NDI simultaneously must keep working at full 1080p50 quality: **stay at 3-4 VLANs**. 2 VLANs puts that scenario at 90% utilization with no real margin.
 - If that event is acceptable to handle operationally instead: **2 VLANs (6 theatres each)** at the default ingest scope halves VLAN cost and keeps real-time camera + program ingest.
- **Fold both VMix nodes into their adjacent theatre's VLAN group** rather than requesting dedicated VLANs for them, but re-check their real bitrate against the margins above once it's confirmed.
- **Resolve the presenter-internet question** (shared with production VLANs or separate venue circuit) before treating any of the above as final — it's folded into the downstream figures above as a conservative shared-infrastructure assumption, and downstream isn't the binding constraint under normal SRT operation, but it should still be confirmed.

Tailscale — DERP relay avoidance

Who's actually on the tailnet

Only **15 nodes** run the Tailscale client: the 14 GL-iNet Slate AX routers (12 theatres + 2 VMix nodes) and the mothership's Tailscale container. Each advertises its own subnet (`--advertise-routes`) and accepts the others' (`--accept-routes`) — see `config/tailscale-up-all-devices.sh` .

No individual client device runs Tailscale. Not the PowerPoint/VT laptops, not the control laptops, not the ATEM, not BirdDog Play/Central, not the VMix PCs or P400 cameras. They all reach the rest of the network purely through their GL-iNet's normal LAN gateway — subnet routing makes `192.168.1.0/24` (and every other theatre's subnet, where ACLs allow it) reachable transparently, with zero Tailscale software installed on any of them.

Why direct connections are likely

Every node ultimately sits behind one shared mothership internet connection — a favorable case for direct (non-relayed) connections, since Tailscale can either use the true private LAN path or hairpin through the single shared public IP.

What determines it in practice:

1. **Whether the Ubiquiti firewall allows UDP between VLANs/subnets.** If yes → direct private-path connections, no internet round-trip. The uplink VLANs are otherwise blocked from Mothership-LAN, so `config/unifi/network-config.yaml` carries an explicit allow for UDP from the uplink VLANs to the mothership's Tailscale port (`192.168.1.2:41641`) ahead of that block.
2. **If inter-VLAN UDP is blocked,** Tailscale falls back to NAT hairpin via the shared public IP — usually works, but depends on the Cloud Gateway's hairpin-NAT behaviour. (The GL-iNet's own NAT is not stacked in this path: `tailscaled` runs on the GL-iNet itself, so its tunnel packets originate from the WAN address — only the Cloud Gateway's NAT applies.)
3. **DERP relay** is only the fallback when both of the above fail. Tailscale continuously retries and upgrades to direct when possible.

Verifying after setup

```
tailscale status      # shows direct vs relay per peer
tailscale ping <peer>
```

Self-hosted DERP server (decided — running on the mothership)

Runs as a Docker container on the consolidated Unraid server — `derp-server` , `192.168.1.16` (see `docs/ip-address-map.md`) — using Tailscale's own `derper` binary (official docs). There is **no official `tailscale/derper` image** — Tailscale's README says to build `cmd/derper` yourself — so `config/docker-compose.yml` builds one (`local/derper:built`). Pin `derper` and the subnet router's `tailscaled` to the same release, since `-verify-clients` checks one against the other.

Why this still needs a real, reachable hostname. DERP isn't reached over an already-established Tailscale tunnel to the peer that needs relaying — a node holds a persistent connection to its DERP server over its own normal WAN path, independent of any specific peer, precisely so it can bootstrap connectivity when direct WireGuard isn't available yet. That means the venue's shared-internet-connection hairpin trick (see above) is exactly what makes this work here: every

theatre's GL-iNet reaches the DERP server's hostname over its *own* WAN path, which loops back through the same shared public IP without the traffic ever actually leaving the venue's router. This requires:

- A real DNS hostname pointed at the mothership's public IP (a domain you control, or a dynamic-DNS hostname if you don't have one). Named `derp.example.net` — needs to actually be registered/pointed (or swapped for whatever domain/DDNS host you do control) before it resolves to anything.
- A port forward on the Cloud Gateway: WAN `443/tcp` → `192.168.1.16:443`, and WAN `3478/udp` → `192.168.1.16:3478` (STUN). See `config/unifi/network-config.yaml` . **443** for the DERP listener — matches Tailscale's own DERP fleet and is the most firewall-friendly port (many restrictive networks allow outbound 443 and nothing else), though it doesn't matter much for reachability here specifically since it's an internal hairpin, not roaming clients behind a hotel firewall.
- Let's Encrypt issues the cert automatically against that hostname (`-certmode=letsencrypt`) since the port forward makes it genuinely reachable for the ACME challenge. Upstream also recommends allowing 80/tcp; whether to forward it is open (`docs/open-questions.md` #2).

```
docker run -d --name derp-server \
  --network macvlan-mothership --ip 192.168.1.16 \
  -v /mnt/user/appdata/derper/certs:/app/certs \
  -v /var/run/tailscale/tailscaled.sock:/var/run/tailscale/tailscaled.sock \
  local/derper:built \
  -hostname=derp.example.net \
  -certmode=letsencrypt -certdir=/app/certs \
  -stun -stun-port=3478 \
  -verify-clients
```

(Attached to the `macvlan-mothership` network at `192.168.1.16` — the same macvlan setup every other mothership container uses, so no `-p` port publishes, which are inert under macvlan anyway. The canonical definition of this service lives in `config/docker-compose.yml` ; this standalone command is just the same thing spelled out.)

`-verify-clients` restricts relay use to nodes already on this tailnet — needs the container to reach the host's `tailscaled`, which is exactly what the `tailscaled.sock` mount above is for. Recommended so the DERP server isn't an open relay; drop both the flag and the mount if that's not a concern.

Register it with the tailnet by adding a custom region to the ACL policy — see the `derpMap` block in `config/tailscale-acl.json`. Kept `OmitDefaultRegions: false` (the safer default) — Tailscale prefers this DERP node when it's faster, but still falls back to the public regions if the self-hosted one goes down, rather than losing relay fallback entirely.

Verify with `tailscale netcheck` — it reports which DERP region a node prefers.

Config sketch

See `config/tailscale-up-commands.sh` for the per-role `tailscale up` invocations and `config/tailscale-acl.json` for the ACL policy. Note that a tag in an ACL (e.g. `tag:theatre`) only covers the tagged routers' own Tailscale IPs, not the subnets they advertise — so the ACL also lists each subnet as a CIDR host alias (`mothership-lan`, `theatre-1-lan`, ...), as a destination *and* as a source, because a device behind a subnet router keeps its own LAN address when it starts a connection (an ATEM's monitoring stream to Overseer, a Play's preview to Flock, a container pulling from an ATEM). Theatre ranges are only ever paired with the mothership range, so theatres still can't reach each other. Without those, the ATEM ingest, SRT fan-out and every other device-to-device flow would be denied (subnet routers + ACLs).

Routes still need approving in the Tailscale admin console (or via `autoApprovers` keyed to tag/CIDR) before they take effect — advertising a route alone is not sufficient.

Open questions

Resolved

- **Subnet numbering.** Spec said 12 theatres on "192.168.2-12.x", which is only 11 subnets. Confirmed mapping: mothership on `.1.x`, Theatre 1 → `.2.x` ... Theatre 12 → `.13.x` (contiguous after mothership). Used throughout this repo.
- **Mothership router: Ubiquiti Cloud Gateway.** Cloud Gateway (UniFi-OS-based, same family as the Dream Machine) generally can't run Tailscale natively without an unofficial container hack. Decision: the mothership subnet-router role runs as a container on the consolidated services server instead of the Cloud Gateway itself — see `docs/topology.md`.
- **Mothership service consolidation: single physical server running Unraid.** Nextcloud, Restreamer, BirdDog Central, and the NDI Discovery Server all move onto one box instead of being separate machines — the router and the 4 ready-room PCs stay physical. BirdDog Central is Windows-only, so it runs as the design's one Windows VM — routing/control, not encode, no GPU needed. (An earlier revision also carried a mothership VMix instance VM with a passthrough GPU; it was removed — see #10 below.) Everything else (Nextcloud, Restreamer, NDI Discovery Server, Tailscale subnet router) runs as Docker containers. Chose **Unraid over TrueNAS SCALE** for its polished one-click container experience (the original GPU-passthrough-maturity tiebreaker no longer applies with the VMix VM gone), accepting a license cost that TrueNAS (free) doesn't have — as of 2026 Unraid sells Starter (49, *upto6storagedevices*) and *Unleashed* (109, unlimited) with one year of updates then an optional 36/yr, *or Lifetime* (249, updates included) (Unraid pricing). The old "\$129 Lifetime" figure was the pre-2024 Pro price. This build's pools are 6 drives with the mirrored recording pool, 8 with RAID-Z1, so Starter is at or over its limit — Unleashed or Lifetime. See `docs/topology.md` for the full breakdown.
- **Theatre router: GL-iNet Slate AX (GL-AXT1800)** (*previously the A-1300 (Slate Plus); replaced 2026-10-06 — see the router-change entry below*). Qualcomm IPQ6000, 4× Cortex-A53 @ 1.2 GHz (64-bit), 1× WAN + 2× LAN gigabit — the same port layout as the A-1300, so the LAN1 → ATEM / LAN2 → switch wiring is unchanged — WireGuard up to 550 Mbps (vendor, kernel WireGuard), \$119.99 list (GL.iNet specs). Expected Tailscale throughput **~150–250 Mbps combined — an estimate** extrapolated from a Beryl AX measurement, not a Slate AX benchmark (#19). Each theatre's router only ever carries *its own* theatre's traffic — the incoming SRT feed to its BirdDog Play, its ATEM ISO ingest, the Overseer/Flock monitoring streams and its laptops' rclone sync — not the aggregate across all 12 theatres (that converges at the mothership). Per-router upstream at the default live scope (camera ISO + program) is ~73 / 83 / 109 Mbps (low / typical / high); camera + 2 laptops + program ~104 / 135 / 192 Mbps. Full model in `docs/bandwidth-analysis.md` 's "The per-router ceiling: why the A-1300 was replaced by the Slate AX" section.
- **VMix node routers: also GL-iNet Slate AX** (previously A-1300, replaced with the theatre routers). Same model as the 12 theatre routers — 14 Slate AX units total across the network. Both LAN ports bridge together (no dedicated-port split needed here, unlike the theatre routers' BirdDog Play case). Configs generated: `config/gl-inet/vmix-node-1.uci` and `vmix-node-2.uci`.
- **NDI backup path discovery.** Confirmed Tailscale can't help directly — it's point-to-point WireGuard and doesn't forward multicast/mDNS (a still-open Tailscale feature request, not something MagicDNS works around). Plan: run an **NDI Discovery Server** on the mothership (unicast TCP, default port 5959 — built into NDI 5+ specifically for WAN/VPN cases like this), and point BirdDog Central plus all 12 BirdDog Play units at it. See `docs/streaming-flow.md` for the full plan.
- **BirdDog Play Discovery Server support — confirmed.** BirdUI (PLAY's web admin) has a Network panel with an NDI Discovery Server toggle: switch it on, enter a comma-delimited list of server IP(s), apply. BirdDog's own docs describe this as the mechanism for finding sources "on different subnets." Sources: BirdDog PLAY User Guide, BirdUI User Guide. Default admin password (`birddog`) should be changed on all 12 units before the event.

- **Consolidated server: hardware already on hand.** Physical box already exists — no procurement needed. Known so far: 2× gigabit NICs, a "decent" discrete GPU. Full spec (CPU/motherboard, RAM, exact GPU model) to be confirmed later — a calculated **target** minimum spec to check it against (storage pool layout/sizing, RAM, CPU, GPU tier, NIC validation, all worked from this box's actual workload rather than guessed) is now in `docs/server-specification.md`.

- **Full device inventory + configs generated.** Every device on the network now has a concrete IP — see `docs/ip-address-map.md` (133 devices total) and the redrawn `diagrams/topology.svg`. Configs generated to match: `config/gl-inet/` (UCI network config for all 14 routers — 12 theatres + 2 VMix nodes), `config/tailscale-up-all-devices.sh` (all 15 tailnet nodes instantiated), and `config/unifi/` (Cloud Gateway VLANs/DHCP/firewall reference).

Two things introduced *during config generation* that weren't decided before — flagged here since they're new, not previously confirmed:

- **Uplink VLAN transit addressing** (`10.10.1-4.0/24` , VLAN tags 101–104) for the 4 physical VLAN groups' GL-iNet WAN ports. Doesn't affect anything inside the theatres (still NAT'd behind each Slate AX) or Tailscale (rides over whatever WAN IP is handed out) — purely a Cloud Gateway-side detail. Change freely.
 - **Docker networking mode per mothership service:** Nextcloud/Restreamer/NDI Discovery Server/DERP server assigned dedicated IPs via macvlan; Tailscale subnet router uses host networking (required for route advertisement, shares the Unraid host's `.2`). Both are standard Unraid patterns, not unusual choices, but worth confirming once you're actually configuring Docker on the box.
- **2 gigabit NICs: bonded (802.3ad/LACP), not split.** SRT bandwidth isn't constant, and this box handles many simultaneous flows (12 theatres' SRT fan-out to distinct destinations, plus rclone/Nextcloud/BirdDog Central/NDI Discovery Server/DERP traffic) — LACP's hash-based distribution spreads those across both links for real aggregate headroom, rather than a static split that leaves one NIC idle whenever traffic doesn't match the assumed pattern. See `docs/topology.md` for the full reasoning, the Unraid setup steps, and the LACP rate/hash-policy gotcha with Ubiquiti gear.
- **Self-hosted DERP server added.** Runs as a Docker container on the Unraid box (`192.168.1.16`), registered with the tailnet as DERP region 900, `RegionCode: "example"` , `hostname derp.example.net` , port **443** (matches Tailscale's own DERP fleet, most firewall-friendly choice) — see `config/tailscale-acl.json` . Kept `OmitDefaultRegions: false` so Tailscale's public DERP regions remain available as a fallback if this one goes down. See `docs/tailscale.md` for the full setup (container flags, port forward, cert handling).
- **Cloud Gateway assumed to support LAG.** (*A working assumption, not a confirmed fact — still to verify, see #17.*) Proceeding on that assumption rather than confirming the exact model first. Fallback already agreed if it turns out not to: drop an intermediate LACP-capable switch in between and bond the Unraid server's NICs through that instead of directly into the Cloud Gateway — the bond itself doesn't care which device terminates it, only that *something* in the path speaks LACP.
- **ATEM ISO ingest architecture decided, two documented approaches.** Each ATEM's built-in FTP server (confirmed live-accessible during recording) is pulled continuously by a new Unraid container (`192.168.1.17`) over Tailscale subnet routing. Plain rsync can't connect to the ATEM at all (no SSH/rsync daemon), and a naive whole-file re-sync is ruled out by the bandwidth math for any real session length — so genuinely incremental transfer is required, achieved either via **FTP's REST /resume command directly** (`pull-iso.py` , the default — fewer moving parts) or via `rclone mount + rsync --append` (`rclone-mount-rsync/` — off-the-shelf tools, at the cost of 12 FUSE mounts to supervise). Both write directly into Nextcloud's External Storage (Local) mount, so there's no separate upload step and no need to slice/chunk the video file itself either way. Full comparison in `docs/atem-iso-ingest.md` .
- **VMix record ingest added, same mechanism as the ATEM ingest.** All 4 VMix PCs' recordings pulled by a new Unraid container (`192.168.1.18`) via `rclone mount + rsync --append` , targeting each PC's Windows SMB share instead of an FTP server. SMB is a native network filesystem protocol, so this is arguably a better fit for a mount-based approach than the ATEM's FTP-only case was, not a stretch of the same pattern. Each VMix node has its own dedicated Slate AX uplink (not shared with the theatre it sits near), so this traffic never competes with that

theatre's ATEM ingest or SRT feed. Full design in `docs/vmix-record-ingest.md`, tooling in `config/vmix-record-ingest/`.

- **Self-hosted UniFi Controller added to the consolidated server.** A UniFi Network Application container (`192.168.1.19`) joins the rest of the Docker stack, same self-hosted-over-cloud.ui.com rationale already used for DERP. The Cloud Gateway has its own built-in controller and doesn't require this — adopting it in is optional, purely a local/independent management console, not a change to any traffic path in `config/unifi/network-config.yaml`. See `config/unifi/README.md`.
- **Self-hosted GLKVM-Cloud added for centralized administration of the 14 GL-iNet routers.** No physical KVM hardware involved — this uses GLKVM-Cloud's separately documented HTTP/HTTPS web-proxy and device-management support for embedded OpenWrt devices (which the Slate AX routers are), giving one browser-based SSH terminal + web-admin proxy for all 14 routers instead of 14 separate sessions. Same self-hosted-over-vendor-cloud rationale as DERP and the UniFi Controller, avoiding GL.iNet's own `glkvm.com`. Two containers (`rtty`s on `192.168.1.20`, `coturn` on `192.168.1.22`); router registration rides the existing tailnet (routers already accept routes to `192.168.1.0/24`), no WAN exposure needed for that part. Surfaced a real conflict during design: GLKVM-Cloud's documented ports (`443/tcp`, `3478/tcp+udp`) collide with the DERP server's existing WAN forwards on those same numbers — resolved by forwarding different WAN-side port numbers (`8443`, `3479`) to GLKVM-Cloud's unchanged internal ports (for the admin's own browser access), rather than moving DERP off 443 and losing its firewall-friendliness rationale. Not yet confirmed against a real deployment: whether `GLKVM_ACCESS_IP` (the env var that tells the app what address to hand back to clients) accepts this WAN-side remap cleanly, or whether `coturn` needs its own separate external-address setting — also unclear whether `coturn` /TURN is needed at all for the SSH-terminal/web-proxy features actually in use here, versus only for GLKVM-Cloud's KVM-specific remote-desktop feature, which doesn't apply to routers. Full design in `docs/glkvmm-cloud.md`, stack in `config/glkvmm-cloud/`.
- **Bandwidth modeled against venue-supplied VLANs — 2 VLANs (not 4) recommended, but reconsider given how thin that margin has gotten.** The 4 uplink VLANs turned out to be venue-supplied, 1 Gbps each, with expensive ports — changes the goal from "isolate cleanly" to "minimize VLAN count at adequate performance." Real numbers (recomputed 2026-10-06 for the common input map and corrected ISO bitrates — see the entries below): per-theatre upstream is ~83 Mbps typical / ~109 high at the default live scope (camera ISO + program), ~135 / ~192 with both laptop inputs live. At the current 3-theatres-per-VLAN split that's ~250–330 Mbps per VLAN at the default scope (~405–577 with both laptops) — 4 VLANs are comfortable at every scope. 2 VLANs (6 theatres each) run ~500 Mbps typical at the default scope and ~654 with one laptop input, viable; ~810 typical with both laptop inputs is too tight. **1 VLAN for all 12 theatres is at capacity (~1,000 Mbps typical) even at the default scope** — viable only if ATEM ingest is deferred to end-of-session pulls instead of real-time. Full model in `docs/bandwidth-analysis.md`. (Note for the 2-VLAN option: both VMix nodes uplink next to Theatres 1 and 4, so under a 2-VLAN split they both land in the Theatre 1-6 group — re-check that group, and the mass-NDI-fallback case, against that doc's "VMix nodes' uplinks" section.)
- **ATEM Overseer + ATEM Fleet Admin added to the mothership.** Two of the user's own separate projects, deployed as Docker containers (`192.168.1.21` and `192.168.1.23`) alongside everything else — Overseer for fleet-wide monitoring/tally (every theatre's ATEM sends it a dedicated 10 Mbps monitoring stream, folded into the bandwidth model above as a flat per-theatre addition that doesn't scale with ATEM channel count), Fleet Admin for bulk provisioning (reaching all 12 theatres' ATEMs the same way ATEM ISO Ingest does, over the existing Tailscale subnet routing). Neither is built from source in this repo — `config/docker-compose.yml` uses placeholder image references pending confirmation of each project's actual published container image.
- **Flock added to the mothership.** A third of the user's own projects, deployed alongside the other two (`192.168.1.24`) — the BirdDog Play fleet manager already referenced elsewhere in this design (LAN discovery, tag-based grouping, BirdUI-parity settings, batch edits — see `docs/birddog-play-rationale.md`). Each theatre's BirdDog Play sends Flock a 10 Mbps SRT preview stream (confirmed by the user, not assumed), folded into the bandwidth model the same way as the Overseer stream — another flat per-theatre addition. Same not-built-from-source caveat as Overseer/Fleet Admin above.

- **Theatre/VMix router changed: GL.iNet Slate AX (GL-AXT1800) ×14 replaces the A-1300 (2026-10-06).** The A-1300's ~170 Mbps is GL.iNet's *kernel* WireGuard figure (GL.iNet); Tailscale does its crypto in userspace Go (`wireguard-go`), and on the A-1300's 32-bit IPQ4018 (4× Cortex-A7 ~717 MHz, 256 MB RAM, firmware stuck at 4.5.x with Tailscale 1.58) Go has no optimised ChaCha20-Poly1305 assembly. Comparable gaps: GL-MT1300 ~90 Mbps WireGuard vs. ~20 Tailscale (tailscale#10524); Beryl AX (64-bit) 300 WireGuard vs. ~150 Mbps each way Tailscale (GL.iNet forum). Planning estimate for Tailscale on an A-1300: ~30–70 Mbps combined (unmeasured) — below even the default live scope. The Slate AX (64-bit, same 1 WAN + 2 LAN layout) is estimated at ~150–250 Mbps combined, pending a benchmark (#19). Rejected alternatives: Slate 7 (GL-BE3600) — only one LAN port; Flint 2 (GL-MT6000, ~300–450 Mbps Tailscale est.) — the headroom option, but desktop form factor. Fallback if the Slate AX can't be had: the A-1300 running GL.iNet's kernel WireGuard instead of Tailscale (~120–170 Mbps combined; camera + program only, no NDI fallback). Router count unchanged at 14.
- **Common theatre input map + selective live ISO pull (2026-10-06).** Every ATEM is patched identically: input 1 camera, 2 presenter laptop 1 (PowerPoint Main), 3 laptop 2 (VT Main / second presenter), 4–8 backups/spare. The ATEM records all inputs to its SSD regardless (ISO recording is all-or-nothing, bitrate not user-settable), but only the **program file plus the ISO inputs in `ATEM_ISO_INPUTS`** (default `1`, the camera; `1,2` / `1,2,3` add the laptops where headroom allows) are pulled live; the program record quality is set to ~8–10 Mbps. Everything else arrives via the physical DIT offload (`docs/live-editing.md`). Implemented in `pull-iso.py` (recursive walk + input filter, `ATEM_ISO_INPUT_PATTERN` override) and in the `rclone-mount-rsync/` alternative (rsync filters from the same variable); `docker-compose.yml` and `.env.template` carry `ATEM_ISO_INPUTS=1`. Design in `docs/atem-iso-ingest.md` § Common theatre input map.
- **Live editing subsystem added — new `192.168.22.0/24` subnet, own 10GbE LAN, not on the Tailscale mesh.** 2× MacBook Pro editing workstations plus dedicated fast storage at the mothership, editing event content as it arrives rather than after the event. Deliberately separate from the theatre-facing network — different traffic pattern (sustained multi-gigabit editing I/O vs. the bursty low-bitrate streams everything else here is sized for) and a genuinely different failure domain. Key decisions made: dedicated NAS (not Nextcloud's own storage), dual-write from the same rclone/rsync pull already built for ingest rather than a chained re-sync; a dedicated Mac mini running both the Resolve Project Server and Remote Render (needed because resolve-configurator builds one shared show project both editors work against, not independent copies — confirmed **not** possible on a Blackmagic Cloud Store appliance itself, storage-only, no general compute); Blackmagic Cloud's internet sync service not needed for a single-venue setup. Full design, including the ATEM-pull-to-finished-edit workflow and the DIT physical-media ingest tool comparison (ShotPut Pro/OffShoot vs. Silverstack vs. FoolCat/o/PARASHOOT as companion tools), in `docs/live-editing.md`.

Still open — verify before building

0. **Resolved by decision (2026-10-06) — measurement still outstanding.** (Numbered 0, not 1, so the long-standing #1-#20 references across the repo stay valid.) The old model's 10 Mbps per ISO stream was wrong: Blackmagic specifies ISO files at up to 70 Mb/s, not user-settable (~45 Mbps at 24/25/30p, ~70 at 50/60p), with user reports averaging ~30–45 Mbps. **Decided:** the common input map and a live pull of the program recording plus only the ISO inputs in `ATEM_ISO_INPUTS` (default: the camera), the rest by physical DIT offload, on a Slate AX router — see the two Resolved entries above. Planning figures: camera ISO 35 / 45 / 70 Mbps, laptop ISO 15 / 25 / 40 Mbps (estimate), program 10 Mbps. **Still to measure on a real unit** (`ffprobe`, or `size ÷ duration`), at the event's frame rate and firmware (older firmware recorded at 90–120 Mbps): a camera ISO file, a laptop/slides ISO file, and the program file at the chosen Streaming/record quality setting (target ~8–10 Mbps). The laptop figure is pure estimate; one file pins it. Figures in `docs/atem-iso-ingest.md` § Bandwidth, `docs/bandwidth-analysis.md` and `docs/server-specification.md`.
1. **Check the existing server's real spec against the calculated target in `docs/server-specification.md`.** In particular: does it have validated (not merely tolerated) ECC support, enough PCIe lanes for the two NVMe pools (up to 6 drives) plus the 10GbE edit-LAN NIC, and enough RAM/cores (target: 12 cores/24 threads, 64–96 GB RAM) to run the BirdDog Central Windows VM plus 12 Docker containers concurrently without contention during a live

event. (GPU passthrough/IOMMU checks dropped from this list — nothing needs them since the VMix VM was removed, see #10.)

2. **Actually register/point `derp.example.net`** (or substitute whatever domain/DDNS host you end up controlling) at the mothership's public IP, and confirm the WAN port forward (443/tcp, 3478/udp) is in place before relying on it — the name and port are picked, but nothing resolves until the DNS record and port forward both exist. Upstream's `derper` guide also says to permit **80/tcp** (the design forwards only 443 + 3478 — decide whether to add it), and that there is no published `derper` package/image: `config/docker-compose.yml` now builds it from source, and `- verify-clients` wants `derper` and the subnet router's `tailscaled` built from the same release (derper README).
3. **Verify the DERP hairpin path doesn't get caught by the uplink-VLAN firewall block.** `config/unifi/network-config.yaml` blocks the 4 uplink VLANs from reaching Mothership-LAN directly (defense-in-depth for cross-theatre isolation) — that should be a different path from the DERP hairpin (which arrives via the WAN zone after NAT, not directly from an Uplink-VLAN zone), but exact zone/hairpin behavior is Cloud-Gateway-firmware-specific. Confirm with `tailscale netcheck` on a theatre router once built, before assuming the self-hosted DERP server is actually reachable.
4. **Confirm the real ATEM ISO bitrates, and empirically verify partial-file playability.** The live scope is now fixed by the common input map rather than an assumed active-channel count; the per-source bitrates still need measuring (#0). Whether a file copied mid-recording is genuinely valid (rather than just "very likely, given Blackmagic's marketed edit-while-recording capability") should be tested against a real unit before relying on it operationally. See `docs/atem-iso-ingest.md` for the full open-items list, including confirming the ATEM's actual FTP credentials/file layout and tuning the pull/scan intervals and Nextcloud version-retention settings once this is running for real.
5. **Confirm what VMix is actually recording, and set up real SMB shares/credentials.** Neither the recording mode (program mix vs. per-input ISO) nor bitrate/codecs, nor the actual share name/folder path/account on the 4 VMix PCs, is assumed here — see `docs/vmix-record-ingest.md` for the full open-items list.
6. **Confirm with the venue: final VLAN count, the full-duplex-1Gbps assumption, and whether presenter internet shares the production VLANs or is separate infrastructure.** `docs/bandwidth-analysis.md` recommends 2 VLANs over the current 4, but that's a real cost/procurement decision, not something this repo can finalize alone. If real-time ATEM ingest turns out incompatible with the venue's final VLAN allocation, deferring it to end-of-session pulls is the fallback — same doc.
7. **Confirm the WAN-remapped GLKVM-Cloud ports (8443 , 3479) work end-to-end.** Partly answered from upstream's own endpoint and templates (`docker-compose/`): `GLKVM_ACCESS_IP` is written into `rttys`'s `webrtc-ip` and `coturn`'s `external-ip`, and its auto-detect fallback only accepts IPv4 — so it should be the venue's **public IPv4**, not the `kvm.example.net` hostname. `coturn` gets its external address from that same variable (no separate setting needed), and `TURN_PORT` is both the port `rttys` advertises and the port `coturn` listens on, so the compose files now set it per container (3479 advertised, 3478 listened). Still untested: whether the web UI behaves on :8443, and what happens if the public IP isn't static. See `docs/glkvmm-cloud.md` for the fallback (second public IP or an SNI-routing reverse proxy) if the plain env var doesn't cover it. Also confirm whether `coturn` is needed at all for the SSH-terminal/web-proxy features actually in use here — it may only matter for GLKVM-Cloud's KVM-specific remote-desktop feature, unused since there's no KVM hardware in this design.
8. **New assumptions introduced by `config/docker-compose.yml`**, none confirmed against a real deployment: the NDI Discovery Server image (`pnxr/ndi-discovery-minimal`, a third-party community build, not NDI/NewTek-published — Docker Hub shows it last pushed 2022-09-18 on NDI SDK 5.5, so treat it as unmaintained and pin/fork or replace it), the MongoDB version the UniFi Controller needs (drifts with the controller image's own version, not fixed here), and MariaDB + Redis as Nextcloud's DB/cache backend (chosen over SQLite — this box has a lot of concurrent writers across 12 theatres' rclone syncs plus both ingest pipelines, which official Nextcloud guidance says SQLite handles poorly; not benchmarked here either way).
9. **Confirm actual event duration (day count × active-recording hours/day)** before treating the 8 TB recording pool in `docs/server-specification.md` as settled — that document calculates it covers roughly ~16.5-23 hours at the

default live scope (camera ISO + program per theatre, high to typical rates; ~1.6-2.9 typical event days), or only ~8.7-12.9 hours if both laptop inputs are pulled live fleet-wide — on planning figures, not measurements (#0) — but this repo doesn't establish the event's real length anywhere. If it runs longer without a periodic archive-off step, either the pool needs to grow or an offload step needs adding to the ingest design.

10. **Resolved — by removal.** This slot used to ask what the mothership's own VMix instance VM actually did (its role was never established anywhere in this repo, unlike the well-documented VMix *node* PCs). The answer arrived as a design change: **the mothership VMix VM was removed entirely**. The theatre program feeds originate from the VMix node PCs via Restreamer (`docs/streaming-flow.md`), which never depended on it. Knock-ons all applied: no GPU/passthrough/IOMMU requirement remains anywhere in the spec, RAM/CPU budgets dropped (see `docs/server-specification.md`), BirdDog Central is now the design's only VM, and the device count is 133. (Number kept so cross-references to #11-#15 stay stable.)
11. **Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's actual published container images** (or that they need to be built from source instead) before deploying `config/docker-compose.yml` — all three use placeholder `ghcr.io/...` references. Also confirm the actual mechanism behind both monitoring streams `docs/bandwidth-analysis.md` takes as given inputs, not verified against real hardware: Overseer's 10 Mbps ATEM stream (does the ATEM Mini Extreme ISO genuinely support two independent simultaneous stream outputs — its own hardware streaming engine feeding both Restreamer and Overseer at once?), and Flock's 10 Mbps BirdDog Play preview stream (same question for BirdDog Play — decoding its primary program feed while simultaneously *originating* a second SRT stream back to Flock is a different capability than the receive-only role it plays everywhere else in this design).
12. **Decide how much of the event's footage the live-editing subsystem actually needs access to** — the full 8TB recording pool, or a curated subset — before sizing the edit-suite NAS in `docs/live-editing.md` Decision 1.
13. **Confirm combining Project Server and Remote Render on one Mac mini holds up under real load** — `docs/live-editing.md` Decision 2 treats this as architecturally sound but thinly documented by both Blackmagic and practitioners; worth a real pre-event test, not just a design-time assumption.
14. **Confirm whether this event uses any standalone cameras with SD/CFexpress cards** beyond the NDI-fed BirdDog P400s — affects whether the DIT physical-media ingest workflow in `docs/live-editing.md` has real camera cards to process beyond the ATEM's own USB SSD, and whether Silverstack's deeper metadata/lens-data feature set becomes worth its cost over ShotPut Pro/OffShoot.
15. **Implement the second write destination in the actual ingest scripts.** `docs/live-editing.md` documents `pull-iso.py` and `mount-and-sync.sh` writing to the edit-suite NAS alongside Nextcloud — that's a real code change neither script has yet (`config/atem-iso-ingest/pull-iso.py` , `config/vmix-record-ingest/mount-and-sync.sh` currently write one destination each). Not done as part of documenting the design.
16. **Verify the settled NDI-redistribution feed into BirdDog Central.** The mechanism is decided (`docs/streaming-flow.md`): the source VMix node PC's native NDI output → NDI Discovery Server registration → Central receives over that node's uplink (fallback-only ~130 Mbps against the Slate AX's estimated ~150–250 Mbps Tailscale ceiling, #19, with that node's record ingest paused for the duration) → Central re-sends to the theatre PLAYS. The alternatives were researched and ruled out: Restreamer cannot emit NDI (upstream FFmpeg removed NDI in 2019 over a NewTek GPL violation; datarhei confirmed no NDI in their build and declined on licensing grounds — discussion #391), and Central cannot ingest SRT (NDI-only per its own user guide). What remains is verification, not design: confirm VMix's NDI output registers with the discovery server and Central picks it up across the tailnet; confirm Central's re-transmit actually re-originate the stream (rather than pointing receivers at the original source, which would bypass the mothership and break both the ACL model and the bandwidth math); and measure the node uplink during a rehearsed fallback against the modelled ~130 Mbps.
17. **Confirm the Cloud Gateway's exact model — LAG support and routing capacity.** Listed as a resolved assumption above, but nothing in this repo confirms it, and `docs/deployment-runbook.md` Phase 0 still treats it as a to-do. Port aggregation is only supported on the UDM Pro / SE / Pro Max, UXG Enterprise and EFG (Ubiquiti) — a literal "Cloud Gateway" (UCG Ultra/Max/Fiber) has none. Separately, every theatre packet is *routed* by this box from an uplink

VLAN onto Mothership-LAN: on a UDM Pro the LAN switch reportedly reaches the CPU over a single 1 Gbps link, capping inter-VLAN routing near 1 Gbps whatever the bond does — below the ~1,120 Mbps typical / ~1,430 Mbps high inbound at the default live scope in `docs/server-specification.md`. Check the real unit's inter-VLAN throughput, and which ports the uplink VLANs and the server land on.

18. **Finish and verify the mothership-side routing path into the tailnet, and the ACL rules for it.** Everything on the mothership that talks to a theatre (both ingest containers, Fleet Admin, Restreamer, Overseer, Flock, GLKVM-Cloud, and the BirdDog Central VM) does so through the subnet router at `192.168.1.2`. The outbound leg is now specified (gateway static routes); the rest is not yet proven:
 - **Macvlan containers can't exchange traffic with their own host** — the same isolation Unraid applies by default (the host can't reach its own macvlan containers unless "Host access to custom networks" is on). A container using `192.168.1.2` as its next hop, or the host forwarding a decrypted reply to a container, both hit it. And by default the subnet router SNATs tailnet traffic to `192.168.1.2`, so even theatre-initiated streams arrive "from the host" and the container's reply has to get back to it. Candidate fixes: Unraid's "Host access to custom networks" (check that it covers a compose-created macvlan network, not just Unraid's own), or moving the subnet router off host networking.
 - **Static routes on the Cloud Gateway** (`192.168.2-13.0/24`, `.20`, `.21` → `192.168.1.2`) are now in `config/unifi/network-config.yaml`. They solve the outbound next hop, but every packet from a container or the VM crosses the bond twice — see the NIC section of `docs/server-specification.md` — and they don't solve the *return* leg: the host delivers replies to `192.168.1.13-.24` on-link, which macvlan isolation drops unless host access is on.
 - **ACL: now names the subnets in both directions.** `config/tailscale-acl.json` previously granted only `tag:mothership → tag:theatre:*`; a tag matches the tagged nodes' own tailnet addresses, not the `/24`s they advertise, so `192.168.1.17 → 192.168.2.2` (and a theatre router registering with GLKVM-Cloud at `192.168.1.20`) would have been denied. It now lists every `/24` as a host alias, as source and destination, with theatre ranges only ever paired with the mothership range. Still to do: confirm against current Tailscale docs, decide on `--snat-subnet-routes=false` (Tailscale's site-to-site guide sets it so the far end sees real source addresses), and re-test cross-theatre isolation once built.
 - `config/docker-compose.yml`'s subnet router previously ran in the Tailscale image's default **userspace** mode, which can't forward LAN → tailnet traffic at all; it is now set to kernel mode (`TS_USERSPACE=false`).
19. **Benchmark Tailscale on one Slate AX before buying 14.** The ~150–250 Mbps combined ceiling every per-router figure in `docs/bandwidth-analysis.md` is checked against is an *estimate* extrapolated from a Beryl AX measurement — no Slate AX Tailscale benchmark was found, and GL.iNet's 550 Mbps is its kernel WireGuard figure, not Tailscale's userspace `wireguard-go` (the gap that ruled out the A-1300). Run `iperf3 --bidir` between a theatre LAN host and the mothership through Tailscale, on a **direct** path (confirm with `tailscale status / tailscale ping` — not via DERP). Target: **above ~160 Mbps combined** — the NDI-fallback-with-ingest-paused floor is ~151–166 Mbps. If it falls short, the Flint 2 (GL-MT6000) is the headroom option.
20. **Set the SRT payload size to 1128 bytes** on the VMix node PCs' SRT outputs and on Restreamer's outputs. SRT's default 1316-byte payload makes a 1360-byte IP packet, above Tailscale's 1280-byte tunnel MTU, so every SRT packet would fragment or drop; 1128 bytes (6 × 188-byte TS packets) gives a 1172-byte packet that fits — see `docs/bandwidth-analysis.md`.
21. **Confirm which ATEM Mini Extreme ISO model is on site — original or G2 — and its ISO file naming.** The ingest design assumes FTP is the only way in; check whether the model in use exposes its recording media as a network drive as well (reported for the G2), which would change the cheapest ingest path. Confirm against the unit, not the product page alone. Also confirm the ISO files carry `CAM <n>` in their names: live input selection (`ATEM_ISO_INPUTS`) depends on it, and a file that doesn't match is treated as a program recording and pulled — so a naming mismatch silently pulls every input. Override with `ATEM_ISO_INPUT_PATTERN` (`pull-iso.py`) or edit `build_filters` (`rclone-mount-rsync/mount-and-sync.sh`) if it differs.
22. **Resolved — `pull-iso.py` now recurses.** It used to list `ATEM_FTP_REMOTE_DIR` only, non-recursively, while ATEM ISO recordings are written into per-recording project folders. `pull-iso.py` now walks the tree (MLSD, falling back to

NLST + a CWD probe; up to three levels deep) and keeps the folder structure in the mirror. **Untested against a real ATEM** — check the walk and depth limit when the FTP layout is confirmed on a unit (#4).

23. **Reconcile the smart-bin path filter with the NAS layout.** The Resolve smart-bin recipes in docs/live-editing.md filter clips by record-drive path, but the ingest dual-write lands them under TheatreN/ISO/<recording folder>/ on the NAS — the filter has to match the path the ingest actually writes, not the ATEM's own drive name.
24. **Give the Unraid server a 10GbE port and an address on 192.168.22.0/24** for the edit-suite dual-write (as docs/live-editing.md and the NIC section of docs/server-specification.md require). Not yet in docs/ip-address-map.md , and the hardware isn't confirmed on the on-hand box (#1).

Alternative: plain switches + Tailscale on generic Linux, no GL-iNet routers

Same theatre topology (isolated /24 per theatre, subnet routing, Tailscale ACLs doing cross-theatre isolation) — different hardware. Instead of a GL-iNet Slate AX doing router+NAT+DHCP+Tailscale in one appliance, use a plain unmanaged switch for LAN fanout plus a small generic Linux box (mini-PC/NUC, or the theatre's existing control laptop) running Tailscale as a subnet router with NAT/DHCP built from standard Linux tools (nftables + dnsmasq). Config in `config/alt-tailscale-switch/`.

Why a router-equivalent device is still required, not eliminated: the ATEM and BirdDog Play are closed appliances — they can't run the Tailscale client themselves. Some device must still advertise the theatre's subnet on their behalf. "Pure Tailscale, no router" only works if literally every device can run Tailscale; since two can't, this alternative just moves the subnet-router role onto generic Linux instead of GL-iNet's firmware. The `tailscale up` invocations themselves are identical either way — see `config/tailscale-up-all-devices.sh` — Tailscale doesn't care what hardware runs it.

Comparison

	GL-iNet Slate AX (chosen)	Generic Linux box + switch (alternative)
Setup effort	Config file, done (see <code>config/gl-inet/</code>)	Build + harden NAT/DHCP/Tailscale from scratch per box
Throughput confidence	Vendor datasheet, ~550 Mbps — but that is kernel WireGuard; Tailscale's userspace <code>wireguard-go</code> is estimated at ~150-250 Mbps combined on the Slate AX, unbenchmarked (see <code>docs/bandwidth-analysis.md</code>). The previous choice, the A-1300, was dropped on 2026-10-06 for exactly this gap (170 Mbps kernel vs ~30-70 Mbps Tailscale est.)	Unverified until benchmarked on whatever's chosen (an x86 mini-PC will very likely beat any travel router under Tailscale)
Physical footprint	One compact unit per theatre	Separate PC + separate switch + 2 power supplies
Dedicated ATEM port	Built-in (2 LAN ports)	Needs a 2nd NIC/dongle to replicate
Wi-Fi	Built-in	Extra hardware if needed at all
Spares/swap-in	Re-flash identical UCI config	Re-provision a whole OS image
Maintenance	Vendor firmware updates	Ongoing OS patching, our responsibility
Flexibility/CPU headroom	Fixed, embedded-class	Higher, if it matters later
Unit cost at required throughput	~\$120 travel router (Slate AX list)	Often more, once a real NIC + case + PSU are added

Recommendation: GL-iNet

This is 12 identical small router/switch appliances for a touring event that need to just work — exactly the product category a travel router is built for. A generic Linux box gives more flexibility and CPU headroom we don't need here, at the cost of more setup work, more failure modes, and a bulkier kit per theatre. (A travel router's throughput "guarantee" is weaker than it looks under Tailscale — that is why the A-1300 was replaced by the Slate AX — and if the Slate AX benchmark in `docs/bandwidth-analysis.md` comes back low too, CPU headroom stops being something "we don't need" and this comparison should be revisited.) The one real advantage of the alternative — no vendor lock-in — doesn't outweigh those costs for this deployment. Keep GL-iNet as the primary; this alternative is documented for completeness, not as an equally-weighted option.

Deployment runbook

Ordered build sequence synthesizing every doc/config in this repo into one checklist. Each step links to the doc with the actual detail — this is a map, not a duplicate.

Phase 0 — Before touching anything

- ☐ **First:** on one real ATEM at the event frame rate and firmware, measure a camera ISO file, a laptop/slides ISO file and the program file (`ffprobe` , or `size ÷ duration`) against the planning figures (camera 35 / 45 / 70 Mbps, laptop 15 / 25 / 40 *estimated*, program ~10) — they set the live ISO scope (`ATEM_ISO_INPUTS`) and the 8 TB pool sizing — `docs/open-questions.md` #0
- ☐ Confirm the consolidated server's full spec (validated ECC, PCIe lanes for the NVMe pools, RAM/cores — no GPU/IOMMU checks needed any more) — `docs/open-questions.md` #1
- ☐ Confirm the Cloud Gateway model — LAG support (assumed, not confirmed) and its real inter-VLAN routing throughput — `docs/open-questions.md` #17
- ☐ Benchmark Tailscale throughput on one Slate AX (GL-AXT1800) with `iperf3 --bidir` , on a direct path (not DERP), **before buying all 14** — target above ~160 Mbps combined; the ~150–250 Mbps in the bandwidth model is an estimate — `docs/open-questions.md` #19
- ☐ Decide a real hostname/DDNS for the DERP server and confirm you can port-forward on the Cloud Gateway — `docs/tailscale.md`
- ☐ Confirm VMix's actual recording mode/bitrate and get SMB sharing set up on all 4 VMix PCs — `docs/vmix-record-ingest.md`
- ☐ Confirm ATEM FTP credentials and recording path on a real unit — `docs/atem-iso-ingest.md`
- ☐ Decide a real hostname/DDNS for GLKVM-Cloud, and note the venue's public IPv4 — `GLKVM_ACCESS_IP` takes the IP, not the hostname — `docs/glkvcloud.md`
- ☐ Confirm ATEM Overseer's, ATEM Fleet Admin's, and Flock's real container images, and whether the ATEM and BirdDog Play can actually originate their monitoring/preview streams alongside their primary feeds — `docs/open-questions.md` #11
- ☐ Settle the live-editing open questions — footage-volume sizing, the Mac mini's combined Project Server + Remote Render role, standalone cameras in scope or not — `docs/open-questions.md` #12-14

Phase 1 — Mothership networking (Cloud Gateway)

1. Apply `config/unifi/network-config.yaml` via the UniFi Network UI/API — 4 uplink VLANs, firewall baseline, LAG port profile, DERP + GLKVM-Cloud port forwards. Steps in `config/unifi/README.md` .
2. Wire the 4 VLAN uplink cable runs (3 theatres per group) — `docs/topology.md` .
3. Bond the consolidated server's 2 NICs (802.3ad/LACP) into the Cloud Gateway (or an intermediate switch if LAG isn't supported) — `docs/topology.md` , watch the LACP rate/hash-policy gotcha with Ubiquiti gear.
4. Configure and verify the gateway's **static routes** — all 14 theatre/VMix /24 s (`192.168.2-13.0/24` , `192.168.20.0/24` , `192.168.21.0/24`) via `192.168.1.2` , per the `static_routes` block in `config/unifi/network-config.yaml` — and the uplink-VLAN allow for `192.168.1.2:41641/udp` . Check the routing table in the UniFi UI lists all 14; once Phase 3 is done, `traceroute` to a theatre ATEM from a ready-room PC should show `192.168.1.1` then `192.168.1.2` . These routes are how the macvlan containers and the BirdDog Central VM reach the tailnet; that traffic crosses the server's bond twice (see `docs/server-specification.md` 's NIC section).

Phase 2 — Consolidated services server (Unraid)

1. Install Unraid (Unleashed or Lifetime licence — the pools exceed Starter's 6-device limit, see `docs/open-questions.md`). In Settings → Docker, turn on "Host access to custom networks" — the host has to hand tailnet

- replies back to the macvlan containers (docs/open-questions.md #18).
2. Create the BirdDog Central Windows VM — the design's only VM, no GPU passthrough needed — docs/topology.md . Recommended: give it its own persistent routes to the theatre /24 s via 192.168.1.2 (route -p add), so its NDI-fallback traffic doesn't hairpin through the gateway and cross the bond twice.
 3. Bring up every Docker container in one shot with config/docker-compose.yml — Nextcloud, Restreamer, NDI Discovery Server, DERP, ATEM ISO Ingest, VMix Record Ingest, UniFi Controller, GLKVM-Cloud (rttys + coturn), ATEM Overseer, ATEM Fleet Admin, Flock, Tailscale subnet router. See config/README.md for the .env setup and prerequisites first — DERP and GLKVM-Cloud specifically need their hostnames/ port-forwards from Phase 0/1 in place before they're actually useful, and ATEM Overseer/Fleet Admin/Flock need their real container images confirmed first (see docs/open-questions.md #11) — DERP and GLKVM-Cloud will start but be non-functional without their hostnames/forwards, and the three placeholder-image services won't start at all until real images are filled in. Two services aren't plain pulls: derp-server is built from source on first up , and GLKVM-Cloud needs an upstream gl-inet/glkvcloud checkout for its endpoint and templates (git clone https://github.com/gl-inet/glkvcloud.git into GLKVM_UPSTREAM).
 4. Optionally adopt the Cloud Gateway into the UniFi Controller for local management — config/unifi/README.md . Purely a local console; the network config in Phase 1 doesn't depend on it.
 5. Router registration into GLKVM-Cloud happens later, in Phase 4, once the routers themselves exist.

Phase 3 — Tailscale tailnet

1. Load config/tailscale-acl.json into the tailnet admin console — cross-theatre isolation ACLs + the self-hosted DERP region.
2. Bring up the mothership's subnet router — the tailscale-router container applies the same flags as the mothership line in config/tailscale-up-all-devices.sh itself (routes, tag:mothership , --accept-routes , port 41641), so this is just a valid TAILSCALE_AUTHKEY pre-tagged tag:mothership . Confirm the ACL allows the subnet CIDRs, not just the tags (docs/open-questions.md #18).
3. Approve routes in the admin console (or configure autoApprovers) — nothing routes until approved, regardless of what's advertised.

Phase 4 — Theatre + VMix node routers (GL-iNet Slate AX / GL-AXT1800 ×14)

1. Flash/reset all 14 units, apply Wi-Fi lockdown per venue policy (not covered by the generated configs).
2. Apply each theatre's UCI config from config/gl-inet/ — replace every REPLACE_WITH_MAC_nn with real device MACs first (config won't do anything useful otherwise). Steps in config/gl-inet/README.md .
3. Run each router's tailscale up line from config/tailscale-up-all-devices.sh .
4. Confirm the physical wiring matches the current plan: ATEM on the dedicated LAN 1 port, everything else (including BirdDog Play) via the Netgear switch on LAN 2 — docs/topology.md .
5. Register each router against GLKVM-Cloud (brought up in Phase 2) using the connection script from its web UI, run over SSH on each Slate AX — rides the tailnet already set up in step 3, no WAN exposure needed for this part — docs/glkvcloud.md .

Phase 5 — BirdDog Play + NDI discovery

1. Change the default birddog admin password on all 12 PLAY units.
2. Point BirdDog Central, all 12 PLAY units, and both VMix node PCs' NDI config at the NDI Discovery Server (192.168.1.15:5959) — BirdUI's Network panel on the PLAYS, NDI Access Manager on the Windows machines — docs/streaming-flow.md .
3. Set the SRT payload size to **1128 bytes** on every VMix node PC's SRT output and on Restreamer's outputs — the 1316-byte default doesn't fit Tailscale's 1280-byte MTU — docs/open-questions.md #20.

Phase 5b — ATEMs (all 12)

1. Patch every ATEM to the **common theatre input map** — identical on all 12: input 1 camera, 2 presenter laptop 1 (PowerPoint Main), 3 laptop 2 (VT Main / second presenter), 4–8 backups/spare — `docs/atem-iso-ingest.md`. The live ingest selects inputs by number fleet-wide, so a theatre patched differently pulls the wrong source.
2. Set each ATEM's **Streaming/record quality** so the program recording is **~8–10 Mbps** (ISO bitrates are fixed by frame rate and can't be set). Check one recorded program file's bitrate.
3. On one unit, record a short test and confirm the FTP folder layout and that ISO files carry `CAM <n>` in their names — `docs/open-questions.md` #21, #22.

Phase 6 — Nextcloud external storage + ingest pipelines

1. Run `config/atem-iso-ingest/setup-nextcloud-external-storage.sh` and `config/vmix-record-ingest/setup-nextcloud-external-storage.sh` — mounts the External Storage folders and prints the cron lines for targeted rescans. Add those cron entries.
2. Fill in real credentials, set **ATEM_ISO_INPUTS** in `.env` (default `1` = camera + program; `1,2` / `1,2,3` add the laptop inputs, for all 12 theatres at once, only if router and mothership-NIC headroom allow — see `docs/atem-iso-ingest.md` § Bandwidth), and start `pull-iso.py` (or the `rclone-mount-rsync/` alternative, which reads the same variable) — `config/atem-iso-ingest/README.md`. Inputs not pulled live stay on each ATEM's SSD — schedule the physical DIT offload of those drives (`docs/live-editing.md`).
3. Fill in real SMB credentials/share names and start `mount-and-sync.sh` — `config/vmix-record-ingest/README.md`.
4. Configure Nextcloud version-retention for both ingest folders — both ingest docs flag this as worth tuning once running for real.
5. **Dual-write caveat:** the second write destination (the edit-suite NAS, see `docs/live-editing.md`) is documented but **not yet implemented** in either ingest script — implement it or consciously defer it before the event (`docs/open-questions.md` #15). As shipped, both scripts write only the Nextcloud destination.

Phase 6b — Live editing subsystem

The edit suite (`docs/live-editing.md`) is its own 10GbE LAN, off the tailnet, so it can be built any time after Phase 2 — it only depends on the ingest pipelines (Phase 6) for its footage feed:

1. Stand up the `192.168.22.0/24` 10GbE LAN — switch, edit-suite NAS (`.22.2`), routed connection back to the mothership LAN for the dual-write (no Tailscale).
2. Set up the Mac mini (`.22.20`): install DaVinci Resolve Studio + the Project Server app, create the shared project library, enable it as a Remote Render node.
3. Connect both MacBook Pros (`.22.11` / `.22.12`) via Thunderbolt-to-10GbE, mount the NAS on all three machines with identical paths (Resolve's Remote Render requires the media volume mounted on every machine).
4. Run `resolve-configurator` against the event's session CSV to build the shell project; apply each smart-bin recipe once, by hand, in Resolve's UI.

Phase 7 — Verify before the event

- ☐ `tailscale status` / `tailscale ping` from a few nodes — confirm direct vs relay — `docs/tailscale.md`
- ☐ `tailscale netcheck` on a theatre router — confirm the self-hosted DERP region is reachable and not blocked by the uplink-VLAN firewall rule — `docs/open-questions.md` #3
- ☐ Copy an ATEM file mid-recording and confirm it opens/scrubs — empirically verify the "safe to read mid-write" assumption — `docs/atem-iso-ingest.md`
- ☐ From inside a macvlan container, reach a theatre device and get an answer back — e.g. `docker exec atem-iso-ingest python -c "import socket; socket.create_connection(('192.168.2.2', 21), 5)"` — proves the gateway static route, the host return leg and the subnet ACLs together (`docs/open-questions.md` #18)
- ☐ Confirm ingested files actually appear in Nextcloud's UI (not just on disk) for both pipelines

- ☐ Confirm cross-theatre isolation: from one theatre's subnet, verify you cannot reach another theatre's devices, only the mothership — this is the core ACL guarantee the whole design depends on (`config/tailscale-acl.json`)
- ☐ From off-venue (a network that isn't the mothership's own), confirm the GLKVM-Cloud web UI is actually reachable through `https://kvm.example.net:8443` , and that all 14 routers show up registered and their SSH terminal/web-proxy access works — `docs/glkvm-cloud.md`
- ☐ Confirm all 12 theatres' monitoring/preview streams actually reach ATEM Overseer and Flock and show up live in each dashboard — this is the first real-hardware test of the dual-stream assumption flagged in `docs/open-questions.md` #11
- ☐ Live editing: confirm footage lands on the edit-suite NAS as it's ingested (once the dual-write is implemented — see Phase 6 step 5), both editors can open the shared show project simultaneously, the smart bins auto-fill as clips arrive, and a test export dispatched to the Mac mini via Remote Render completes — `docs/live-editing.md`
- ☐ Rehearse an NDI fallback end-to-end: pause the source node's VMix record ingest, confirm the node's VMix NDI output appears via the Discovery Server, confirm BirdDog Central picks it up and **re-originates** it (the PLAY must connect to Central, not directly to the node — direct would bypass the ACL model and the bandwidth math), switch one theatre's PLAY over via Flock, and measure the node uplink against the modelled ~130 Mbps — `docs/streaming-flow.md` , `docs/open-questions.md` #16

Troubleshooting

Known gotchas surfaced during design — check here before assuming something's broken.

"Theatre can't reach the mothership at all"

- **Routes not approved.** Advertising a route with `--advertise-routes` isn't enough — it must be approved in the Tailscale admin console (or via `autoApprovers`). Check first; this is the single most common reason a fresh subnet router doesn't work.
- **Check `tailscale status`** on the theatre's Slate AX — confirms whether the peer connection is up at all before debugging further.

"Two theatres can talk to each other" (should be impossible)

- This should never happen if `config/tailscale-acl.json` is loaded correctly — theatres can only reach `tag:mothership`. Confirm the ACL is actually applied in the admin console, not just present in this repo.
- Remember: physical VLAN grouping provides **zero** isolation on its own — it's cabling organization only. If ACLs are missing/misconfigured, cross-theatre reachability is the default, not a bug in the VLANs.

"Tailscale is using DERP relay instead of direct"

- Expected sometimes — see `docs/tailscale.md` for why direct connections are likely but not guaranteed here (depends on whether the Cloud Gateway allows inter-VLAN UDP).
- Run `tailscale netcheck` to see which DERP region is preferred. If it's not the self-hosted `example` region, check: is `derp.example.net` actually resolving? Is the WAN port forward (443/tcp, 3478/udp) actually in place? Neither works until both exist — see `docs/open-questions.md`.
- If the self-hosted DERP is reachable from the mothership's own LAN but not from a theatre, suspect the uplink-VLAN firewall block in `config/unifi/network-config.yaml` catching the hairpin path — see the caveat attached to that rule.

"The NIC bond won't form / only shows one active link"

- **Bond mode / LACP rate.** Unraid's bonding default mode is not 802.3ad — set it explicitly. Ubiquiti gear uses LACP rate `fast`; Unraid's (Linux's) default is `slow`. The standard allows the two ends to differ, so a mismatch usually slows failure detection rather than stopping the bond forming, but set both to `fast` to rule it out — see `docs/topology.md`. (Hash policy is a per-end transmit choice; a mismatch there won't stop the bond forming, it just affects how evenly each direction spreads.)
- **The gateway doesn't support LAG at all.** Only the UDM Pro / SE / Pro Max, UXG Enterprise and EFG do; the Cloud Gateway Ultra/Max/Fiber don't. If so, bond through an intermediate LACP-capable switch instead — already the agreed fallback, not a new problem (`docs/open-questions.md` #17).

"Mothership containers can't reach a theatre (ingest stalls, Fleet Admin can't connect)"

- **Check the path hop by hop.** From another LAN machine, `traceroute 192.168.2.2` should go via `192.168.1.1` then `192.168.1.2`; if it stops at the gateway, the static routes in `config/unifi/network-config.yaml` are missing.

- **The return leg.** The Unraid host can't hand packets to its own macvlan containers unless "Host access to custom networks" (or an equivalent shim) covers the compose network — symptoms are SYNs leaving and nothing coming back. See `docs/open-questions.md #18`.
- **ACL.** Tag-based rules don't cover advertised subnets; `tailscale ping` between the routers working while container traffic fails points here (same #18).
- **Subnet router in userspace mode.** `docker exec tailscale-router ip link show tailscale0` should show the interface; if it doesn't, `TS_USERSPACE=false` isn't in effect.

"BirdDog Play isn't finding the NDI backup source"

- Confirm the Discovery Server IP (`192.168.1.15`) is actually configured on *both* ends — BirdUI's Network panel on every PLAY unit, and NDI Access Manager on the BirdDog Central VM (a Windows app, so no BirdUI) and the VMix node PCs. One-sided config does nothing.
- Plain mDNS will never work across theatres; if Discovery Server isn't configured, this is expected behavior, not a fault. See `docs/streaming-flow.md`.

"Ingested files (ATEM or VMix) aren't showing up in Nextcloud"

- Writing to the folder isn't enough — Nextcloud needs `occ files:scan` to index it. Check the cron job from `setup-nextcloud-external-storage.sh` is actually installed and running, not just printed once during setup.
- If files appear but seem stuck at an old size, check the ingest container's logs — a failed FTP/SMB reconnect will silently stall the pull loop rather than crash it.

"Ingest is falling behind / theatre uplink saturated"

- Confirm the real ATEM bitrate/active-channel count and VMix recording settings — the bandwidth math in `docs/atem-iso-ingest.md` and `docs/vmix-record-ingest.md` assumes figures that were never verified against real hardware.
- If using the `rcclone-mount-rsync` alternative, confirm `rsync --append` is actually being used (not falling back to a full re-copy) — check `--itemize-changes` output.

"A copied ATEM/VMix file won't play"

- Confirmed only "very likely" safe to read mid-write, never empirically tested — see the open item in both ingest docs. If it fails, the safest fallback is to only pull once recording has fully stopped (loses the near-real-time property, but guarantees a valid file).

"ATEM Overseer / Flock isn't showing a theatre's live preview"

- Confirm the ATEM/BirdDog Play in that theatre is actually configured to originate its second monitoring/preview stream, not just its primary program feed — this is a genuinely unconfirmed capability, not assumed working out of the box, see `docs/open-questions.md #11`.
- Check the stream is actually reaching the container's IP (`192.168.1.21` for Overseer, `192.168.1.24` for Flock — see `docs/ip-address-map.md`) over the tailnet, same routing path as every other theatre-to-mothership stream in this design.
- If every other theatre works but one doesn't, suspect that theatre's own Slate AX hitting its combined-load ceiling (Tailscale throughput estimated ~150-250 Mbps, unbenchmarked — check `ATEM_ISO_INPUTS` for laptop ISOs enabled on that theatre) rather than anything Overseer/Flock-side — see the per-router finding in `docs/bandwidth-analysis.md`.

"GL-iNet static DHCP leases aren't applying"

- Every `REPLACE_WITH_MAC_nn` placeholder in `config/gl-inet/` must be swapped for the device's real MAC first — the generated configs ship with placeholders deliberately, not real defaults.